Index

1. Module 1 : Propositional Logic
   - Syntax and Parse Trees
   - Semantics, Equivalence, Satisfiability and Validity
   - Natural Deduction as a Proof System
   - Normal Forms
2. Module 2 : Satisfiability
   - Resolution Rule
   - DPLL and CDCL
3. Module 3 : Predicate Logic
   - Syntax and Semantics
   - Natural Deduction
   - Soundness, Completeness and Expressibility
   - Normal Forms and Satisfiability Modulo Theories
4. Module 4 : Hoare Logic
   - Syntax and Proof Calculus
   - Total Correctness
5. Module 5 : Temporal Logic
   - Models and Temporal Connectives
   - LTL Syntax and Semantics
   - CTL Syntax and Semantics
   - Model Checking and Conclusion

CS228 : Logic for Computer Science

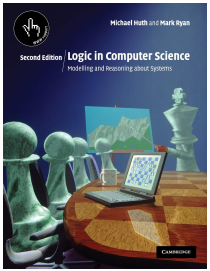
Lecture 1: First Module on Logic and Reasoning

Instructor: S. Akshay

IIT Bombay, India

Logistics

- Course webpage: `https://www.cse.iitb.ac.in/~akshayss/courses/cs228`
- Piazza invites sent!
- Textbook: For the first part of the course, we will use Huth and Ryan's book on Logic in Computer Science.



Module 1: Logic, the language/calculus for reasoning

A whole zoo of symbolic logics out there!

We will go from simple to more complicated

- Propositional Logic
- Predicate Logic
  - First order logic
  - Monadic second order logic
  - Higher order logics
- Temporal logics
- Modal logics

Topic 1.1

Propositional logic: Syntax

An example beyond good and evil

In a country, every person is either good or evil. Good people always tell the truth and evil people always lie.

You meet two citizens A and B. A says, "I am evil or B is good".
What are A and B? **Exercise 1.1: Work this out! How do we find all possibilities?**

This lecture: How to formalize this reasoning?

- Step 1: Introduce symbols!
  - Let p_A denote the statement: A is good. It is either true or false.
  - Similarly, let p_B denote "B is good."
- Step 2: Encode puzzle using the symbols: $\underbrace{(\neg p_A \vee p_B)}_{\text{Statement of A}} \Leftrightarrow p_A$
- Step 3: Solve the puzzle: Assign truth values to p_A and p_B such that the above "formula" holds. (how? trial and error?)

From English to Math

So in this example we went from an English puzzle to a symbolic logic problem.

What does a language have?

- Symbols: good, A, B, is, not
- Syntax: Grammar
- Semantics: Meaning

But any natural language has so much ambiguity!

"I saw the man with the telescope."

A Symbolic logic

The logic is over a list of propositions. Some examples:

- A is good. p_A
- This tree is green q
- The fridge is broken r
- ... *many more*
  - A proposition: any sentence for which we can answer if it is True or False.
  - We do not care what each one says: we can give them a symbol.
  - Not a proposition: "May the force be with you!"

Exercise 1.2

- Give two examples of sentences that are not propositions.
- Give three examples of propositions.

Propositions as Boolean variables

We start with a set **AP** of finite or countably many (atomic) propositions.

- Since **AP** is countable, we assume that they are indexed.

$$\mathbf{AP} = \{p_1, p_2, \dots\}$$

- These are just names/symbols without inherent meaning
- We may also use $p, q, r, .., x, y, z$ to denote them
- Propositions are Boolean variables (as they take true or false as value!)

Topic 1.2

Propositional Formulas

Logical connectives to build propositions

Starting from "atomic" propositions, we can combine them as follows:

Building new propositions from existing ones

- a statement that says negation of another, e.g., A is not good.
- two statements are true together, e.g., A is good & it is raining.
- at least one of the two statements are true, e.g., A is good or it is raining.
- Implication: if A is true then B is also true, e.g., If it rains then trees will grow.
- Equivalence: truth value of two statements are same, e.g., water is blue if and only if sky is black.

We need True and False also!

- always true: e.g., If A is good, then A is good.
- always false: e.g., If A is good, then A is bad.

Logical connectives

formal name	symbol	read as	
true	$\top$	top	0-ary symbols
false	$\bot$	bot	
negation	$\neg$	not	unary symbols
conjunction	$\wedge$	and	binary symbols
disjunction	$\vee$	or	
implication	$\rightarrow$	implies	
equivalence	$\leftrightarrow$	iff	
exclusive or	$\oplus$	xor	
open parenthesis	(		punctuation
close parenthesis	)		

- We assume that the logical connectives are not in **AP**.
- Do we need all these connectives? Which can be derived from others?

Propositional formulas

A propositional formula is a finite string containing symbols in **AP** and logical connectives. The set of all propositional formulas over **AP** is denoted **Prop**. Well, not every string...

Formal Definition (Note! This is an inductive definition)

The set of propositional formulas is the smallest set **Prop** such that

- $\top, \bot \in$ **Prop**
- if $p \in$ **AP** then $p \in$ **Prop**
- if $F \in$ **Prop** then $\neg F \in$ **Prop**
- if $\circ$ is a binary symbol and $F, G \in$ **Prop** then $(F \circ G) \in$ **Prop**

Propositional formulas

Another way to write this definition . This is called the Backus Naur Form.

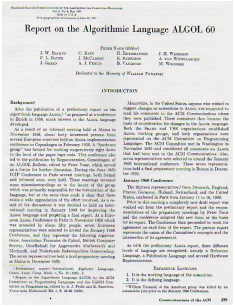
$F \in$ **Prop** if

$$F ::= p \mid \top \mid \bot \mid \neg F \mid (F \vee F) \mid (F \wedge F) \mid (F \to F) \mid (F \leftrightarrow F) \mid (F \oplus F)$$

where $p \in$ **AP**. Any such formula is also called well-formed.

A bit of history

Named after John Backus and Peter Naur, who used it to formally specify the grammar of ALGOL 60 in 1963 – one of the earliest programming languages, and ancestor of Pascal, C, and more!



Report on the Algorithmic Language ALGOL 60

INTRODUCTION

Examples of propositional formulas

Exercise 1.3 Which of the following belong to **Prop** (i.e., are well-formed)?

1. $\top \rightarrow \bot$
2. $(\top \rightarrow \bot)$
3. $(p_1 \rightarrow \neg p_2)$
4. (p_1)
5. $\neg\neg\neg\neg\neg\neg\neg p_1$

Obs: Not all strings over **AP** and logical connectives are in **Prop**.

How can we argue that a string does or does not belong to **Prop**?

The Lady Or the Tiger

A prisoner must choose between three rooms. One room has a Lady and other two have tigers. If the prisoner chooses the room with the Lady, he wins! If he chooses a wrong room, he (probably) gets eaten by the tiger!

Each room has a sign in front of it. BUT Only one of the signs is true; the other two are false.

(**Room1**) *The Lady is not in this room*

(**Room2**) *The Lady is not in this room*

(**Room3**) *The Lady is in Room 2*

Which room has the Lady?!

Exercise 1.4

1. Solve the above puzzle!
2. Write each of the above statements in propositional logic.
3. Reduce the problem to a question in this logic!

Formalizing Exercise 1.4

- Propositions $S1, S2, S3$ represent whether the sign of room 1, 2, 3 (resp.) is true.
- Propositions $L1, L2, L3$ represent Lady in room 1, 2, 3 resp.
- The statements given:
  - $S1 \leftrightarrow \neg L1$, $S2 \leftrightarrow \neg L2$, $S3 \leftrightarrow L2$.
  - $(L1 \wedge \neg L2 \wedge \neg L3) \vee (L2 \wedge \neg L1 \wedge \neg L3) \vee (L3 \wedge \neg L1 \wedge \neg L2)$.
  - $(S1 \wedge \neg S2 \wedge \neg S3) \vee (S2 \wedge \neg S1 \wedge \neg S3) \vee (S3 \wedge \neg S1 \wedge \neg S2)$.
- Is there a way to give truth values to propositions such that all these constraints are satisfied?
- **Bonus:** Can you encode this using only 3 propositions instead of 6?

Bonus: Encoding with only 3 propositions

Intuition

$S1, S2, S3$ were never really needed – they're just names for what the signs say. We can substitute their definitions directly wherever they're used.

- exactly one of $\neg L1,\ \neg L2,\ L2$ holds (exactly one sign is true)

$$(\neg L1 \wedge L2 \wedge \neg L2) \vee (L1 \wedge \neg L2 \wedge \neg L2) \vee (L1 \wedge L2 \wedge L2)$$

- exactly one of $L1, L2, L3$ holds (Lady is in exactly one room)

$$(L1 \wedge \neg L2 \wedge \neg L3) \vee (L2 \wedge \neg L1 \wedge \neg L3) \vee (L3 \wedge \neg L1 \wedge \neg L2)$$

Another Bonus: Can we encode it with just two propositions?

CS228 : Logic for Computer Science

Module 1: Propositional Logic

Lecture 2: Parse Trees, Semantics and Problems of Interest

Instructor: S. Akshay

IIT Bombay, India

Logistics

Updates

- Safe should be up and running
- Piazza is up and running - many interesting questions already!
- Problemsheets: This Thursday ...
- Optional Help Session: For a few students, already contacted!

Module 1: Logic, the language/calculus for reasoning

A whole zoo of symbolic logics out there!

We will go from simple to more complicated

- Propositional Logic
- Predicate Logic
- Temporal logics
- Modal logics

Recap: Propositional Logic

1. What are propositions?
2. Atomic propositions and how to combine them
3. Syntax of propositional logic: Propositional formulae

Recall: Propositional formulas

Is this a proposition?

It is raining.

Exercise 1.6: Convert these to propositional logic

1. If it rains, you will get wet.
2. If it does not rain, you will not get wet.
3. If it rains, you will get wet unless you have an umbrella.
4. If it rains, you will get wet or you won't get wet.

Topic 2.1

Parse Trees

Representing Propositional Formulae

A propositional formula is a finite string containing symbols in **AP** and logical connectives.

$F \in$ **Prop** if

$$F ::= p \mid \top \mid \bot \mid \neg F \mid (F \vee F) \mid (F \wedge F) \mid (F \rightarrow F) \mid (F \leftrightarrow F) \mid (F \oplus F)$$

where $p \in$ **AP**.

Representation of Propositional formulas as strings

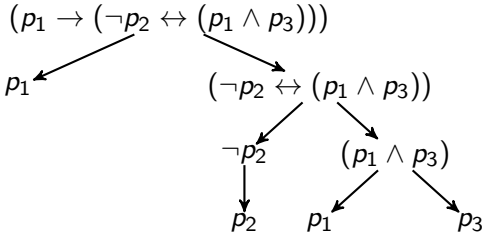
- Any $F \in$ **Prop** is a string over **AP** and logical connectives
- Two issues:
  1. Not all strings are formulas. e.g., $(\neg)()pq \rightarrow$
  2. If we don't bracket properly, we cannot parse it uniquely!
     - A partial solution : Preference order $\neg$ then $\vee, \wedge$ then $\rightarrow, \leftrightarrow$. So $\neg p \vee q \rightarrow r$ is to be read as $((\neg p \vee q) \rightarrow r)$.
- A better solution: Tree representation!

Representation as Parse trees

Definition: A parse tree of a formula $F \in$ **Prop** is a tree s.t.,

- the root is F,
- leaves are atomic propositions, and
- each internal node is formed by applying some logical connective to its children.

Example



Theorem

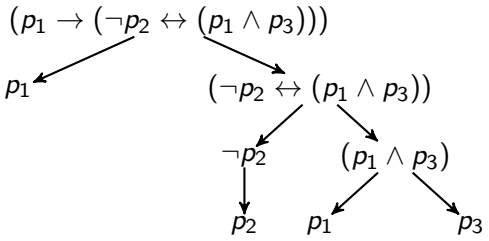
$F \in$ **Prop** iff F has a parse tree

Representation as Parse trees

Definition: A parse tree of a formula $F \in$ **Prop** is a tree s.t.,

- the root is F,
- leaves are atomic propositions, and
- each internal node is formed by applying some logical connective to its children.

Example



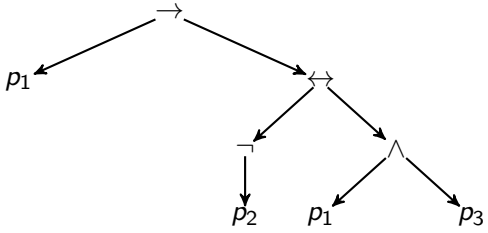
- Therefore, writing the tree allows to check that formula is well-formed.
- Formulas on internal nodes and leaves are called sub-formulas.

Representation as Parse trees

Definition: A parse tree of a formula $F \in$ **Prop** is a tree s.t.,

- the root is F,
- leaves are atomic propositions, and
- each internal node is formed by applying some logical connective to its children.

Example



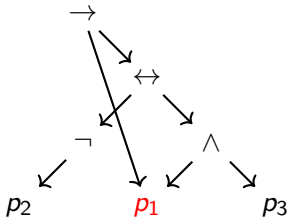
- Therefore, writing the tree allows to check that formula is well-formed.
- Formulas on internal nodes and leaves are called sub-formulas.
- Don't need brackets in the tree!

Representation as Parse trees / DAGs

Definition: A parse tree of a formula $F \in$ **Prop** is a tree s.t.,

- the root is F,
- leaves are atomic propositions, and
- each internal node is formed by applying some logical connective to its children.

Example



- Therefore, writing the tree allows to check that formula is well-formed.
- Formulas on internal nodes and leaves are called sub-formulas.
- Share leaves to get a directed-acyclic graph (DAG) instead of Tree!

Some exercises

Exercise 2.1

- Are these formulas in **Prop**? If so, draw the parse trees and dags for the formulas below:
  1. $(p \wedge (\neg q \rightarrow \neg p))$
  2. $\neg((\neg q \wedge (p \rightarrow r)) \wedge (r \rightarrow q))$
  3. $(s \vee ((\neg p) \rightarrow (\neg p)))$
  4. $(p \rightarrow (\neg q \wedge r \vee \neg p))$

Where are we?

- So far: Syntax — which strings of symbols count as valid formulas.
- So far: Parse Trees — how to represent a formula's structure (and share sub-formulas via DAGs).
- Next: Semantics — given a well-formed formula, what does it actually *mean*? How do we compute its truth value?

Topic 2.2

Semantics

Semantics

What is the meaning of a formula?

- We start with truth values $\mathcal{B} := \{0, 1\}$, where we interpret 0 as False and 1 as True.
- Each atomic proposition $p \in \mathbf{AP}$ is either True or False.
  e.g., "It is raining" is True right now (monsoon season!).
- Each propositional formula $F \in \mathbf{Prop}$ is built from atomic propositions
  e.g., "It is raining $\wedge$ I have an umbrella" is built from two atomic propositions.
- So "assigning" truth values to **AP** gives "value" for every $F \in \mathbf{Prop}$.
  e.g., if both of the above are True, the formula "It is raining $\wedge$ I have an umbrella" is True too.

Definition

- An *assignment* is a map from $\mathbf{AP} \to \mathcal{B}$.
- A *Valuation or a model m* of a propositional formula $F \in \mathbf{Prop}$ is an assignment of each atomic proposition in F to a truth value.

Summarizing the truth values: Truth Tables

- For each assignment of variables, i.e., propositions, we obtain a value of a formula F.
- If we write down the values for ALL possible assignments, we get the "truth table".
- Thus, the semantics of $F \in$ **Prop**, denoted $[|F|]$, is a map from $\{0,1\}^{\mathbf{AP}} \to \{0,1\}$.

What about $G \to H$?

G	H	$G \to H$
1	1	1
1	0	0
0	1	1
0	0	1

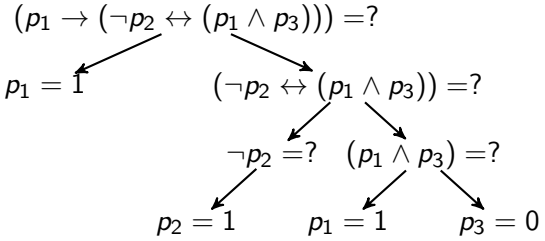
- **Exercise 2.2:** Write down the truth tables for $\vee$, $\neg$, $\leftrightarrow$, $\oplus$.
- Note that Truth table for True is just $\begin{array}{|c|}\hline \top \\ \hline 1 \\ \hline \end{array}$ and False is $\begin{array}{|c|}\hline \bot \\ \hline 0 \\ \hline \end{array}$.

How do we generalize this for an arbitrary formula F?

Evaluating a formula using its Parse tree

Question

If $p_1 = 1$, $p_2 = 1$, $p_3 = 0$, what is $(p_1 \rightarrow (\neg p_2 \leftrightarrow (p_1 \wedge p_3)))$?



- For a given formula F and a given assignment, how long does it take to evaluate it? A. Linear time in size of the tree!
- But what is the size of the tree in worst case?

Semantics of Propositional formula as Truth Tables

Consider: $(p_1 \rightarrow (\neg p_2 \leftrightarrow (p_1 \wedge p_3)))$

p_1	p_2	p_3	$(p_1$	$\rightarrow$	$(\neg$	p_2	$\leftrightarrow ($	p_1	$\wedge$	p_3 $)))$
0	0	0	0	1	1	0	0	0	0	0
0	0	1	0	1	1	0	0	0	0	1
0	1	0	0	1	0	1	1	0	0	0
0	1	1	0	1	0	1	1	0	0	1
1	0	0	1	0	1	0	0	1	0	0
1	0	1	1	1	1	0	1	1	1	1
1	1	0	1	1	0	1	1	1	0	0
1	1	1	1	0	0	1	0	1	1	1

- Can think of it as going up the Parse tree.
- So we are applying an inductive evaluation!
  - $[|F \wedge G|] = 1$ iff $[|F|] = 1$ and $[|G|] = 1$
  - $[|\neg G|] = 1$ iff $[|G|] = 0$
  - Complete the rest!

Back to the example beyond good and evil

- In a country, every person is either good or evil (i.e., NOT good).
- Good people always tell the truth and evil people always lie.
- You meet two citizens A and B. A says, "I am evil or B is good". What are A and B?

Using propositional logic

- Step 1: Introduce atomic propositions: p_A= A is good, p_B= B is good.
- Step 2: Encode puzzle using the propositional formula: $F = \underbrace{(\neg p_A \vee p_B)}_{\text{Statement of A}} \leftrightarrow p_A$

Exercise 2.3

Write out the Truth Table for F

Back to the example beyond good and evil

- In a country, every person is either good or evil (i.e., NOT good).
- Good people always tell the truth and evil people always lie.
- You meet two citizens A and B. A says, "I am evil or B is good". What are A and B?

Using propositional logic

- Step 1: Introduce atomic propositions: p_A= A is good, p_B= B is good.
- Step 2: Encode puzzle using the propositional formula: $F = \underbrace{(\neg p_A \vee p_B)}_{\text{Statement of A}} \leftrightarrow p_A$
- Step 3: Solve the puzzle: Is there a unique assignment of truth values to p_A and p_B under which $[|F|] = 1$? How do we do this? Check for 1 in Truth Table!

p_A	p_B	$\neg p_A$	$\neg p_A \vee p_B$	F
0	0	1	1	0
0	1	1	1	0
1	0	0	0	0
1	1	0	1	1

More examples

A prisoner must choose between three rooms. One room has a Lady and other two have tigers. If the prisoner chooses the room with the Lady, he wins! If he chooses a wrong room, he (probably) gets eaten by the tiger!

Each room has a sign in front of it. BUT Only one of the signs is true; the other two are false.

(**Room1**) *The Lady is not in this room*

(**Room2**) *The Lady is not in this room*

(**Room3**) *The Lady is in Room 2*

Which room has the Lady?!

Exercise 2.4

1. Take the encoding of this problem you had written last lecture. Write the Truth Table for that formula.
2. Use the Truth Table to solve the problem.

Topic 2.3

Problems of Interest

Satisfiability

Definition (Satisfiability)

$F \in$ **Prop** is said to be satisfiable if

- there exists a valuation m of F such that $m(F) = 1$, i.e., $[|F|] = 1$.
- in other words, we say F has a *satisfying* assignment.

Examples

- Country with Good and Bad people example
- Lady and the Tiger example
- Many many more!

How did we solve it?Check for 1 entry.

Validity

Definition: $F \in \mathbf{Prop}$ is said to be valid if

- for all valuations m of F, we have $m(F) = 1$ i.e., $[|F|] = 1$.
- Valid formulas are also called tautologies.
- Can we read it from the Truth Table?! Is $G = (p_1 \rightarrow (\neg p_2 \leftrightarrow (p_1 \wedge p_3)))$ valid?
- Just check if all entries in col for G are 1.

p_1	p_2	p_3	G
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Link between Satisfiability and Validity

What is the link between Satisfiability and Validity?

Theorem

Let $F \in \textbf{Prop}$. F is satisfiable iff $\neg F$ is not valid.

Proof:

- ($\Longrightarrow$) Suppose F is sat. By defn, $\exists$ valuation m of F such that $m(F) = 1$, but then $m(\neg F) = 0$. That is, $\neg F$ is not valid.
- ($\Longleftarrow$) Conversely, suppose $\neg F$ is not valid. There must exist valuation m' of $\neg F$ such that $m'(\neg F) = 0$. Then $m'(F) = 1$, which implies that F is satisfiable.

In other words, if you have an algorithm to check for Satisfiability of propositional formulas, you can use it to check for validity of formulas and vice versa!

Semantic Entailment

Definition: Consider propositional formulas F, G over AP.

Whenever F evaluates to 1, G also evaluates to 1 — we say, $F \models G$.
That is, for all assignments m of **AP**, if $m(F) = 1$ then $m(G) = 1$.

Exercise 2.5: Do the following hold? Write True/False

- $p \wedge q \models p$
- $p \vee q \models q$

Definition (Semantic Equivalence)

We say that F and G are semantically equivalent, denoted $F \equiv G$, if $F \models G$ and $G \models F$.
Example: $(p \rightarrow q) \equiv (\neg p \vee q)$.

CS228 : Logic for Computer Science

Module 1: Propositional Logic

Lecture 3: Formal Proofs and Natural Deduction

Instructor: S. Akshay

IIT Bombay, India

Recap

Propositional Logic

- Syntax
  - Atomic propositions
  - Propositional formula
  - Parse trees
- Semantics
  - Truth tables

Examples of puzzles encoded into Propositional logic.

Problems of interest

- Satisfiability and validity
  - Does there exist a model/satisfying assignment?
- Semantic entailment and equivalence

Semantic Entailment and Equivalence (Recap)

Definition: Consider propositional formulas F, G over $\mathbf{AP}$.

Whenever F evaluates to 1, G also evaluates to 1 — we say, $F \models G$.
That is, for all assignments m of $\mathbf{AP}$, if $m(F) = 1$ then $m(G) = 1$.

Definition (Semantic Equivalence)

We say that F and G are semantically equivalent, denoted $F \equiv G$, if $F \models G$ and $G \models F$.
That is, for all assignments m of $\mathbf{AP}$, $m(F) = 1$ if and only if $m(G) = 1$.
Example: $(p \rightarrow q) \equiv (\neg p \vee q)$.

- How do we check semantic entailment and equivalence using truth tables?
  - If two formulas have same truth table, then they are equivalent!
  - What about the converse? Are there two equivalent formulas that have different truth tables?

Semantic equivalence can be used to simplify formulas

- $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$ (Associativity)
- $(p \oplus q) \oplus r \equiv p \oplus (q \oplus r)$ (Associativity)
- $(p \wedge q) \vee r \equiv (p \vee r) \wedge (q \vee r)$ (Distributivity)

p	q	r	$p \wedge q$	$(p \wedge q) \wedge r$	$q \wedge r$	$p \wedge (q \wedge r)$
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	1	0
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	1	0	0	0
1	1	1	1	1	1	1

Same in every row $\Rightarrow$ the two formulas are semantically equivalent!

Pop Quiz!

Question 1 (1 mark)

Let $p =$ "you attend the lectures", $q =$ "you fall asleep", $r =$ "you will pass the course".
Write a propositional formula for: "If you attend the lectures, and you don't fall asleep, then you will pass the course."

Question 2 (2 marks)

Show, using truth tables, that $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$ (Distributivity).

Question 3 (3 marks)

Consider the formula $\varphi = (p \to q) \wedge (q \to r) \to (p \to r)$.

(i) Construct its truth table.

(ii) Is φ valid, i.e., does it evaluate to true under all assignments?

(iii) Is $\neg\varphi$ satisfiable and/or valid?

An aside: Minimal logical connectives

We used 2 0-ary, 1 unary, and 5 binary connectives to describe the propositional logic.

However, it is not the minimal set needed for the maximum expressivity.

Exercise 2.8

Show that $\neg$ and $\vee$ can define the whole propositional logic. That is, show the following:

1. $\top \equiv p \vee \neg p$ for some $p \in \mathbf{AP}$
2. $\bot \equiv \neg\top$
3. $(p \wedge q) \equiv \neg(\neg p \vee \neg q)$
4. $(p \oplus q) \equiv (p \wedge \neg q) \vee (\neg p \wedge q)$
5. $(p \to q) \equiv (\neg p \vee q)$
6. $(p \leftrightarrow q) \equiv (p \to q) \wedge (q \to p)$

Similarly, show that $\neg$ and $\wedge$ can define all the other connectives.

Exercise 2.9

A universal connective is a logical connective, that can alone define all of propositional logic.

1. Give an example of a universal connective. Are there several universal connectives?
2. * Show $\oplus$ is not a universal connective!

How can we show that $\oplus$ is not universal?

We will show that $\oplus$ alone cannot define $\neg$.We start with a property about $\oplus$-only formulas.

Lemma

Any formula φ built only from $\oplus$ and propositional variables (no constants) evaluates to false whenever all its variables are false.

Proof (by structural induction on the formula)

- Base case: $\varphi = p$. Under the all-false valuation, φ evaluates to 0.
- Inductive step: $\varphi = \varphi_1 \oplus \varphi_2$. By IH, both φ_1 and φ_2 evaluate to 0 under the all-false valuation, so $\varphi_1 \oplus \varphi_2 = 0 \oplus 0 = 0$.

Now, we argue by contradiction.

Suppose some $\oplus$-only formula $\varphi(p, q_1, \ldots, q_n)$ satisfied $\varphi \equiv \neg p$. Set p and all q_i to false: by Lemma, φ evaluates to 0, but $\neg p$ evaluates to 1 when $p = 0$. Contradiction! □

Other problems of interest

- Uniqueness: Does a given formula have a unique satisfying assignment?
- Counting: How many satisfying assignments does the formula have?
  - Decision Variant: Given Formula F and number k, is the number of satisfying assignments of F more than k?

All these can be answered from the truth table

So what's the big problem?

The problem with truth tables

- If the formula has 100 variables (typically encodings that are useful in practice require this!)
- Then, number of lines of truth table is 2^{100} ($\sim 10^{30}$).
- More than all the digital storage on Earth!
- So, this approach doesn't work for anything except toy examples...

So what can we do?

Solution:

1. Formal proof systems
2. Knowledge compilation

Topic 3.1

A Formal Proof system

How do we reason with formulas?

Reasoning about formulas via Truth Tables is not always practical

But do we really need the full Truth Table always?

- Is $((p \vee q) \rightarrow (q \wedge r)) \wedge (\neg((p \vee q) \rightarrow (q \wedge r)))$ satisfiable?
- Does $p \wedge q \models p$ hold?
- If q is true then $p \vee q$ is also true.
- If I have my umbrella open, and I only open it when it's raining, then it is raining.

For all these questions, do we need to write Truth tables?

We use rules between propositions and connectives!

Can we formalize them?

A proof system

Our goal is to define rules so that we can write a proof!

Definition: Sequent

Suppose we have

- Formulas $F_1, \ldots F_n$: call them premises
- Another formula G: call it conclusion
- We write

$$F_1, \ldots, F_n \vdash G$$

to express the *claim* that, starting from $F_1, \ldots F_n$, by applying the "rules" repeatedly, one can derive G.

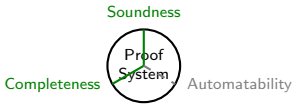
  - This expression is called a sequent.
  - The sequent is said to be valid if such a derivation actually exists, i.e., you can actually derive G from $F_1, \ldots F_n$ using the rules.

A proof system

What are our guiding principles to define rules?

E.g., we want to obtain $F, F \to G \vdash G$, but not $p, q \vdash p \wedge \neg q$.

- Soundness: whatever we prove is "correct":
  - If $F_1, \ldots, F_n \vdash G$ is valid, then $F_1 \ldots F_n \models G$ holds.
- Completeness: whatever is correct can be proved
  - If $F_1, \ldots, F_n \models G$ holds, then $F_1 \ldots F_n \vdash G$ is valid.
- Automatability



(a logician might not mind losing this 3rd spoke!)

(General) Semantic Entailment

$$F_1, \ldots, F_n \models G$$

if for all valuations in which $F_1, \ldots, F_n$ evaluate to 1, G also evaluates to 1.

(Recall: a valuation is just an assignment of a truth value to each atomic proposition.)

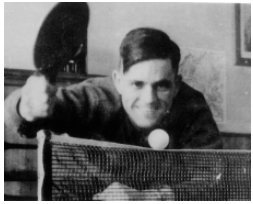
A formal proof system: Natural deduction

Natural deduction rules (from Huth & Ryan's Book)

12 Axioms or rules:

1. Introducing and Eliminating $\wedge$
2. Introducing and Eliminating $\rightarrow$
3. Introducing and Eliminating $\vee$
4. Negation rules: Introducing, Eliminating and Double negation
5. Bot rule

4 Derived/inferred rules to make life easier!



Gerhard Gentzen (1909-1945)

Conjunction Rules

If my cat is brown and it is fat then my cat is brown and fat!

$$\frac{F \qquad G}{F \wedge G}\ \wedge i \qquad \frac{F \wedge G}{F}\ \wedge e_1 \qquad \frac{F \wedge G}{G}\ \wedge e_2$$

- Rules to Introduce and Elimination $\wedge$
- **Exercise 3.1 Show that $p \wedge q, r \vdash q \wedge r$ is a valid sequent.**

1.	$p \wedge q$	premise
2.	r	premise
3.	q	$\wedge e_2$ 1
4.	$q \wedge r$	$\wedge i$ 3,2

Conjunction Rules

If my cat is brown and it is fat then my cat is brown and fat!

$$\frac{F \quad G}{F \wedge G} \wedge i \qquad \frac{F \wedge G}{F} \wedge e \qquad \frac{F \wedge G}{G} \wedge e$$

- Rules to Introduce and Elimination $\wedge$
- **Exercise 3.1 Show that $p \wedge q, r \vdash q \wedge r$ is a valid sequent.**

1.	$p \wedge q$	premise
2.	r	premise
3.	q	$\wedge e$ 1
4.	$q \wedge r$	$\wedge i$ 3,2

When clear we omit subscripts e_1, e_2 and just write e.

CS228 : Logic for Computer Science

Module 1: Propositional Logic

Lecture 4: Natural Deduction Rules

Instructor: S. Akshay

IIT Bombay, India

Logistics

Quiz 1

- Aug 28th 5.10pm
- Venue: Most likely LH 101, 102, LT 105 (More details on Piazza)
- Syllabus: All material covered till Aug 25th.

Problemsheets and help sessions

- First problemsheet is out!
- Tomorrow 3.30pm there is an optional help session at CC101, 103, 105.

Recap: Lectures 1 and 2

Lecture 1: Syntax

- Atomic propositions; logical connectives ($\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \oplus, \top, \bot$)
- Propositional formulas (defined inductively)

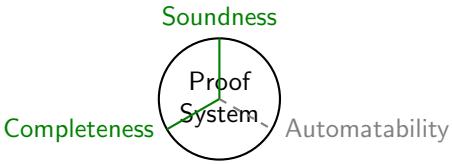
Lecture 2: Parse Trees, Semantics & Problems of Interest

- Parse trees / DAGs for formulas
- Semantics via truth tables
- Satisfiability, validity, semantic entailment ($\models$)

Recap: Lecture 3

Towards Formal Proofs

- Semantic equivalence ($\equiv$); used it to simplify formulas
- Minimal/universal connectives
- Truth tables don't scale (2^{100} rows!)
- Need a formal proof system
  - Sequents $F_1, \ldots, F_n \vdash G$ is valid if we can derive G from $F_1, \ldots, F_n$ using the rules of the proof system
    - Soundness: $F_1, \ldots, F_n \vdash G$ valid $\Rightarrow F_1, \ldots, F_n \models G$
    - Completeness: $F_1, \ldots, F_n \models G \Rightarrow F_1, \ldots, F_n \vdash G$ valid
  - Started Natural deduction: saw the $\wedge$ rules



Topic 4.1

Natural Deduction Rules

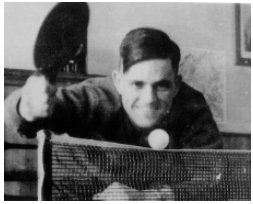
A formal proof system: Natural deduction

Natural deduction rules (from Huth & Ryan's Book)

12 Axioms or rules:

1. Introducing and Eliminating $\wedge$
2. Introducing and Eliminating $\rightarrow$
3. Introducing and Eliminating $\vee$
4. Negation rules: Introducing, Eliminating and Double negation
5. Bot rule

4 Derived/inferred rules to make life easier!



Gerhard Gentzen (1909–1945)

Conjunction Rules

If my cat is brown and it is fat then my cat is brown and fat!

$$\frac{F \qquad G}{F \wedge G}\ \wedge i \qquad \frac{F \wedge G}{F}\ \wedge e \qquad \frac{F \wedge G}{G}\ \wedge e$$

- Rules to Introduce and Eliminate $\wedge$
- **Exercise 4.1 Show that $p \wedge q, r \vdash q \wedge r$ is a valid sequent.**

1.	$p \wedge q$	premise
2.	r	premise
3.	q	$\wedge e$ 1
4.	$q \wedge r$	$\wedge i$ 3,2

When clear we omit subscripts e_1, e_2 and just write e.

Double negation rules

If it isn't the case that I don't like chocolate, then I like chocolate!

$$\frac{\neg\neg F}{F}\ \neg\neg e \qquad\qquad \frac{G}{\neg\neg G}\ \neg\neg i$$

Exercise 4.2

Prove the sequent: $p, \neg\neg(q \wedge r) \vdash \neg\neg p \wedge r$ using only the rules till now!

1.	p	premise
2.	$\neg\neg(q \wedge r)$	premise
3.	$q \wedge r$	$\neg\neg e$ 2
4.	r	$\wedge e$ 3
5.	$\neg\neg p$	$\neg\neg i$ 1
6.	$\neg\neg p \wedge r$	$\wedge i$ 5,4

(Negation rules come later!)

Implication Rules

Whenever it rains, I carry an umbrella. It's raining — so I have my umbrella!

The implies elimination rule or Modus Ponens MP

$$\frac{F \qquad F \to G}{G} \to e \text{ or MP}$$

Exercise 4.3: Show that $p, p \to q, p \to (q \to \neg\neg r) \vdash r$

1.	$p \to (q \to \neg\neg r)$	premise
2.	$p \to q$	premise
3.	p	premise
4.	$q \to \neg\neg r$	MP 1,3
5.	q	MP 2,3
6.	$\neg\neg r$	MP 4,5
7.	r	$\neg\neg e$ 6

Implication Rules -2

If I were a cheetah then I would run fast. I cannot run fast, hence I am not a cheetah!

Another implies elimination rule or Modus Tollens MT

$$\frac{F \to G \qquad \neg G}{\neg F}\ MT$$

Exercise 4.4

Show the following sequents are valid

1. $\neg p \to q, \neg q \vdash p$
2. $p \to \neg q, q \vdash \neg p$

1.	$p \to \neg q$	premise
2.	q	premise
3.	$\neg\neg q$	$\neg\neg i$ 2
4.	$\neg p$	MT 1,3

Implication Rules -3

- By MT, we have $p \rightarrow q, \neg q \vdash \neg p$.
- This looks like the contrapositive! i.e., can we prove $p \rightarrow q \vdash \neg q \rightarrow \neg p$?
- But we don't know how to introduce $\rightarrow$ yet!
- Given $p \rightarrow q$, let us assume $\neg q$. Can we then prove $\neg p$?
- Yes, from premises AND assumption, using MT we get:

1.	$p \rightarrow q$	premise
2.	$\neg q$	assumption
3.	$\neg p$	MT 1,2
4.	$\neg q \rightarrow \neg p$	$\rightarrow i$ 2-3

Implication Rules -3 (Contd)

To convince someone that "if it rains, I'll get wet," you don't check every possible day — you just imagine it raining, and show that you'd get wet!

$$\frac{\boxed{\begin{array}{c} F \\ \vdots \\ G \end{array}}}{F \to G} \to i$$

- Can use F and any premises so far; Can nest boxes, have multiple boxes etc.
- Line immediately after a closed box must match the pattern!
- F and G can be same! Consider premise p which shows $p \vdash p$, then

1.	p	assumption
2.	$p \to p$	$\to i$ 1-1

- We write $\vdash p \to p$ to denote it doesn't depend on any premises.
- Formula F such that the sequent $\vdash F$ is valid is called a theorem.

Implication Rules -3 (Contd)

In English: "If mammals breathe air, and non-mammals aren't whales, then whales breathe air!"

Example: Using all the rules, show that $\vdash (q \to r) \to [(\neg q \to \neg p) \to (p \to r)]$

1.	$q \to r$	assumption
2.	$\neg q \to \neg p$	assumption
3.	p	assumption
4.	$\neg\neg p$	$\neg\neg i$ 3
5.	$\neg\neg q$	MT 2,4
6.	q	$\neg\neg e$ 5
7.	r	MP 1,6
8.	$p \to r$	$\to i$ 3-7
9.	$(\neg q \to \neg p) \to (p \to r)$	$\to i$ 2-8
10.	$(q \to r) \to [(\neg q \to \neg p) \to (p \to r)]$	$\to i$ 1-9

Converts proof of $F_1, \ldots, F_n \vdash G$ to $\vdash F_1 \to (F_2 \to (\ldots (F_n \to G) \ldots))$

Remember the Pop Quiz?

Back in the Pop Quiz, we asked whether $\varphi = (p \to q) \wedge (q \to r) \to (p \to r)$ is valid — and we had to check it with a truth table.

Now we can prove it instead!

1.	$(p \to q) \wedge (q \to r)$	assumption
2.	$p \to q$	$\wedge e_1$ 1
3.	$q \to r$	$\wedge e_2$ 1
4.	p	assumption
5.	q	MP 2,4
6.	r	MP 3,5
7.	$p \to r$	$\to i$ 4-6
8.	$(p \to q) \wedge (q \to r) \to (p \to r)$	$\to i$ 1-7

Some exercises

Exercise 4.5

Consider the following sequent

$$p \wedge q \to r \vdash p \to (q \to r)$$

1. Is it a valid sequent?
2. Which of the steps in your derivation are deterministic?
3. Write down the "converse" of the sequent. Is it a valid sequent?

Exercise 4.6

Using not more than 10 lines, prove that the following sequents are valid.

1. $(p \wedge q) \wedge r \vdash p \wedge (q \wedge r)$
2. $\vdash (p \wedge q) \to q$
3. $p \to q \wedge r \vdash (p \to q) \wedge (p \to r)$

Exercise 4.5: Solution

$p \wedge q \to r \vdash p \to (q \to r)$

1.	$p \wedge q \to r$	premise
2.	p	assumption
3.	q	assumption
4.	$p \wedge q$	$\wedge i$ 2,3
5.	r	MP 1,4
6.	$q \to r$	$\to i$ 3-5
7.	$p \to (q \to r)$	$\to i$ 2-6

$p \to (q \to r) \vdash p \wedge q \to r$

1.	$p \to (q \to r)$	premise
2.	$p \wedge q$	assumption
3.	p	$\wedge e_1$ 2
4.	q	$\wedge e_2$ 2
5.	$q \to r$	MP 1,3
6.	r	MP 5,4
7.	$p \wedge q \to r$	$\to i$ 2-6

Disjunction rules

If the ball is round then the ball is either round or green.

$$\frac{F}{F \vee G}\ \vee i_1 \qquad \frac{G}{F \vee G}\ \vee i_2$$

$$\frac{F \vee G \qquad \boxed{\begin{array}{c} F \\ \vdots \\ H \end{array}} \qquad \boxed{\begin{array}{c} G \\ \vdots \\ H \end{array}}}{H}\ \vee e$$

- If $F \vee G$ is true and from BOTH F and G we are able to obtain H, then we can deduce H

Whether it's sunny or cloudy, I'll go for a walk — either way, I'm walking!

Disjunction rules (Contd)

Example: $q \to r \vdash (p \vee q) \to (p \vee r)$

1.	$q \to r$	premise
2.	$p \vee q$	assumption
3.	p	assumption
4.	$p \vee r$	$\vee i_1$ 3
5.	q	assumption
6.	r	MP 1,5
7.	$p \vee r$	$\vee i_2$ 6
8.	$p \vee r$	$\vee e$ 2, 3-4, 5-7
9.	$(p \vee q) \to (p \vee r)$	$\to i$ 2-8

CS228 : Logic for Computer Science

Module 1: Propositional Logic

Lecture 5: Soundness and Completeness of Natural Deduction

Instructor: S. Akshay

IIT Bombay, India

Recap: Rules of Natural Deduction

Conjunction:

$$\frac{F \quad G}{F \wedge G}\ \wedge i \qquad \frac{F \wedge G}{F}\ \wedge e_1 \qquad \frac{F \wedge G}{G}\ \wedge e_2$$

Double Negation:

$$\frac{\neg\neg F}{F}\ \neg\neg e \qquad \frac{G}{\neg\neg G}\ \neg\neg i$$

Implication (MP, MT & Intro):

$$\frac{F \quad F \to G}{G}\ \text{MP} \qquad \frac{F \to G \quad \neg G}{\neg F}\ \text{MT} \qquad \frac{\begin{array}{|c|}\hline F \\ \vdots \\ G \\ \hline\end{array}}{F \to G}\ \to i$$

Disjunction:

$$\frac{F}{F \vee G}\ \vee i_1 \qquad \frac{G}{F \vee G}\ \vee i_2 \qquad \frac{F \vee G \qquad \begin{array}{|c|}\hline F \\ \vdots \\ H \\ \hline\end{array} \qquad \begin{array}{|c|}\hline G \\ \vdots \\ H \\ \hline\end{array}}{H}\ \vee e$$

Single Negation Rules

- We use contradiction, an expression of the form $F \wedge \neg F$, for a propositional formula F.
- Two contradictions are equivalent: $p \wedge \neg p$ is equivalent to $\neg r \wedge r$, so we will use $\bot$!

$$\frac{\bot}{F}\ \bot e \qquad\qquad \frac{F \qquad \neg F}{\bot}\ \neg e$$

$$\frac{\begin{array}{|c|}\hline F \\ \vdots \\ \bot \\ \hline\end{array}}{\neg F}\ \neg i$$

Exercise 5.1: Derive $p \rightarrow \neg p \vdash \neg p$.

1.	$p \rightarrow \neg p$	premise
2.	p	assumption
3.	$\neg p$	MP 1,2
4.	$\bot$	$\neg e$ 2,3
5.	$\neg p$	$\neg i$ 2-4

Some Derived Rules

Exercise 5.2

(i) Prove that Modus Tollens rule can be derived from the other rules above! Are there any other such rules that can be derived? Hint: what about $\neg\neg i$?

(ii) Prove "proof by contradiction", i.e., show that if from $\neg F$ we can derive $\bot$, then we can deduce F. Hint: use only $\rightarrow$ intro/elim rules and negation intro/double negation elimination.

(iii) Prove the theorem called Law of Excluded Middle (also called LEM): $\vdash F \vee \neg F$ is valid!

Exercise 5.2(i)-(ii): MT and Proof by Contradiction

(i) $F \to G, \neg G \vdash \neg F$

1.	$F \to G$	premise
2.	$\neg G$	premise
3.	F	assumption
4.	G	MP 1,3
5.	$\bot$	$\neg e$ 4,2
6.	$\neg F$	$\neg i$ 3-5

(ii) $\neg F \to \bot \vdash F$

1.	$\neg F \to \bot$	given (via $\to i$)
2.	$\neg F$	assumption
3.	$\bot$	MP 1,2
4.	$\neg\neg F$	$\neg i$ 2-3
5.	F	$\neg\neg e$ 4

Exercise 5.2(iii): Law of Excluded Middle (LEM)

$$\vdash F \vee \neg F$$

1.	$\neg(F \vee \neg F)$	assumption
2.	F	assumption
3.	$F \vee \neg F$	$\vee i_1$ 2
4.	$\bot$	$\neg e$ 3,1
5.	$\neg F$	$\neg i$ 2-4
6.	$F \vee \neg F$	$\vee i_2$ 5
7.	$\bot$	$\neg e$ 6,1
8.	$\neg\neg(F \vee \neg F)$	$\neg i$ 1-7
9.	$F \vee \neg F$	$\neg\neg e$ 8

Summary: All Rules of Natural Deduction

Conjunction:

$$\frac{F \qquad G}{F \wedge G}\ \wedge i \qquad \frac{F \wedge G}{F}\ \wedge e_1 \qquad \frac{F \wedge G}{G}\ \wedge e_2$$

Disjunction:

$$\frac{F}{F \vee G}\ \vee i_1 \qquad \frac{G}{F \vee G}\ \vee i_2 \qquad \frac{F \vee G \qquad \begin{array}{c}[F]\\ \vdots \\ H\end{array} \qquad \begin{array}{c}[G]\\ \vdots \\ H\end{array}}{H}\ \vee e$$

Double Negation:

$$\frac{\neg\neg F}{F}\ \neg\neg e$$

Implication:

$$\frac{F \qquad F \to G}{G}\ \text{MP} \qquad \frac{\begin{array}{c}[F]\\ \vdots \\ G\end{array}}{F \to G}\ \to i$$

Negation & Bot:

$$\frac{\begin{array}{c}[F]\\ \vdots \\ \bot\end{array}}{\neg F}\ \neg i \qquad \frac{F \qquad \neg F}{\bot}\ \neg e \qquad \frac{\bot}{F}\ \bot e$$

Derived Rules:

$$\frac{F \to G \qquad \neg G}{\neg F}\ \text{MT} \qquad \frac{\begin{array}{c}[\neg F]\\ \vdots \\ \bot\end{array}}{F}\ \text{PBC}$$

$$\frac{G}{\neg\neg G}\ \neg\neg i$$

LEM: $\vdash F \vee \neg F$

Topic 5.1

Soundness of Natural Deduction for Propositional Logic

Soundness of Natural Deduction for Propositional Logic

Theorem (What is derivable is true!)

If $F_1, \ldots, F_n \vdash G$ is valid, then $F_1, \ldots, F_n \models G$ holds

i.e., G being provable from $F_1, \ldots, F_n$ means: for every valuation, whenever $F_1, \ldots, F_n$ evaluate to 1, G also evaluates to 1.

Proof: By Induction (on what?!)

- Assume $F_1, \ldots, F_n \vdash G$.
- Thus there is some proof that yields G. Induction is on number of lines, k, of this proof!
- When $k = 1$, there is only one line in the proof, which is the premise, say F. This gives the sequent $F \vdash F$. But in this case $F \models F$!
- Induction Hypothesis: Whenever $F_1, \ldots, F_n \vdash G$ is derived using a proof of length $\leq k - 1$, we have $F_1, \ldots, F_n \models G$.
  Note! We are using "Strong Induction"

Soundness of Natural Deduction for Propositional Logic (Contd.)

- Consider a proof with k lines.
- How did we arrive at G? Which proof rule gave G as the last line? Case Analysis!
  - Case 1: Assume G was obtained using $\wedge i$ rule.
    - Then G is of the form $G_1 \wedge G_2$.
    - G_1 and G_2 were proved earlier, say at lines $k_1, k_2 < k$.
    - i.e., we have shorter proofs $F_1, \ldots, F_n \vdash G_1$ and $F_1, \ldots, F_n \vdash G_2$
    - By inductive hypothesis, $F_1, \ldots, F_n \models G_1$ and $F_1, \ldots, F_n \models G_2$. This implies, $F_1, \ldots, F_n \models G_1 \wedge G_2$ (by definition!).
  - Case 2: Assume G was obtained using $\rightarrow i$, i.e., G is of the form $G_1 \rightarrow G_2$.
    - A box starting with G_1 was opened at some line $k_1 < k$.
    - Last line in box is G_2 and line just after the box is G.
    - Add G_1 as premise, along with $F_1, \ldots, F_n$, i.e., get proof $F_1, \ldots, F_n, G_1 \vdash G_2$, of length $k - 1$. By inductive hypothesis, $F_1, \ldots, F_n, G_1 \models G_2$. But, this is same as $F_1, \ldots, F_n \models G_1 \rightarrow G_2$
  - All other cases are by considering each rule separately.
  - **Exercise 5.4: Check all other 10 cases and complete the proof!** □

Topic 5.2

Completeness of Natural Deduction for Propositional Logic

Completeness of Natural Deduction for propositional logic

Theorem (Anything that is correct is provable)

If $F_1, \ldots, F_n \models G$ holds, then $F_1, \ldots, F_n \vdash G$ is valid

Whenever $F_1, \ldots, F_n$ semantically entail G, then G can be derived from $F_1, \ldots, F_n$ using the ND rules.

Proof: Let $F_1, \ldots, F_n \models G$. Proof is in three steps

- Step 1: Show that $\models F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ holds. This is easy!
  - By contradiction, suppose $\not\models F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$.
  - This is a nested implication, is false only if there exists an assignment s.t. G evaluates to 0 and $F_1, \ldots F_n$ evaluate to 1 .
  - Which contradicts $F_1, \ldots, F_n \models G$.
- Step 2: Show that $\vdash F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ is valid
- Step 3: Show that $F_1, \ldots, F_n \vdash G$ is valid

Completeness of Natural Deduction for propositional logic

Theorem (Anything that is correct is provable)

If $F_1, \ldots, F_n \models G$ holds, then $F_1, \ldots, F_n \vdash G$ is valid

Whenever $F_1, \ldots, F_n$ semantically entail G, then G can be derived from $F_1, \ldots, F_n$ using the ND rules.

Proof: Let $F_1, \ldots, F_n \models G$. Proof is in three steps

- Step 1: Show that $\models F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ holds.
- Step 2: Show that $\vdash F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ is valid
- Step 3: Show that $F_1, \ldots, F_n \vdash G$ is valid This is also easy!
  - Take the proof from Step 2.
  - Add $F_1, \ldots F_n$ as premises as done in soundness proof.
  - Then apply MP n times to arrive at G.

Completeness of Natural Deduction for propositional logic

Theorem (Anything that is correct is provable)

If $F_1, \ldots, F_n \models G$ holds, then $F_1, \ldots, F_n \vdash G$ is valid

Whenever $F_1, \ldots, F_n$ semantically entail G, then G can be derived from $F_1, \ldots, F_n$ using the ND rules.

Proof: Let $F_1, \ldots, F_n \models G$. Proof is in three steps

- Step 1: Show that $\models F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ holds.
- Step 2: Show that $\vdash F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ is valid
- Step 3: Show that $F_1, \ldots, F_n \vdash G$ is valid

So, the only thing left is step 2 from step 1. This is the hard part!

Completeness of Natural Deduction for propositional logic

Step 2 follows from the following theorem:

Theorem

For any propositional formula G, if $\models G$ holds, then $\vdash G$ is valid. That is, if G is a tautology, then it is a theorem!

Proof

- Let $p_1, \ldots, p_m$ be the propositional variables in G; all 2^m valuations make G true.
- From just this, can derive a proof of G?
  - Step 2.1 Key Idea: Encode each line of truth table as a sequent!
    - If in line ℓ of truth table, p_i has value 1, then set $\hat{p_i} = p_i$ otherwise set $\hat{p_i} = \neg p_i$
    - Sequent for line ℓ: $\hat{p_1} \ldots \hat{p_m} \vdash G$
  - Step 2.2: Construct proofs for each of 2^m sequents.
    - For each line ℓ show that the sequent $\hat{p_1} \ldots \hat{p_m} \vdash G$ is valid.
  - Step 2.3: Assemble them to obtain proof of G

Completeness Proof contd

Lemma

Let F be any propositional formula with variables $p_1, \ldots, p_m$. Let ℓ be a line number in Truth Table of F. Let $\hat{p}_i = p_i$ if p_i has value 1 in line ℓ, and $= \neg p_i$ if p_i has value 0 in line ℓ. Then

1. $\hat{p}_1, \ldots, \hat{p}_m \vdash F$ is valid if F evaluates to 1 in line ℓ
2. $\hat{p}_1, \ldots, \hat{p}_m \vdash \neg F$ is valid if F evaluates to 0 in line ℓ

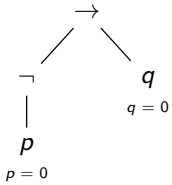
Proof by Structural Induction on structure of F, i.e., induction on height of parse tree for F.

- Base Case : If $F = p$, a proposition, then $p \vdash p$ and $\neg p \vdash \neg p$.
- Induction Hyp: Assume Lemma for formulae with parse tree height $\leq k - 1$
- Case Negation : $F = \neg F_1$
  - If F evaluates 1 at line ℓ, then F_1 evaluates to 0. By inductive hypothesis, $\hat{p}_1, \ldots, \hat{p}_m \vdash \neg F_1$, i.e., $\hat{p}_1, \ldots, \hat{p}_m \vdash F$.
  - If F evaluates 0 at line ℓ, then F_1 evaluates to 1. By inductive hypothesis, $\hat{p}_1, \ldots, \hat{p}_m \vdash F_1$. Use $\neg\neg i$ rule to obtain proof of $\hat{p}_1, \ldots, \hat{p}_m \vdash \neg\neg F_1$, i.e., $\hat{p}_1, \ldots, \hat{p}_m \vdash \neg F$.

Seeing the Induction: An Example

Let $F = \neg p \to q$, i.e., $F_1 = \neg p$ and $F_2 = q$. We derive the proof for line ℓ below.

p	q	$\neg p$	F	
0	0	1	0	$\leftarrow \ell$
0	1	1	1	
1	0	0	1	
1	1	0	1	



Goal: $\hat{p}, \hat{q} \vdash \neg F$.

- Leaves: $p = 0 \Rightarrow \hat{p} = \neg p$, so $\neg p \vdash \neg p$ (base case). $q = 0 \Rightarrow \hat{q} = \neg q$, so $\neg q \vdash \neg q$ (base case).
- $\neg$ node ($F_1 = \neg p$): F_1 evaluates to 1 $\Rightarrow$ using the leaf, $\neg p \vdash \neg p$, i.e., $\neg p \vdash F_1$.
- $\to$ node (F, root): F evaluates to 0 at this line. We have $\neg p$ (from the $\neg$ node) and $\neg q$ (from the leaf) as premises — using these, we derive $\neg F$ on the right:

1.	$\neg p$	premise
2.	$\neg q$	premise
3.	$\neg p \to q$	assumption
4.	q	MP 3,1
5.	$\bot$	$\neg e$ 4,2
6.	$\neg(\neg p \to q)$	$\neg i$ 3-5

CS228 : Logic for Computer Science

Module 1: Propositional Logic

Lecture 6: Completeness of Natural Deduction

Instructor: S. Akshay

IIT Bombay, India

Recap: The Story so far

Module 1 Propositional Logic (Lectures 1–5)

- L1: Syntax
- L2: Parse Trees, Semantics & Problems of Interest
- L3: Formal Proofs & Natural Deduction
- L4: Natural Deduction Rules
- L5: Soundness & Completeness

Soundness: Anything that is provable is (semantically) correct

If $F_1, \ldots, F_n \vdash G$ is valid, then $F_1, \ldots, F_n \models G$ holds

Completeness: Anything that is correct is provable in ND!

If $F_1, \ldots, F_n \models G$ holds, then $F_1, \ldots, F_n \vdash G$ is valid

Recap: Completeness of Natural Deduction for propositional logic

Theorem (Anything that is correct is provable)

If $F_1, \ldots, F_n \models G$ holds, then $F_1, \ldots, F_n \vdash G$ is valid

Whenever $F_1, \ldots, F_n$ semantically entail G, then G can be derived from $F_1, \ldots, F_n$ using the ND rules.

Proof: Let $F_1, \ldots, F_n \models G$. Proof is in three steps

- Step 1: Show that $\models F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ holds. This is easy!
  - By contradiction, suppose $\not\models F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$.
  - This nested implication is false only if there exists an assignment s.t. G evaluates to 0 and $F_1, \ldots F_n$ evaluate to 1 .
  - Which contradicts $F_1, \ldots, F_n \models G$.
- Step 2: Show that $\vdash F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ is valid
- Step 3: Show that $F_1, \ldots, F_n \vdash G$ is valid

Recap: Completeness of Natural Deduction for propositional logic

Theorem (Anything that is correct is provable)

If $F_1, \ldots, F_n \models G$ holds, then $F_1, \ldots, F_n \vdash G$ is valid

Whenever $F_1, \ldots, F_n$ semantically entail G, then G can be derived from $F_1, \ldots, F_n$ using the ND rules.

Proof: Let $F_1, \ldots, F_n \models G$. Proof is in three steps

- Step 1: Show that $\models F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ holds.
- Step 2: Show that $\vdash F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ is valid
- Step 3: Show that $F_1, \ldots, F_n \vdash G$ is valid This is also easy!
  - Take the proof from Step 2.
  - Add $F_1, \ldots F_n$ as premises as done in soundness proof.
  - Then apply MP n times to arrive at G.

Recap: Completeness of Natural Deduction for propositional logic

Theorem (Anything that is correct is provable)

If $F_1, \ldots, F_n \models G$ holds, then $F_1, \ldots, F_n \vdash G$ is valid

Whenever $F_1, \ldots, F_n$ semantically entail G, then G can be derived from $F_1, \ldots, F_n$ using the ND rules.

Proof: Let $F_1, \ldots, F_n \models G$. Proof is in three steps

- Step 1: Show that $\models F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ holds.
- Step 2: Show that $\vdash F_1 \rightarrow (F_2 \rightarrow (\ldots (F_n \rightarrow G) \ldots))$ is valid
- Step 3: Show that $F_1, \ldots, F_n \vdash G$ is valid

So, the only thing left is Step 2. This is the hard part!

Completeness of Natural Deduction for propositional logic

Step 2 follows from the following theorem:

Theorem

For any propositional formula G, if $\models G$ holds, then $\vdash G$ is valid. That is, if G is a tautology, then it is a theorem!

Proof

- Let $p_1, \ldots, p_m$ be the propositional variables in G; all 2^m valuations make G true.
- From just this, can derive a proof of G?
  - Step 2.1 Key Idea: Encode each line of truth table as a sequent!
    - If in line ℓ of truth table, p_i has value 1, then set $\hat{p_i} = p_i$ otherwise set $\hat{p_i} = \neg p_i$
    - Sequent for line ℓ: $\hat{p_1} \ldots \hat{p_m} \vdash G$
  - Step 2.2: Construct proofs for each of 2^m sequents.
    - For each line ℓ show that the sequent $\hat{p_1} \ldots \hat{p_m} \vdash G$ is valid.
  - Step 2.3: Assemble them to obtain proof of G

Completeness Proof contd

Lemma

Let F be any propositional formula with variables $p_1, \ldots, p_m$. Let ℓ be a line number in Truth Table of F. Let $\hat{p}_i = p_i$ if p_i has value 1 in line ℓ, and $= \neg p_i$ if p_i has value 0 in line ℓ. Then

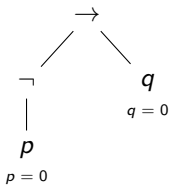
1. $\hat{p}_1, \ldots, \hat{p}_m \vdash F$ is valid if F evaluates to 1 in line ℓ
2. $\hat{p}_1, \ldots, \hat{p}_m \vdash \neg F$ is valid if F evaluates to 0 in line ℓ

Proof by Structural Induction on structure of F, i.e., induction on height of parse tree for F.

An Example

Let $F = \neg p \to q$, i.e., $F_1 = \neg p$ and $F_2 = q$. We derive the proof for line ℓ below.

p	q	$\neg p$	F	
0	0	1	0	$\leftarrow \ell$
0	1	1	1	
1	0	0	1	
1	1	0	1	



Goal: $\hat{p}, \hat{q} \vdash \neg F$.

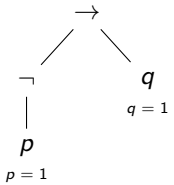
- Leaves: $p = 0 \Rightarrow \hat{p} = \neg p$, so $\neg p \vdash \neg p$ (base case).
  $q = 0 \Rightarrow \hat{q} = \neg q$, so $\neg q \vdash \neg q$ (base case).
- $\neg$ node ($F_1 = \neg p$): F_1 evaluates to 1 $\Rightarrow$ using the leaf, $\neg p \vdash \neg p$, i.e., $\neg p \vdash F_1$.
- $\to$ node (F, root): F evaluates to 0 at this line. We have $\neg p$ (from the $\neg$ node) and $\neg q$ (from the leaf) as premises — using these, we derive $\neg F$ on the right:

1.	$\neg p$	premise
2.	$\neg q$	premise
3.	$\neg p \to q$	assumption
4.	q	MP 3,1
5.	$\bot$	$\neg e$ 4,2
6.	$\neg(\neg p \to q)$	$\neg i$ 3-5

An Example (contd.)

Same $F = \neg p \to q$ ($F_1 = \neg p$, $F_2 = q$). Now let's try line ℓ: $p = 1, q = 1$.

p	q	$\neg p$	F	
0	0	1	0	
0	1	1	1	
1	0	0	1	
1	1	0	1	$\leftarrow \ell$



Goal: $\hat{p}, \hat{q} \vdash F$.

- Leaves: $p = 1 \Rightarrow \hat{p} = p$, so $p \vdash p$ (base case); $q = 1 \Rightarrow \hat{q} = q$, so $q \vdash q$ (base case) — both leaves now positive.
- $\neg$ node ($F_1 = \neg p$): F_1 now evaluates to 0 (since $p = 1$)! By IH, $p \vdash p$; use $\neg\neg i$ to get $p \vdash \neg\neg p$, i.e., $p \vdash \neg F_1$.
- $\to$ node (F, root): F_1 evaluates to 0, F_2 evaluates to 1 — a third sub-subcase! We have $p \vdash \neg F_1$ and $q \vdash F_2$ as premises — using these, we derive F on the right:

1.	p	premise
2.	q	premise
3.	$\neg\neg p$	$\neg\neg i$ 1
4.	$\neg p$	assumption
5.	$\bot$	$\neg e$ 4,3
6.	q	$\bot e$ 5
7.	$\neg p \to q$	$\to i$ 4-6

Completeness Proof contd

Lemma

Let F be any propositional formula with variables $p_1, \ldots, p_m$. Let ℓ be a line number in Truth Table of F. Let $\hat{p}_i = p_i$ if p_i has value 1 in line ℓ, and $= \neg p_i$ if p_i has value 0 in line ℓ. Then

1. $\hat{p}_1, \ldots, \hat{p}_m \vdash F$ is valid if F evaluates to 1 in line ℓ
2. $\hat{p}_1, \ldots, \hat{p}_m \vdash \neg F$ is valid if F evaluates to 0 in line ℓ

Proof by Structural Induction on structure of F, i.e., induction on height of parse tree for F.

- Base Case : If $F = p$, a proposition, then $p \vdash p$ and $\neg p \vdash \neg p$.
- Induction Hyp: Assume Lemma for formulae with parse tree height $\leq k - 1$
- Case Negation : $F = \neg F_1$
  - If F evaluates 1 at line ℓ, then F_1 evaluates to 0. By inductive hypothesis, $\hat{p}_1, \ldots, \hat{p}_m \vdash \neg F_1$, i.e., $\hat{p}_1, \ldots, \hat{p}_m \vdash F$.
  - If F evaluates 0 at line ℓ, then F_1 evaluates to 1. By inductive hypothesis, $\hat{p}_1, \ldots, \hat{p}_m \vdash F_1$. Use $\neg\neg i$ rule to obtain proof of $\hat{p}_1, \ldots, \hat{p}_m \vdash \neg\neg F_1$, i.e., $\hat{p}_1, \ldots, \hat{p}_m \vdash \neg F$.

Completeness Proof contd

Case $\rightarrow$: $F = F_1 \rightarrow F_2$

Two subcases:

- Subcase 1: F evaluates to 0 in line ℓ, (exactly our first example!)
  - then F_1 evaluates to 1 and F_2 to 0.
  - Let $\{q_1, \ldots, q_k\}$ be the variables of F_1 and let $\{r_1, \ldots, r_j\}$ be the variables in F_2; $\{q_1, \ldots, q_k\} \cup \{r_1, \ldots, r_j\} = \{p_1, \ldots, p_m\}$.
  - By inductive hypothesis, $\hat{q_1}, \ldots, \hat{q_k} \vdash F_1$ and $\hat{r_1}, \ldots, \hat{r_j} \vdash \neg F_2$.
  - Then, $\hat{p_1}, \ldots, \hat{p_m} \vdash F_1 \wedge \neg F_2$.
  - **Exercise 6.1:** Prove that $F_1 \wedge \neg F_2 \vdash \neg(F_1 \rightarrow F_2)$ and conclude this case!
- Subcase 2: F evaluates to 1 in line ℓ, there are 3 subsubcases.
  - If both F_1, F_2 evaluate to 1, then $\hat{p_1}, \ldots, \hat{p_m} \vdash F_1 \wedge F_2$.
    Exercise 6.2(i): Prove $F_1 \wedge F_2 \vdash F_1 \rightarrow F_2$ and conclude.
  - If both F_1, F_2 evaluate to 0, then $\hat{p_1}, \ldots, \hat{p_m} \vdash \neg F_1 \wedge \neg F_2$.
    Exercise 6.2(ii): Prove $\neg F_1 \wedge \neg F_2 \vdash F_1 \rightarrow F_2$ and conclude.
  - If F_1 evaluates to 0 and F_2 evaluates to 1, (our second example!) then $\hat{p_1}, \ldots, \hat{p_m} \vdash \neg F_1 \wedge F_2$.
    Exercise 6.2(iii): Prove $\neg F_1 \wedge F_2 \vdash F_1 \rightarrow F_2$, and complete this case.

Completeness proof (Contd.)

Exercise 6.3

- Complete similar proofs for the cases $\wedge, \vee$.

Or at least read it from Huth & Ryan's book!

This completes the proof of the Lemma, finishing Step 2.2 of the overall proof.

Completeness of Natural Deduction for propositional logic

Back to Step 2: If $\models G$ holds, then $\vdash G$ is valid; if G is a tautology, then it is a theorem!

Proof

- Let $p_1, \ldots, p_m$ be the propositional variables in G; all 2^m valuations make G true.
- From this, we derive a proof of G
  - Step 2.1 Key Idea: Encode each line of truth table as a sequent!
    - If in line ℓ of truth table, p_i has value 1, then set $\hat{p_i} = p_i$ otherwise set $\hat{p_i} = \neg p_i$
    - Sequent for line ℓ: $\hat{p_1} \ldots \hat{p_m} \vdash G$
  - Step 2.2: Use Previous Lemma to Construct proofs for each of 2^m sequents.
    - For each line ℓ, by Previous Lemma, we have $\hat{p_1} \ldots \hat{p_m} \vdash G$ is valid.
  - Step 2.3: Assemble the sequents to obtain proof of G
    - Use LEM repeatedly on all the propositional variables of G and $\vee$-e to obtain $\vdash G$.

An example

Consider the tautology $F = p \wedge q \rightarrow p$.

Exercise 6.4 Show that it is a theorem by applying all steps of proof above.

- Has two propositions p, q. So 4 valuations
- Then applying Lemma we have (derive them!):
  - $p, q \vdash p \wedge q \rightarrow p$
  - $\neg p, q \vdash p \wedge q \rightarrow p$
  - $p, \neg q \vdash p \wedge q \rightarrow p$
  - $\neg p, \neg q \vdash p \wedge q \rightarrow p$
- Using LEM to show $\vdash F$...

An example: Assembling the Proof

Assemble the four sequents via LEM and $\vee e$, applied three times — exactly as in Huth & Ryan:

1.	$p \vee \neg p$	LEM
2.	p	assumption
3.	$q \vee \neg q$	LEM
4.	q	assumption
5.	$p \wedge q \rightarrow p$	Use Lemma
6.	$\neg q$	assumption
7.	$p \wedge q \rightarrow p$	Use Lemma
8.	$p \wedge q \rightarrow p$	$\vee e$ 3,4-5,6-7
9.	$\neg p$	assumption
10.	$q \vee \neg q$	LEM
11.	q	assumption
12.	$p \wedge q \rightarrow p$	Use Lemma
13.	$\neg q$	assumption
14.	$p \wedge q \rightarrow p$	Use Lemma
15.	$p \wedge q \rightarrow p$	$\vee e$ 10,11-12,13-14
16.	$p \wedge q \rightarrow p$	$\vee e$ 1,2-8,9-15

Conclusion!

Propositional Logic is sound and complete.

This means we can freely switch between semantic ($\models$) and syntactic ($\vdash$) reasoning — pick whichever is easier for the problem at hand!

Are there other proof systems

- What we have looked at is called Gentzen-style Natural Deduction.
- Was introduced in 1934!

Hilbert-style Natural Deduction Proof System

- Has only 3 axioms and ONE inference rule! Modus Ponens
- Everything is derived from this...
- As a result derivations are long and hard, but showing structural induction is easy!

There are MANY more proof systems....

Intuitionistic Logic: No LEM! No double-negation elimination!

- If you want to prove F, you should prove F, not disprove $\neg F$!
- No proof by contradiction also!

Exercise 6.5 Why are Intuitionistic logics interesting? What was Brouwer-Hilbert controversy?

A notable point

Showing invalidity of a sequent

- To show a sequent $F_1, \ldots F_n \vdash G$ is valid, you just derive a proof.
- How do you show that it is NOT valid?

Use soundness!

1. Just find an assignment such that $F_1 = 1, \ldots F_n = 1$ and $G = 0$.
2. This means $F_1 \ldots F_n \not\models G$.
3. Now soundness implies: if $F_1 \ldots F_n \vdash G$ is valid, then $F_1 \ldots F_n \models G$ holds; the contrapositive gives: if $F_1 \ldots F_n \not\models G$, then $F_1 \ldots F_n \nvdash G$.
4. This completes the proof $F_1 \ldots F_n \nvdash G$.

Exercise 6.6: Prove or disprove: $p \vdash p \wedge q$ valid.

Another note regarding reusing what has been derived: The Copy Rule

Consider the theorem
$\vdash p \to (q \to p)$

1.	p	assumption
2.	q	assumption
3.	p	copy 1
4.	$q \to p$	$\to i$ 2-3
5.	$p \to (q \to p)$	$\to i$ 1-4

So, when can you copy? From above ... unless they depend on temporary assumptions whose box has been closed!

CS228 : Logic for Computer Science

Module 1: Propositional Logic

Lecture 7: Normal Forms

Instructor: S. Akshay

IIT Bombay, India

So far: Propositional Logic

Module 1: Propositional Logic - Encoding knowledge into simple formulas

Logic: a formal language & calculus for reasoning

- Propositional Logic Syntax (write a grammar)
- Propositional Logic Semantics (via truth tables)
- Problems of interest - Satisfiability, Validity, Equivalence, Counting, etc.
- Ways to solve these problems:
  1. Use the Truth Table
     - Advantages: Can solve by just table lookup.
     - Disadvantages: TOO large!
  2. Use the rules of Natural Deduction: a formal proof system
     - Advantages: Purely symbolic, often shorter than Truth Table, Sound and Complete.
     - Disadvantages: Which rule to use when requires ingenuity, worst case proof will be huge.

Can we say when these problems are easy?

- Exploit their structure!

Topic 7.1

Normal Forms

Normal forms

Questions

1. Can some representation make the formula easier to analyze?
   - Example: is $(x \vee \neg x) \vee (\neg x \wedge y) \vee (x \wedge y \wedge w) \vee (\neg x \wedge \neg z \wedge \neg w)$ valid?
     - Would you write the truth table with 16 rows?
     - Easier: would you try to derive it from no premises?
     - Even easier: just observe that it's a disjunction of many subformulas, and the first one is valid, so the entire thing is valid!
2. Are there "Normal forms" which allow for easy and uniform algorithmic analysis?
   - Building algorithms and tools for handling arbitrary formulas is difficult
   - Normalization also results in standardization and interoperability of tools.

First step : remove $\oplus$, $\rightarrow$, and $\leftrightarrow$.

Use equivalences to remove $\oplus$, $\rightarrow$, and $\leftrightarrow$ from a formula. Recall that $p \equiv q$ denotes semantic equivalence.

Is there a difference between $p \equiv q$ and $\vdash p \leftrightarrow q$?

- $(p \rightarrow q) \equiv (\neg p \vee q)$
- $(p \oplus q) \equiv (p \vee q) \wedge (\neg p \vee \neg q)$
- $(p \leftrightarrow q) \equiv \neg(p \oplus q)$

For the ease of presentation, we will assume you can remove them at will.

So our propositional formulas are only in terms of $\neg, \vee, \wedge$.

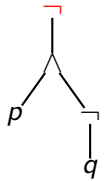
Topic 7.2

Negation normal form

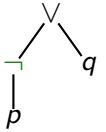
Negation normal form

Definition

A formula is in Negation Normal Form (NNF) if $\neg$ is only in front of the propositional variables and only other connectives are $\wedge, \vee$.



$\neg(p \wedge \neg q)$ ✗ Not in NNF



$\neg p \vee q$ ✓ In NNF

Negation normal form (contd.)

Theorem

For every formula F, there is a formula F' in NNF such that $F \equiv F'$.

Proof.

Convert to formula with only $\neg, \wedge, \vee$ and then apply DeMorgan's laws! □

Exercise 7.1

Convert the following formulas into NNF

- $\neg(p \rightarrow q)$
- $\neg(\neg((s \rightarrow \neg(p \leftrightarrow q))) \oplus (\neg q \vee r))$
- $\neg((p \rightarrow q) \rightarrow (p \rightarrow q))$
- $\neg\neg\neg p$

Exercise 7.2 (Is the NNF "canonical"?)

Do there exist two distinct formulae that are both in NNF but are equivalent?

Topic 7.3

Conjunctive normal form

Some terminology

- Propositional variables are also referred as atoms
- A literal is either an atom or its negation
- A clause is a disjunction of literals.

Examples

- p is an atom but $\neg p$ is not.
- $\neg p$ and p both are literals.
- $p \vee \neg p \vee p \vee q$ is a clause.

idempotent:
$p \vee p \equiv p$
(repeats don't matter)

Since $\vee$ is associative, commutative, and idempotent, a clause is sometimes referred to as a set of literals.
e.g., $p \vee \neg p \vee p \vee q$ can be written as $\{p, \neg p, q\}$.

Conjunctive normal form (CNF)

Definition

A formula is in CNF if it is a conjunction of clauses.

- $\neg p$ and p both are in CNF.
- $F = (p \vee \neg q) \wedge (r \vee \neg q) \wedge \neg r$ is in CNF.

Since $\wedge$ is associative, commutative and idempotent, a CNF formula is sometimes written as a set of clauses. E.g., F above can be written as:

- $\{(p \vee \neg q), (r \vee \neg q), \neg r\}$ or
- $\{\{p, \neg q\}, \{r, \neg q\}, \{\neg r\}\}$

Exercise 7.3 Which of the following are in CNF?

- $(p \vee q) \wedge (\neg p \wedge q)$
- $(p \wedge q) \vee (\neg p \vee q)$
- $p \rightarrow q$
- $(p \vee q) \vee (\neg p \vee q)$

Exercise 7.4 Write a formal grammar for CNF.

$CNF ::= Clause \mid Clause \wedge CNF$
$Clause ::= p \mid \neg p \mid Clause \vee Clause$

CNF conversion

Theorem

For every formula F there is another formula F' in CNF s.t. $F \equiv F'$.

Example

Conversion to CNF

$(p \to (\neg q \wedge r)) \wedge (p \to \neg q) \equiv (\neg p \vee (\neg q \wedge r)) \wedge (\neg p \vee \neg q) \equiv (\neg p \vee \neg q) \wedge (\neg p \vee r) \wedge (\neg p \vee \neg q)$

Proof.

This gives an easy algorithm!

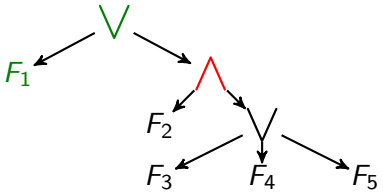
- Step 1: Convert F to NNF.
- Step 2: Flatten the $\wedge$ and $\vee$ gates, e.g., $(x \wedge y) \wedge (z \wedge y) \wedge (w \wedge z) \equiv (x \wedge y \wedge z \wedge w)$
- Step 3: Use distributivity (see next slide!)

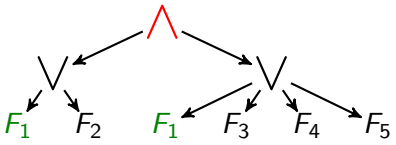
...

CNF conversion (contd.)

Proof(contd.)

Now the formulas have the following form with literals at leaves, e.g., $F_1 \vee (F_2 \wedge (F_3 \vee F_4 \vee F_5))$.





Since $\vee$ distributes over $\wedge$, we can always push $\vee$ inside $\wedge$.
Eventually, we will obtain a formula that is CNF. (Note how F_1 gets duplicated.) □

Exercise 7.5

Convert $(a \vee b) \wedge (c \vee (d \wedge e))$ to CNF using distributivity.

Solution: $(a \vee b) \wedge (c \vee (d \wedge e)) \equiv (a \vee b) \wedge (c \vee d) \wedge (c \vee e)$

CNF Conversion Applications

Checking validity: how hard is it in general?

Given an arbitrary formula F, how do we check if it's valid?

- Truth table: exponential in the number of variables
- Natural deduction: no algorithm – requires ingenuity

What if F is in CNF?

Exercise 7.6 Give an algorithm to check for validity of formula in CNF. What is its complexity?

What about satisfiability?

Fact: Checking satisfiability of a CNF formula is hard – in fact it is NP-complete!

(NP-complete, informally: among the hardest problems whose solutions can be *verified* in polynomial time – if you could solve CNF-SAT quickly, you could solve *every* such problem quickly.)

Advantages of CNF

- Like NNF, it has very few logical connectives
- Simple structure
- Validity is easy
- Many problems naturally encode into CNF.

  We will see more examples on this soon!

- As a result, many solvers take input in CNF

So given formula F, an important question is to convert it to CNF.

Topic 7.4

Tseitin's encoding

Conversion to CNF

We already saw a naive algorithm

- Step 1: Convert F to NNF.
- Step 2: Flatten the $\wedge$ and $\vee$ gates, e.g., $(x \wedge y) \wedge (z \wedge y) \wedge (w \wedge z) \equiv (x \wedge y \wedge z \wedge w)$
- Step 3: Use distributivity

But the transformation using distributivity explodes the formula.

What is the complexity blow-up in converting an arbitrary formula to CNF?

Exercise 7.7 Give an example of a propositional formula with $2n$ variables whose conversion to CNF produces a formula with 2^n clauses.

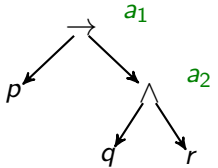
Conversion to CNF (contd.)

- Consider $F_2 = (x_1 \wedge y_1) \vee (x_2 \wedge y_2)$. What is $CNF(F_2)$?
- $CNF(F_2) = (x_1 \vee x_2) \wedge (x_1 \vee y_2) \wedge (y_1 \vee x_2) \wedge (y_1 \vee y_2)$
- Now consider $F_n = (x_1 \wedge y_1) \vee \ldots \vee (x_n \wedge y_n)$. What is $CNF(F_n)$?
- How many clauses does F_n have? 2^n!

Is there a way to avoid the explosion? Yes!

Tseitin's encoding

Tseitin's encoding



Consider $F = (p \to (q \wedge r))$

1. Write the parse tree of F.
2. Introduce a fresh "auxiliary" variable for each internal node, e.g., a_1, a_2 for $\to, \wedge$
3. Add constraints to define these variables: e.g., $a_1 \leftrightarrow (p \to a_2)$, $a_2 \leftrightarrow (q \wedge r)$
4. Conjunct them and add root: $G = (a_1 \leftrightarrow (p \to a_2)) \wedge (a_2 \leftrightarrow (q \wedge r)) \wedge (a_1)$
5. Each constraint can be written as CNF with just 3-4 clauses:
   - $a_1 \leftrightarrow (p \to a_2) \equiv (a_1 \vee p) \wedge (a_1 \vee \neg a_2) \wedge (\neg a_1 \vee \neg p \vee a_2)$
   - $a_2 \leftrightarrow (q \wedge r) \equiv (\neg a_2 \vee q) \wedge (\neg a_2 \vee r) \wedge (a_2 \vee \neg q \vee \neg r)$
6. Note: G is in CNF!

What is the relation between F and G? Are they equivalent?

- No! Not even same set of variables. But they are equisatisfiable: F is satisfiable iff G is.

Theorem: The Tseitin algo converts any formula into an equisatisfiable CNF formula.

How much do we improve using Tseitin?

Consider again $F_n = (x_1 \wedge y_1) \vee \ldots \vee (x_n \wedge y_n)$.

- With Tseitin's encoding we need
  - n auxiliary variables for each $\wedge$ gate.
  - Each gives a constraint of the form $a_i \leftrightarrow (x_i \wedge y_i)$ which can be written in CNF with just 3 constraints.
  - 1 more constraint (not a new variable!) to directly assert the root: $(a_1 \vee \ldots \vee a_n)$. (Small optimization: *root* doesn't really need its own variable, since nothing refers to it – we can assert its defining formula directly instead.)
- So we have $3n + 1$ clauses only (instead of 2^n)!
- Number of variables gone up to $3n$ from $2n$.
- Exponential reduction in size at cost of new variables!

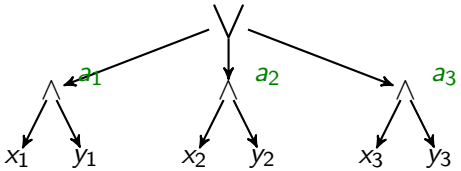
Theorem

Tseitin Encoding always leads to an at most linear blowup!

Proof: Exercise!

Tseitin encoding of F_3

$F_3 = (x_1 \wedge y_1) \vee (x_2 \wedge y_2) \vee (x_3 \wedge y_3)$ – 6 variables.



Tseitin encoding G_3

$G_3 = (\neg a_1 \vee x_1) \wedge (\neg a_1 \vee y_1) \wedge (a_1 \vee \neg x_1 \vee \neg y_1)$
$\wedge(\neg a_2 \vee x_2) \wedge (\neg a_2 \vee y_2) \wedge (a_2 \vee \neg x_2 \vee \neg y_2)$
$\wedge(\neg a_3 \vee x_3) \wedge (\neg a_3 \vee y_3) \wedge (a_3 \vee \neg x_3 \vee \neg y_3)$
$\wedge(a_1 \vee a_2 \vee a_3)$

9 variables, 10 clauses – matches $3n + 1 = 10$ for $n = 3$. (If instead we added one extra variable a_4 for the root, we'd need $n + 1$ more clauses: 4 for $n = 3$, giving $14 = 4n + 2$.)

Topic 7.5

Disjunctive normal form

Disjunctive normal form (DNF)

Definition

A formula is in DNF if it is a disjunction of conjunctions of literals.

Examples: $(p \wedge q) \vee (\neg p \wedge r \wedge s)$ is in DNF, but $(p \vee q) \wedge r$ is not.

Theorem

For every formula F there is another formula F' in DNF s.t. $F \equiv F'$.

Proof.

Proof is similar to CNF. □

Exercise 7.8

a. Give the formal grammar of DNF

b. Give a linear time algorithm to prove satisfiability of a DNF formula

So, CNF makes validity easy, DNF makes satisfiability easy.

CS228 : Logic for Computer Science

Module 2: The Problem of Satisfiability

Lecture 8: (Propositional) Resolution

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date (Module 1: Propositional Logic)

1. Syntax and Semantics
   - Questions of interest: Satisfiability, Validity
2. Formal proof systems
   - Natural Deduction
   - Soundness and Completeness
3. Normal Forms
   - Negation, Conjunctive and Disjunctive Normal Forms (NNF, CNF, DNF)
   - Tseitin's Encoding

Next... (Module 2: The Problem of Satisfiability)

Resolution and SAT-solvers

Why does Satisfiability matter?

Recall from Lecture 7: Checking satisfiability of a CNF formula (called SAT) is NP-complete.

- If we could solve SAT efficiently, we could solve every problem in NP efficiently too!
- Includes: scheduling, planning, circuit verification, cryptanalysis, puzzle-solving , . . . thousands of problems people care about, in industry and research.

The Cook–Levin Theorem

SAT is the first known NP-complete problem – discovered independently, twice.



Stephen Cook
Toronto, 1971



Leonid Levin
Moscow, 1973

So SAT isn't just a hard problem to study – in a precise sense, it is the hard problem.

Coming up: Resolution, and the algorithms powering modern SAT solvers.

Clauses as sets and CNF formulas as set of sets

Recall from Lecture 7

- A clause $\ell_1 \vee \cdots \vee \ell_n$ is written as the set $\{\ell_1, \ldots, \ell_n\}$.
- A CNF formula $C_1 \wedge \cdots \wedge C_n$ as the set of clauses $\{C_1, \ldots, C_n\}$.
- Using commutativity, associativity and idempotence of $\vee, \wedge$.

Notation

We extend the definition of clauses to the empty set of literals: $\bot$ is the empty clause.

Satisfiability and unsatisfiability

Question: given a formula F, is it satisfiable?

- As we saw before, if F were in DNF this is easy, but converting to satisfiability-preserving DNF formula is hard!
- On the other hand we know that by Tseitin encoding, we can easily convert to a satisfiability-preserving CNF formula. Does this help?
- Eg., Is $F = (p \vee q \vee \neg r) \wedge \neg p \wedge (p \vee q \vee r) \wedge (p \vee \neg q)$ satisfiable?
- Recall that the naive algo requires to examine all lines of truth table.
- If satisfiable we stop at some point, but if unsatisfiable we have explore all assignments!

Can we do better using the CNF structure?

- F is unsatisfiable if... we can derive $\bot$ from this set of clauses.

  E.g., $p \wedge \neg p$ gives clauses $\{p\}$ and $\{\neg p\}$, which combine to give $\bot$.
- If F is in CNF, then we can interpret F as a set of clauses. Does this help?

Derivations from CNF formulas

So, Goal is to derive $\bot$ from set of clauses of a CNF formula

- how many rules do we need?
- Recall Natural Deduction had 12 primitive rules to try (plus a few handy derived ones)!
- To automate, we want as few choices as possible...

Answer: We need only one ~~ring to~~ rule ~~them all~~

$$\textsc{Resolution} \quad \frac{F \vee G \qquad \neg F \vee H}{G \vee H}$$

Resolution proof rule

In fact, we apply resolution rule on clauses:

$$\frac{p \vee C \qquad \neg p \vee D}{C \vee D}$$

- clauses $p \vee C$ and $\neg p \vee D$ are called antecedents
- p is a variable called pivot
- $C \vee D$ is a clause too, called resolvent

E.g.: I drink coffee, or I fall asleep in class. I do not drink coffee, or I understand the lecture. Therefore, I fall asleep in class, or I understand the lecture.

Resolution

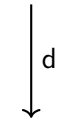
Main idea to check (un)satisfiability of formula F

- Convert F to CNF form as set of clauses (Use Tseitin Encoding!).
- Apply resolution rule and add resolvent to set of clauses, until either $\bot$ is derived or no more applications possible.

Let's try an example

Consider $F = (p \vee q) \wedge (\neg p \vee q) \wedge (\neg q \vee r) \wedge \neg r$
Writing as set of clauses: $\{(p \vee q), (\neg p \vee q), (\neg q \vee r), \neg r\}$

$$\dfrac{\dfrac{\dfrac{p \vee q \qquad \neg p \vee q}{q} \qquad \neg q \vee r}{r} \qquad \neg r}{\bot}$$



Hence we conclude that F is unsatisfiable! This is also called a Resolution Proof (of depth 3).

Resolution

Main idea to check (un)satisfiability of formula F

- Convert F to CNF form as set of clauses (Use Tseitin Encoding!).
- Apply resolution rule and add resolvent to set of clauses, until either $\perp$ is derived or no more applications possible.

Four Important Questions

1. Is this procedure correct/sound?
2. Does it terminate?
3. Can you also use it to show statements (not just to show unsat), e.g., $F \vdash G$?
4. Is it complete? That is, does it suffice to show unsatisfiability? i.e., when it terminates, if we didn't see $\perp$ does this imply F is satisfiable?

1. Soundness of propositional resolution

Theorem

The Resolution rule is sound! i.e., for any clauses C, D and var p: $(p \vee C) \wedge (\neg p \vee D) \models C \vee D$.

Proof 1: Try using natural deduction. **Exercise 8.1**

Proof 2: Can we directly show that $(p \vee C) \wedge (\neg p \vee D) \models C \vee D$

- Consider assignment π satisfying lhs.
- That is, $\pi \models p \vee C$ and $\pi \models \neg p \vee D$
- If $\pi(p) = 1$, then $\pi \models D$ implies $\pi \models C \vee D$.
- If $\pi(p) = 0$, then $\pi \models C$ implies $\pi \models C \vee D$.
- Thus $\pi \models C \vee D$, i.e., rhs.

Thus, we can repeatedly apply this rule and we have,

Corollary

If from F we obtain $\perp$ by a sequence of resolution rule applications then $F \models \perp$, i.e., F is unsat.

Exercise 8.2: Prove this by induction on the length of the resolution sequence.

Proof of Exercise 8.2: Soundness of the Resolution Procedure

Suppose $C_1, \ldots, C_n$ is a resolution sequence from F ending in $C_n = \bot$: each C_i is either

- a clause of F, or
- the resolvent of two earlier clauses C_j, C_k $(j, k < i)$ in the sequence.

Claim (strong induction on i)

For every assignment π with $\pi \models F$: $\pi \models C_i$, for all $i = 1, \ldots, n$.

1. **Base case:** $C_i \in F$.
   - Since $\pi \models F$, π satisfies every clause of F.
   - So $\pi \models C_i$.
2. **Inductive step:** C_i is the resolvent of C_j, C_k with $j, k < i$.
   - By the induction hypothesis (applied to $j, k < i$): $\pi \models C_j$ and $\pi \models C_k$.
   - By the (one-step) Soundness Theorem: $\pi \models C_j$ and $\pi \models C_k$ together imply $\pi \models C_i$.

Conclusion

- Taking $i = n$: if $\pi \models F$ then $\pi \models C_n = \bot$.
- But no assignment satisfies $\bot$ (the empty clause)!
- So no π satisfies F, i.e., $F \models \bot$, i.e., F is unsatisfiable. □

2. Termination of Resolution

Given formula F, we can write the resolution algo as:

Resolution Algorithm

1. Use Tseitin encoding to convert to F' in CNF form and write it as set of clauses.
2. Repeat till fixed point the following:
   - If possible, apply resolution and obtain new clause C i.e., $C \notin F'$. Set $F' := F' \cup \{C\}$.
3. At fixed point, check if $\bot \in F'$.
   - If yes, conclude F is UNSAT. (This follows from Soundness proof!)
   - Else, we wish to conclude that F is SAT. (To prove! - completeness)

But first, why does this terminate?

- Given F we know set of propositional variables is say n.
- So number of clauses possible is 3^n **Proof: Exercise 8.3!**.
- So after these many steps we must stop, as there are no new clauses for sure!

3. Using resolution to prove statements

We showed UNSAT using Resolution. Can we also show $F \vdash G$?

- Consider $F' = F \wedge \neg G$ in CNF as set of clauses.
- We apply the resolution proof method on F'.
- If we derive $\bot$ from F', then $F \models G$ (by Soundness of Resolution),
- and hence $F \vdash G$ is derivable (by Completeness of Natural Deduction, recall L5–L6!).

This is sometimes called a proof by refutation; such a proof system that works by deriving $\bot$ is a refutation proof system.

Exercise 8.4

Convert the above steps into a formal Natural Deduction proof. hint: Proof by contradiction!

Before going to completeness, is this algorithm deterministic?

Non-unique resolvents

Between two clauses we may need to choose the pivot to apply the resolution. We may have multiple choices applicable.

Example

The following resolutions are between two clauses, with different pivots

$$\frac{p \vee q \vee r \qquad \neg p \vee \neg q \vee r}{q \vee \neg q \vee r} \qquad \frac{p \vee q \vee r \qquad \neg p \vee \neg q \vee r}{p \vee \neg p \vee r}$$

- Thus, the algorithm is a non-deterministic proof search!
- Notice: both resolvents here are tautologies (each contains a variable and its negation) – always true, hence useless for deriving $\bot$.
- In practice, we focus on trying to ensure that we do not do unnecessary resolutions!
- However, note that we only have a single rule (resolution) instead of all of ND.
- Thus, we may be more effective in finding the proof strategy. This is what SAT solvers do!

Exercises

Exercise 8.5. Show the following formulas using proof by resolution.

1. Show that $(p \vee q \vee \neg r) \wedge \neg p \wedge (p \vee q \vee r) \wedge (p \vee \neg q)$ is unsatisfiable.
2. Prove that $(p \to q) \wedge (q \to r) \vdash (p \to r)$ is valid.

Solutions: Exercise 8.5

(1) $F = (p \vee q \vee \neg r) \wedge \neg p \wedge (p \vee q \vee r) \wedge (p \vee \neg q)$

$$\dfrac{\dfrac{\dfrac{\dfrac{p \vee q \vee \neg r \quad p \vee q \vee r}{p \vee q} \quad \neg p}{q} \quad p \vee \neg q}{p} \quad \neg p}{\bot}$$

Hence F is unsatisfiable (depth 4).

(2) $(p \to q) \wedge (q \to r) \vdash (p \to r)$?
By refutation: negate the goal.
$F' = (p \to q) \wedge (q \to r) \wedge \neg(p \to r)$
But $\neg(p \to r) \equiv p \wedge \neg r$, giving clauses $\{(\neg p \vee q),\ (\neg q \vee r),\ p,\ \neg r\}$

$$\dfrac{\dfrac{\dfrac{\neg p \vee q \quad p}{q} \quad \neg q \vee r}{r} \quad \neg r}{\bot}$$

Hence unsatisfiable, so $(p \to q) \wedge (q \to r) \models (p \to r)$, i.e., $(p \to q) \wedge (q \to r) \vdash (p \to r)$. □

4. Completeness

- Is resolution proof system, i.e., proof system where the only rule is Resolution, complete?
- In other words, if F is unsatisfiable, are we guaranteed to derive $F \vdash \bot$ via resolution?
- We need to talk about the set of clauses derived by resolution...

Clauses derivable with proofs of depth at most n

We define the set $Res^n(F)$ of clauses that are derivable via resolution proof of depth at most n from the set of clauses F. Formally, given set F of clauses, we define

- $Res^0(F) := F$,
- $Res^{n+1}(F) := Res^n(F) \cup \{C | C \text{ is a resolvent of clauses } C_1, C_2 \in Res^n(F)\}$

Example

Let $F = \{(p \vee q), (\neg p \vee q), (\neg q \vee r), \neg r\}$.

$Res^0(F) = F$

$Res^1(F) = F \cup \{q, p \vee r, \neg p \vee r, \neg q\}$

$Res^2(F) = Res^1(F) \cup \{r, q \vee r, p, \neg p, \bot\}$

$Res^3(F) = Res^2(F)$.

All derivable clauses

Since there are only finitely many variables in F, we can only derive finitely many clauses, so $Res^n(F)$ must saturate at some time point. (Same argument as termination of algo!)

- Let F be a set of clauses. Then there exists m such that $Res^{m+1}(F) = Res^m(F)$.
- We define $Res^*(F) \triangleq Res^m(F)$.
- Note: Res^* only grows as we add more clauses – if $F \subseteq F'$ then $Res^*(F) \subseteq Res^*(F')$. We will call this weakening.

We will now show the main theorem:

Theorem

If a finite set of clauses F is unsatisfiable, $\bot \in Res^*(F)$.

Finiteness matters: for infinite sets of clauses this needs compactness, which we will not cover here.

Completeness

Theorem

If a finite set of clauses F is unsatisfiable, $\perp \in Res^*(F)$.

Proof.

We prove the theorem using induction over number of propositional variables in F.
Wlog, we assume that there are no tautology clauses in F. **Exercise 8.6: Show why we can assume this!**

base case:

p is the only variable in F.
Now, if F is unsatisfiable, then either $\perp \in F$ already (trivial: $\perp \in Res^0(F)$), or we must have $\{p, \neg p\} \subseteq F$.
In the latter case, applying the resolution rule with $C = D = \emptyset$, we have the following resolution derivation of $\perp$.

$$\frac{p \qquad \neg p}{\perp}$$

...

CS228 : Logic for Computer Science

Module 2: The Problem of Satisfiability

Lecture 9: The DPLL Algorithm

Instructor: S. Akshay

IIT Bombay, India

Logistics

Quiz 1 (reminder)

- Aug 28th 5.10pm
- Venue: LH 101, 102,
  LT 105 (for PwD students who wish to avail compensatory time.)
- Syllabus: All of Module 1 (Propositional Logic), up to and including Tseitin's Encoding.

Topics Covered till date

Module 1: Propositional Logic

1. Syntax and Semantics
   - Questions of interest: Satisfiability, Validity
2. Formal proof systems
   - Natural Deduction
   - Soundness and Completeness
3. Normal Forms
   - Negation, Conjunctive and Disjunctive Normal Forms (NNF, CNF, DNF)
   - Tseitin's Encoding

Module 2: The Problem of Satisfiability

- Propositional Resolution - soundness, termination and completeness
- Today: wrapping up completeness, the DPLL algorithm

A Quick Recap: Propositional Resolution

$$\frac{p \vee C \qquad \neg p \vee D}{C \vee D}$$

- Resolution rule: $p \vee C$ and $\neg p \vee D$ are clauses, p is pivot, clause $C \vee D$ resolvent.

Resolution as a Saturation Algorithm

Given propositional logic formula F:

1. Use Tseitin encoding to convert to F' in CNF form and write it as set of clauses.
2. Repeat till possible, the following:
   - Apply Resolution Rule to two clauses in F' to obtain new clause $C \notin F'$. Update $F' := F' \cup \{C\}$.
3. At fixed point (i.e., when set F' does not change), answer YES if $\bot \in F'$, else answer NO.

Properties

- The algo is sound, complete, always terminates and non-deterministic.
- It can be lifted to proof by refutation/contradiction of $F \vdash G$.

4. Completeness

- Is resolution proof system, i.e., proof system where the only rule is Resolution, complete?
- In other words, if F is unsatisfiable, are we guaranteed to derive $F \vdash \bot$ via resolution?

$Res^n(F)$: set of clauses derivable via resolution proof of depth at most n from clauses in F.

- $Res^0(F) := F$, $Res^{n+1}(F) := Res^n(F) \cup \{C | C \text{ is a resolvent of clauses } C_1, C_2 \in Res^n(F)\}$

$Res^*(F)$: the fixed point

Since there are only finitely many variables in F, $Res^n(F)$ must saturate at some point: there exists m such that $Res^{m+1}(F) = Res^m(F)$. We define $Res^*(F) \triangleq Res^m(F)$.

Example

Let $F = \{(p \vee q), (\neg p \vee q), (\neg q \vee r), \neg r\}$.

$Res^0(F) = F$

$Res^1(F) = F \cup \{q, p \vee r, \neg p \vee r, \neg q\}$

$Res^2(F) = Res^1(F) \cup \{r, q \vee r, p, \neg p, \bot\}$

$Res^3(F) = Res^2(F)$ – saturated! $\Rightarrow Res^*(F) = Res^3(F)$.

Completeness: Main Theorem

Theorem

If a finite set of clauses F is unsatisfiable, $\perp \in Res^*(F)$.

Proof.

We prove the theorem using induction over number of propositional variables in F.
Wlog, we assume that there are no tautology clauses in F.

base case:

p is the only variable in F.
Now, if F is unsatisfiable, then either $\perp \in F$ already (trivial: $\perp \in Res^0(F)$), or we must have $\{p, \neg p\} \subseteq F$.
In the latter case, applying the resolution rule with $C = D = \emptyset$, we have the following resolution derivation of $\perp$.

$$\frac{p \qquad \neg p}{\perp}$$

Completeness - 2 (contd.)

Proof(contd.)

induction step:

Assume: theorem holds for all sets of clauses containing at most n propositions.

Consider an unsatisfiable set F of clauses containing propositions $p_1, \ldots p_n, p$.

We define

- $F_p \triangleq$ set of clauses from F that have p.
- $F_{np} \triangleq$ set of clauses from F that have $\neg p$.
- $F_o \triangleq$ set of clauses from F that have neither p nor $\neg p$.

$F = F_p \wedge F_{np} \wedge F_o$

Furthermore, let

- $F'_p \triangleq \{C - \{p\} | C \in F_p\}$, i.e., from each clause in F_p drop the p!
- $F'_{np} \triangleq \{C - \{\neg p\} | C \in F_{np}\}$

Exercise 9.1 ...

Using natural deduction or truth tables, show that $F'_p \models F_p$ and $F'_{np} \models F_{np}$

Completeness - 3 (contd.)

Proof(contd.)

Now consider the formula $G = \underbrace{(F'_p \wedge F_o) \vee (F'_{np} \wedge F_o)}$

Note! p does not appear in this formula

Claim: If G is satisfiable then F is satisfiable.

- Consider a satisfying assignment π of G, i.e., $\pi \models G$. π is defined on $p_1, \ldots p_n$
- Case 1: $\pi \models (F'_p \wedge F_o)$. Let's call this G_1. Consider assignment π', $\pi'(p) = 0$, $\pi'(p') = \pi(p')$ for all $p' \in \{p_1, \ldots p_n\}$.
  - Observe that F'_p and F_o have no p, hence $\pi' \models F'_p$ and $\pi' \models F_o$.
  - Now, since $F'_p \models F_p$, we get $\pi' \models F_p$.
  - Finally, all clauses of F_{np} have $\neg p$ appearing in them, hence $\pi' \models F_{np}$.
  - Therefore we conclude that $\pi' \models F_p \wedge F_{np} \wedge F_o$, i.e., $\pi' \models F$.
- Case 2: $\pi \models (F'_{np} \wedge F_o)$. Call this G_2 **Exercise 9.2: Argue symmetrically that π' with $\pi'(p) = 1$ will satisfy F in this case.**
- Therefore, in both cases we obtain that $F = F_p \wedge F_{np} \wedge F_o$ is satisfiable. ...

Completeness - 4 (contd.)

Proof(contd.)

- As we started with F unsat, we obtain $G = G_1 \vee G_2 = (F'_p \wedge F_o) \vee (F'_{np} \wedge F_o)$ is unsat. (why?)
- Now we apply the induction hypothesis on G_1 and G_2 (which do not contain p).
- Thus we obtain, $\perp \in Res^*(G_1)$ and $\perp \in Res^*(G_2)$.
- Take any derivation ρ_1 of $\perp$ from G_1 and a derivation ρ_2 of $\perp$ from G_2. Two cases:
  - Case 1: $\perp$ was derived using only clauses from F_o in ρ_1 or ρ_2. In this case $\perp \in Res^*(F_o)$. This implies $\perp \in Res^*(F_p \wedge F_{np} \wedge F_o)$ (by weakening: if $F \subseteq F'$ then $Res^*(F) \subseteq Res^*(F')$).
  - Case 2: In ρ_1 a clause from F'_p is involved AND in ρ_2 a clause from F'_{np} is involved.
    - In derivation ρ_1, p does not occur, hence p cannot get resolved. So, if we put back p in the clauses of ρ_1, this derivation of $\perp$ will now give a derivation of p!
    - Similarly, if we put back $\neg p$ in the clauses of ρ_2, the derivation of $\perp$ will give a derivation of $\neg p$!
    - Thus, we obtain $p \in Res^*(F_p \wedge F_o)$ and $\neg p \in Res^*(F_{np} \wedge F_o)$, which (again by weakening) implies $p, \neg p \in Res^*(F_p \wedge F_{np} \wedge F_o)$, and hence $\perp \in Res^*(F_p \wedge F_{np} \wedge F_o)$.

Thus in both cases, $\perp \in Res^*(F)$ which completes the induction step, and hence the proof.

□

An example of the proof of completeness of resolution

Let $F = \{p_1 \vee p, \quad p_2, \quad \neg p_1 \vee \neg p_2 \vee p, \quad \neg p_2 \vee \neg p\}$

- Step 1: $F_p = \{p_1 \vee p, \neg p_1 \vee \neg p_2 \vee p\}$ $\quad F_p' = \{p_1, \neg p_1 \vee \neg p_2\}$ $\quad F_{np} = \{\neg p_2 \vee \neg p\}$ $\quad F_{np}' = \{\neg p_2\}$ $\quad F_o = \{p_2\}$
- Step 2: $G = G_1 \vee G_2 = (F_p' \wedge F_o) \vee (F_{np}' \wedge F_o) = \{p_1, \neg p_1 \vee \neg p_2, p_2\} \vee \{\neg p_2, p_2\}$
- Step 3: Proofs from G_1 to $\bot$ and G_2 to $\bot$
- Step 4: Convert to proof from F to $\bot$ by adding p and $\neg p$ (Note that we are in Case 2)

$$\dfrac{\dfrac{\dfrac{p_1 \vee \mathbf{p} \quad \neg p_1 \vee \neg p_2 \vee \mathbf{p}}{\neg p_2 \vee \mathbf{p}} \qquad p_2}{\bot \vee \mathbf{p}} \qquad \dfrac{\neg p_2 \vee \neg \mathbf{p} \qquad p_2}{\bot \vee \neg \mathbf{p}}}{\bot}$$

Exercises

Exercise 9.3

As in the previous slide, work out the proof of completeness for another set of unsatisfiable clauses namely $F_1 = \{\neg p_1 \vee \neg p_2 \vee \neg p_4, \neg p_2 \vee p_4, \neg p_1 \vee p_2, p_1\}$.

Exercise 9.4

Let F be an unsatisfiable CNF formula with n propositions. Show that there is a resolution proof of $\perp$ from F of size at most $2^{n+1} - 1$ (size = number of clause-occurrences/nodes in the derivation tree).

Tips and Tricks in resolution

Unit Clauses

- Clauses where there is only one literal.

Pure literals

- Literals whose negation does not occur in the formula, i.e., the corresponding variable occurs throughout only "positively" or "negatively" but not both.
  e.g., p and $\neg q$ are pure in $F_2 = (p \vee r \vee \neg q) \wedge (p \vee \neg r) \wedge (\neg q \vee \neg r) \wedge (\neg q \vee r)$ (q occurs only negatively) but r is not.
  - If ℓ is a pure literal, then F is sat iff $F[\ell \mapsto 1]$ is sat.
  - e.g., F_2 is sat iff $F_2[p \mapsto 1, q \mapsto 0]$ is sat.

Tips and Tricks in resolution

Unit Clauses

- Clauses where there is only one literal.

Pure literals

- Literals whose negation does not occur in the formula.

Exercise 9.5 Consider $F_3 = (p_1 \vee p_2) \wedge (p_3 \vee p_2 \vee \neg p_1) \wedge p_3 \wedge (\neg p_4 \vee p_2)$

1. What are the unit clauses in F_3, if any?
2. What are the pure literals in F_3, if any?

Topic 9.1

DPLL

Partial Assignments

Another Idea: Can we build satisfying assignments incrementally?

- A partial assignment is partial map from **AP** to $\{0, 1\}$.
  e.g, if $\mathbf{AP} = \{p_1, p_2, p_3\}$, $\pi_1 = \{p_1 \mapsto 0, p_2 \mapsto 1\}$ is a partial assignment.
- Under partial assignment π,
  - literal ℓ is true if $\pi(\ell) = 1$ and ℓ is false if $\pi(\ell) = 0$. Otherwise, ℓ is unassigned.
    - e.g., under π_1, the states of literals $p_1, p_2, p_3, \neg p_1$ are 0, 1, unassigned, 1, respectively.
  - clause C is true if there is $\ell \in C$ such that ℓ is true and C is false if for each $\ell \in C$, ℓ is false.Otherwise, C is unassigned.
    - e.g., under π_1, $p_1 \vee p_2 \vee p_3$ is true while $p_1 \vee \neg p_2$ is false and $p_1 \vee p_3$ is unassigned.
  - CNF formula F is true if for each $C \in F$, C is true and F is false if there is $C \in F$ such that C is false. Otherwise, F is unassigned.
  - **Exercise 9.6** Under π_1, what is the status of the following formulas?
    - $(p_3 \vee \neg p_1) \wedge (p_1 \vee \neg p_2)$ is false.
    - $(p_1 \vee p_2 \vee p_3) \wedge \neg p_1$ is true.
    - $p_1 \vee p_3$ is unassigned.
    - $\emptyset$ (empty formula) is true.

Lifting definition of unit clauses to partial assignments

Definition (Unit clause)

C is a unit clause under π if a literal $\ell \in C$ is unassigned and the rest are false. Also, ℓ is called unit literal.

Exercise 9.7

Consider again, partial assignment $\pi_1 = \{p_1 \mapsto 0, p_2 \mapsto 1\}$. Which of the following are unit clauses under π_1? Identify the unit literals.

- $p_1 \vee \neg p_3 \vee \neg p_2$ ✓
- $p_1 \vee \neg p_3 \vee p_2$ ✗
- $p_1 \vee \neg p_3 \vee p_4$ ✗
- $p_1 \vee \neg p_2$ ✗

DPLL (Davis-Putnam-Loveland-Logemann) method

- takes CNF input
- maintains a partial assignment, initially $\emptyset$
- assigns unassigned variables 0 or 1 randomly one after another
- sometimes forced to choose assignments due to unit/pure literals(why?)



Martin Davis



Hilary Putnam



George Logemann



Donald Loveland

A bit of history: Davis & Putnam's original 1960 algorithm eliminated variables one at a time using resolution – exactly the $F_p/F_{np}/F_o$ split from our completeness proof! It then resolved every clause in F_p against every clause in F_{np} on pivot p, replacing F with F_o plus all these resolvents. Davis, Logemann & Loveland's 1962 fix replaced that (expensive) elimination with the backtracking search as we shall see next.

DPLL

Algorithm 9.1: DPLL(F)

Input: CNF F **Output:** sat/unsat
return $DPLL(F, \emptyset)$

Algorithm 9.2: DPLL(F,π)

Input: CNF F, partial assignment π **Output:** sat/unsat
if *F is true under π (i.e., all clauses of F are true)* **then return** *sat* ;
if *F is false under π (some clause of F is false, i.e., conflict)* **then return**
if *$\exists$ unit/pure literal ℓ under π* **then**
  return $DPLL(F, \pi[\ell \mapsto 1])$
Choose an unassigned variable p and $\in \{0, 1\}$;
if $DPLL(F, \pi[p \mapsto b]) == sat$ **then**
  return *sat*
else
  return $DPLL(F, \pi[p \mapsto 1 - b])$

Backtracking at conflict

Unit propagation

Decision

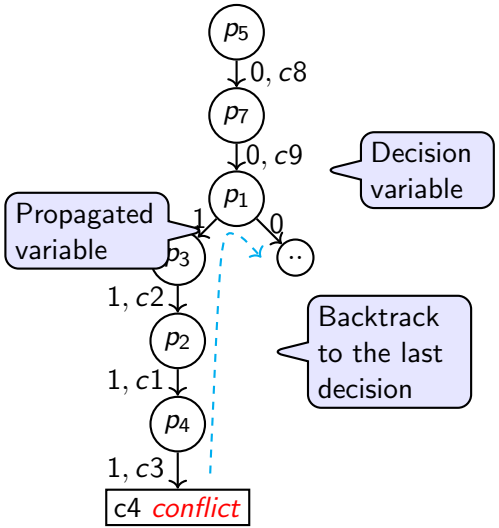
Three actions of DPLL

A DPLL run consists of three types of actions

- Decision
- Unit propagation
- Backtracking

Example: decide, propagate, and backtrack in DPLL

$$c_1 = (\neg p_1 \vee p_2)$$
$$c_2 = (\neg p_1 \vee p_3 \vee p_5)$$
$$c_3 = (\neg p_2 \vee p_4)$$
$$c_4 = (\neg p_3 \vee \neg p_4)$$
$$c_5 = (p_1 \vee p_5 \vee \neg p_2)$$
$$c_6 = (p_2 \vee p_3)$$
$$c_7 = (p_2 \vee \neg p_3 \vee p_7)$$
$$c_8 = (\neg p_5)$$
$$c_9 = (\neg p_7)$$



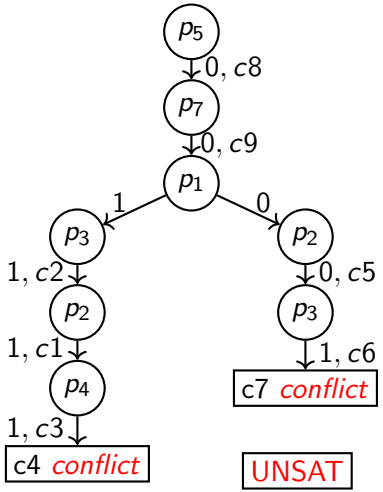
Blue : causing unit propagation

Green/Blue : true clause

Exercise 9.8

Complete the DPLL run

Solution: Exercise 9.8 – the completed DPLL run



p_1 was the only decision – both values fail

Exercise 9.9 Now that you know $c_1, \ldots, c_9$ is UNSAT, derive a resolution proof of $\bot$ from these clauses.

CS228 : Logic for Computer Science

Module 2: The Problem of Satisfiability

Lecture 10: Clause Learning and the CDCL Algorithm

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax and Semantics
   - Questions of interest: Satisfiability, Validity
2. Formal proof systems
   - Natural Deduction
   - Soundness and Completeness
3. Normal Forms
   - Negation, Conjunctive and Disjunctive Normal Forms (NNF, CNF, DNF)
   - Tseitin's Encoding

Module 2: The Problem of Satisfiability

1. Propositional Resolution - soundness, termination and completeness
2. The DPLL algorithm
3. Today: clause learning, the CDCL algorithm, and modern SAT solvers

A Quick Recap: DPLL

A DPLL run makes a sequence of

- decisions: choose an unassigned variable p and a bit b, set $\pi(p) = b$
- unit propagations: if clause $C \in F$ has exactly one unassigned literal ℓ and all its other literals are false under π, set $\pi(\ell) = 1$
- backtracking: on conflict (some clause of F false under π), undo the last decision and try the other bit

Today

We open up DPLL's backtracking step: instead of just undoing the last decision, learn from a conflict which decisions actually caused it and jump back further – this is clause learning, and DPLL + clause learning is CDCL.

Implementation and Optimizations

In fact, in DPLL in practice

- The "pure" literal check is not done (too expensive!)
- Branching literal is not chosen randomly!
- Algo is implemented iteratively; with backtrack stack maintained explicitly! (may help to backtrack more than one level!)

Further, DPLL allows many optimizations and improvements.

- **clause learning**
- 2-watched literals
- ...

We will look at one such optimization – clause learning – in detail, next.

Topic 10.1

Clause learning

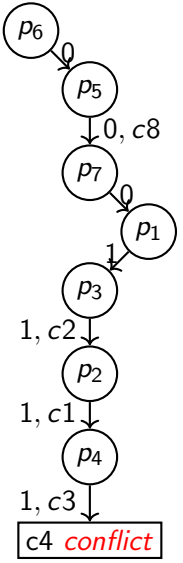
Revisiting DPLL

Exercise 10.1: run DPLL on this set of clauses, deciding p_6, p_7, p_1 in that order: p_6=0, p_7=0, p_1=1

$$c_1 = (\neg p_1 \vee p_2)$$
$$c_2 = (\neg p_1 \vee p_3 \vee p_5)$$
$$c_3 = (\neg p_2 \vee p_4)$$
$$c_4 = (\neg p_3 \vee \neg p_4)$$
$$c_5 = (p_1 \vee p_5 \vee \neg p_2)$$
$$c_6 = (p_2 \vee p_3)$$
$$c_7 = (p_2 \vee \neg p_3 \vee p_7)$$
$$c_8 = (p_6 \vee \neg p_5)$$

Blue : causing unit propagation

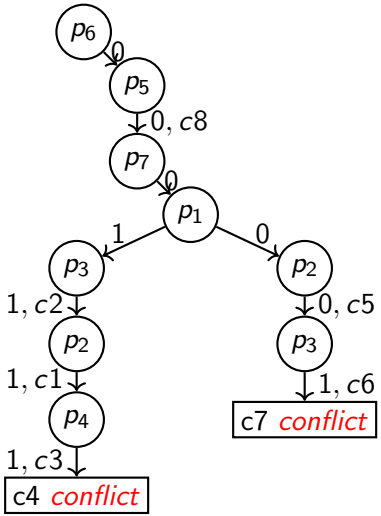
Green/Blue : true clause



Backtrack: both values of p_1 fail

$$c_1 = (\neg p_1 \vee p_2)$$
$$c_2 = (\neg p_1 \vee p_3 \vee p_5)$$
$$c_3 = (\neg p_2 \vee p_4)$$
$$c_4 = (\neg p_3 \vee \neg p_4)$$
$$c_5 = (p_1 \vee p_5 \vee \neg p_2)$$
$$c_6 = (p_2 \vee p_3)$$
$$c_7 = (p_2 \vee \neg p_3 \vee p_7)$$
$$c_8 = (p_6 \vee \neg p_5)$$

Blue : causing unit propagation
Green/Blue : true clause
Red : conflict clause

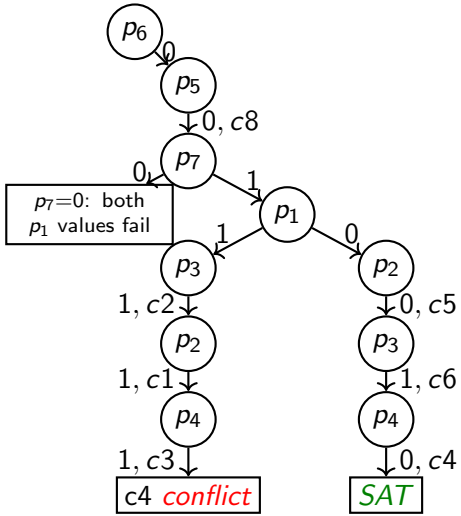


Both values of p_1 fail – backtrack further, to p_7

The complete search tree

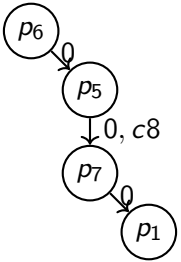
$$c_1 = (\neg p_1 \vee p_2)$$
$$c_2 = (\neg p_1 \vee p_3 \vee p_5)$$
$$c_3 = (\neg p_2 \vee p_4)$$
$$c_4 = (\neg p_3 \vee \neg p_4)$$
$$c_5 = (p_1 \vee p_5 \vee \neg p_2)$$
$$c_6 = (p_2 \vee p_3)$$
$$c_7 = (p_2 \vee \neg p_3 \vee p_7)$$
$$c_8 = (p_6 \vee \neg p_5)$$

Green : true clause
Red : conflict clause



Decision level

Consider just the partial run so far, up to p_1.



Definition

The decision level of a true literal is the number of decisions after which the literal was made true.

Given the above run, we write $\neg p_6@1$ and $\neg p_5@1$ (p_5 was propagated while only one decision had been made), $\neg p_7@2$.

Example: implication graph

$c_1 = (\neg p_1 \vee p_2)$

$c_2 = (\neg p_1 \vee p_3 \vee p_5)$

$c_3 = (\neg p_2 \vee p_4)$

$c_4 = (\neg p_3 \vee \neg p_4)$

$c_5 = (p_1 \vee p_5 \vee \neg p_2)$

$c_6 = (p_2 \vee p_3)$

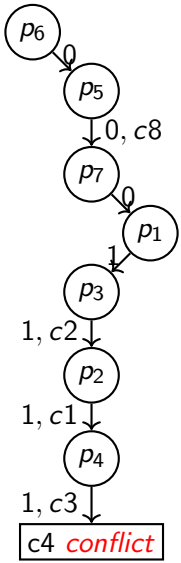
$c_7 = (p_2 \vee \neg p_3 \vee p_7)$

$c_8 = (p_6 \vee \neg p_5)$

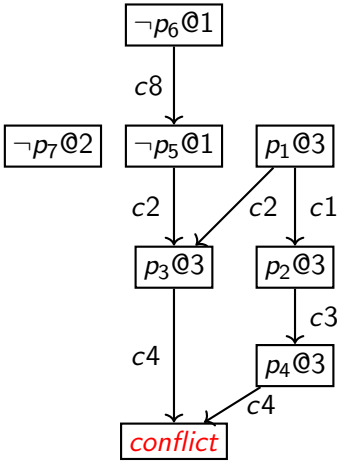
Blue : causing unit propagation

Green/Blue : true clause

Red : conflict clause



Implication graph



Implication graph

During the DPLL run, we maintain the following data structure.

Under a partial assignment π, the implication graph is a labeled DAG (V, E), where

- V is the set of true literals under π and a conflict node
- $E = \{(\ell_1, C, \ell_2) | \ell_2$ was produced by unit propagation at C, with $\ell_2, \neg\ell_1 \in C\}$.

We also annotate each node with its decision level. The decision level of a true literal is the number of decisions after which the literal was made true.

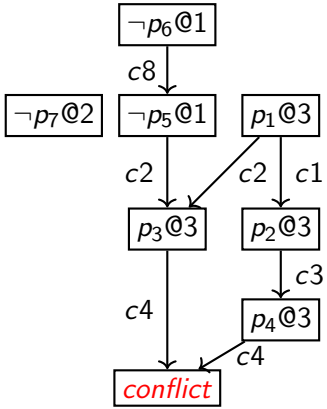
So what is the use of drawing this implication graph?

Conflict clause

We traverse the implication graph backwards to find the set of decisions that caused the conflict.

Definition

The clause of the negations of the causing decisions is called conflict clause.



Conflict clause : $p_6 \vee \neg p_1$

So what do we do with this learned conflict clause?
Add it to our formula!

What happens when we add the learned clause?

$c_1 = (\neg p_1 \vee p_2)$

$c_2 = (\neg p_1 \vee p_3 \vee p_5)$

$c_3 = (\neg p_2 \vee p_4)$

$c_4 = (\neg p_3 \vee \neg p_4)$

$c_5 = (p_1 \vee p_5 \vee \neg p_2)$

$c_6 = (p_2 \vee p_3)$

$c_7 = (p_2 \vee \neg p_3 \vee p_7)$

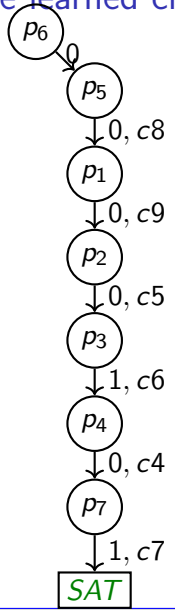
$c_8 = (p_6 \vee \neg p_5)$

$c_9 = (p_6 \vee \neg p_1)$

Blue : causing unit propagation

Green/Blue : true clause

Violet : the learned clause



p_6 is the only decision – everything else is forced!

Clause learning

Clause learning heuristics

- add conflict clause in the input clauses and
- backtrack to just after the second last conflicting decision, and proceed like DPLL

Lemma

Let C be the conflict clause. Then

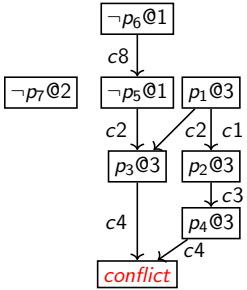
1. $F \models C$ (so adding C does not change the set of satisfying assignments)
2. the conflicting partial assignment will never be tried again

Exercise 10.2

1. Prove the above Lemma. (Hint: for part 1, show $F \wedge \neg C$ is unsat.)
2. Can you prove $F \vdash C$ using resolution?

Implication graphs and resolution proofs!

$c_1 = (\neg p_1 \vee p_2)$
$c_2 = (\neg p_1 \vee p_3 \vee p_5)$
$c_3 = (\neg p_2 \vee p_4)$
$c_4 = (\neg p_3 \vee \neg p_4)$
$c_5 = (p_1 \vee p_5 \vee \neg p_2)$
$c_6 = (p_2 \vee p_3)$
$c_7 = (p_2 \vee \neg p_3 \vee p_7)$
$c_8 = (p_6 \vee \neg p_5)$



We show $F \wedge \neg C$ is unsat, i.e. derive $\perp$ from F assuming $\neg p_6$ and p_1 – retracing the graph above, edge by edge.

$$\dfrac{\dfrac{\dfrac{\neg p_1 \vee p_3 \vee p_5}{(c_2)} \quad p_1}{p_3 \vee p_5} \qquad \dfrac{p_6 \vee \neg p_5}{(c_8)} \quad \neg p_6}{p_3} \qquad \dfrac{\neg p_3 \vee \neg p_4}{(c_4)}}{\neg p_4} \qquad \dfrac{\dfrac{\neg p_1 \vee p_2}{(c_1)} \quad p_1}{p_2} \qquad \dfrac{\neg p_2 \vee p_4}{(c_3)}}{p_4}$$

$$\dfrac{\neg p_4 \qquad p_4}{\perp}$$

Benefit of adding conflict clauses

1. Prunes away search space
2. Records past work of the SAT solver

For example,

In the previous example, we made decisions : $m(p_6) = 0$, $m(p_7) = 0$, and $m(p_1) = 1$

We learned a conflict clause : $p_6 \vee \neg p_1$

Adding this clause to the input clauses results in

- $m(p_6) = 0$, $m(p_7) = 1$, and $m(p_1) = 1$ will never be tried
- In fact, $m(p_6) = 0$ and $m(p_1) = 1$ will never occur simultaneously.

- There are many other clever choices for conflict clauses.
- DPLL+conflicts = CDCL (Conflict-driven clause learning!)

Algorithm 10.1: CDCL

Input: CNF F
$m := \emptyset$; $dl := 0$; $dstack := \lambda x.0$;
UnitPropagation(m, F);
do
  // backtracking
  while $m \not\models F$ **do**
    if $dl = 0$ **then return** *unsat*;
    $(C, dl) :=$ AnalyzeConflict(m, F);
    $m.resize(dstack(dl))$; $F := F \cup \{C\}$; m := UnitPropagation(m, F);
  // Boolean decision
  if m *is partial* **then**
    $dstack(dl) := m.size()$;
    $dl := dl + 1$; Decide(m, F);
while m *is partial or* $m \not\models F$;
return *sat*

dl stands for decision level

dstack records history for backtracking

- UnitPropagation(m, F) - applies unit propagation and extends m
- Decide(m, F) - chooses an unassigned variable in m and assigns a Boolean value
- AnalyzeConflict(m, F) - returns a conflict clause learned using implication graph and a decision level

Other improvements

Decide

Use involved heuristics to decide which unassigned variable to branch from.

- VSIDS: score each variable by how often it appears in recent conflicts (score decays over time); branch on the highest-scoring variable.

Exercise 10.3: for Exercise 10.1's clauses, show that deciding p_1 first reaches SAT with no conflict at all – decision order matters!

First UIP

Instead of tracing back to every causing decision, stop at the first node – through which all paths from the current decision to the conflict must pass.

Implementation-level improvements

- Clause deletion: learned clauses pile up fast – periodically discard the less useful ones to bound memory and keep propagation fast.

Resets

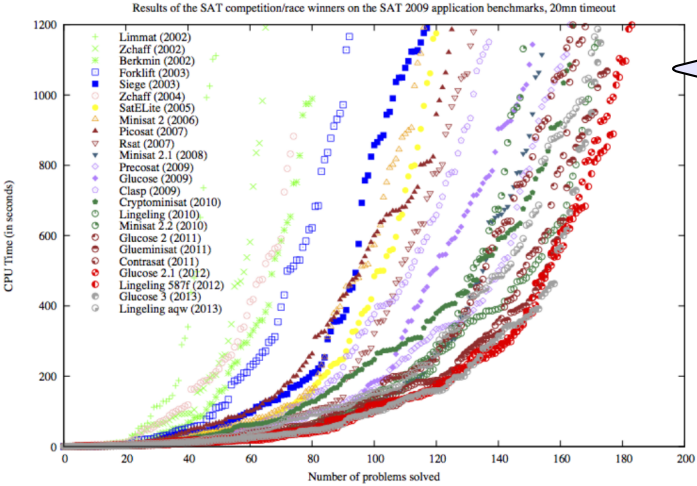
At some point give up and re-start!

Topic 10.2

SAT technology and its impact

Efficiency of SAT solvers over the years

(chart as of 2014 – solvers have improved substantially further since!)



Every solver here, from 2002 on, is a Chaff-lineage CDCL solver – this whole plot is watching CDCL refine itself for two decades!

Cactus plot:

X-axis: # problems solved

Y-axis: CPU time taken

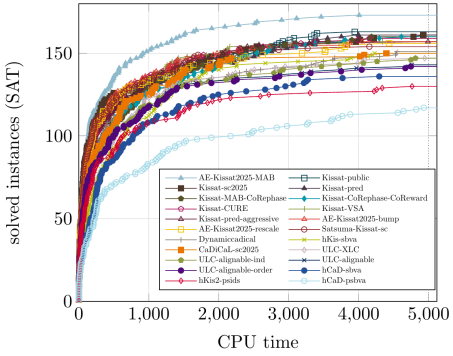
each curve = one solver

further right & lower = faster

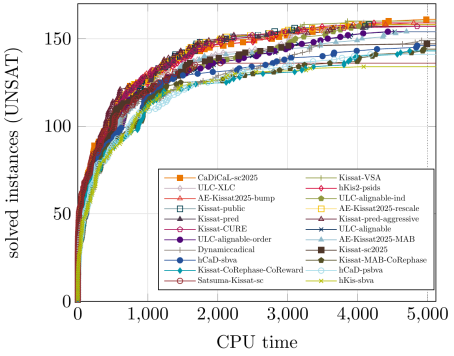
Source: `http://satsmt2014.forsyte.at/files/2014/07/SAT-introduction.pdf`

SAT solvers today: Competition 2025

Main (Sequential) Track SAT Plot



Main (Sequential) Track UNSAT Plot



Source: SAT Competition 2025 results, `https://satcompetition.github.io/2025/satcomp25slides.pdf` Today's solvers routinely handle industrial instances with millions of variables – up from just hundreds in the pre-CDCL era; the largest published instances now reach into the billions of clauses. (e.g. A. Nadel, *Solving Huge Instances with Intel SAT Solver*, SAT 2023.)

Impact of SAT technology

Impact is enormous.

A few are listed below

- Hardware verification and design assistance
  Almost all hardware/EDA companies have their own SAT solver
- AI/Planning: many resource allocation problems are convertible to SAT
- Security: analysis of crypto algorithms
- Solving hard problems, *e. g.*, travelling salesman problem

Next class

We will see a SAT solver in action! Come with your laptop, with a solver installed – instructions will be given on Piazza.

Solving Puzzles with SAT Solvers

Stop writing algorithms. Start stating the rules.

Meet Z3

What on Earth is Z3?

- A SAT solver (and more!) from Microsoft Research
- Solves boolean satisfiability problems: can we make these constraints true?
- You describe **what** must be true
- Z3 works out **how** — or proves it's impossible
- *(Bonus: Z3 also handles integers, reals, arrays, etc. — it's a full SMT solver! You will learn this later.)

You, the wizard

"Exactly one room hides the Lady. Exactly one sign on the doors is telling the truth."

Z3, the genie

"Then she is behind door one." ✓

```
[R1 = True, R2 = False, R3 = False,
...]
```

Declarative, not procedural

Normal programming

- You write the *steps*
- **You** are the algorithm designer
- Loops, conditionals, backtracking...
- Bug in your search $\Rightarrow$ wrong answer

Z3 programming

- You write the *constraints*
- **Z3** is the algorithm designer
- You just state the rules of the game
- Rules right $\Rightarrow$ answer right

You bring the rules.

Z3 brings the answer.

Under the Hood: SAT Solving Algorithms

DPLL Algorithm

You've learned this in class!

- Backtracking search
- Unit propagation
- Pure literal elimination

Modern SAT Solvers (CDCL)

What Z3 actually uses:

- DPLL + learning from conflicts
- When you hit a dead end, analyze *why*
- Add learned clauses to prevent revisiting mistakes
- Vastly faster than vanilla DPLL

Key insight: Z3 is essentially running CDCL in the background. Your constraints get translated into a boolean formula, then solved using highly optimized SAT-solving techniques.

The Z3 Recipe

Five steps. That's the whole API.



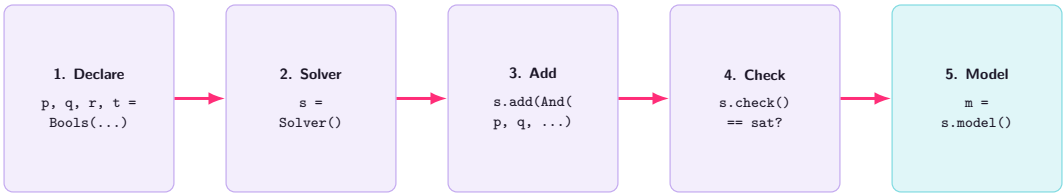
Next up: try the five steps on a quick example.

A quick exercise

Solve this formula:

$$(p \wedge q) \wedge (r \Leftrightarrow p) \wedge (q \Rightarrow t) \wedge (t \vee \neg p)$$

Walk through the five steps:



Propositional Logic

Propositional Logic 101

Building blocks

- Variables: $P, Q, R, \cdots \in \{\top, \bot\}$
  `Bool('P')`

Connectives

- $\neg P$ `Not(P)`
- $P \wedge Q$ `And(P, Q)`
- $P \vee Q$ `Or(P, Q)`
- $P \Rightarrow Q$ `Implies(P, Q)`
- $P \Leftrightarrow Q$ `P == Q`

Satisfiable (sat)

There *exists* an assignment of $\top/\bot$ making every formula true. Z3 hands you one.

Unsatisfiable (unsat)

No assignment works. Your constraints contradict — and Z3 has **proved** it, not merely failed to find one.

Demo 1: Lady or Tiger

An old familiar problem

A prisoner must choose one of three rooms. One holds the Lady; the other two hold tigers.

- Pick the Lady — he walks free
- Pick wrong — he gets eaten

Every door carries a sign, and:

exactly one sign is true

the other two are lies

Room 1

"The Lady is not in this room"

Room 2

"The Lady is not in this room"

Room 3

"The Lady is in Room 2"

Which door should he open?

Propositional encoding

Exactly one room has the Lady

$$(R_1 \wedge \neg R_2 \wedge \neg R_3) \vee (R_2 \wedge \neg R_1 \wedge \neg R_3) \vee (R_3 \wedge \neg R_1 \wedge \neg R_2)$$

Exactly one sign is true

$$(S_1 \wedge \neg S_2 \wedge \neg S_3) \vee (S_2 \wedge \neg S_1 \wedge \neg S_3) \vee (S_3 \wedge \neg S_1 \wedge \neg S_2)$$

What each sign claims

$$S_1 \Leftrightarrow \neg R_1 \quad S_2 \Leftrightarrow \neg R_2 \quad S_3 \Leftrightarrow R_2$$

The one pattern you'll use over and over

How do you say "exactly one of these is true"?

At least one

$$P_1 \vee P_2 \vee \cdots \vee P_n$$

`Or(P1, P2, ..., Pn)`

At most one

$$\bigwedge_{i<j} \neg(P_i \wedge P_j)$$

`Not(And(Pi, Pj))` for every pair

Exactly one = at least one $\wedge$ **at most one** — one queen per row, one digit per cell, one lady behind one door.

Demo 2: Sudoku

The Puzzle

- A 9×9 grid, digits 1–9
- Every row, column and 3×3 box holds each digit exactly once
- `grid[i][j][k]` — "digit $k{+}1$ sits in cell (i, j)"
- That is $9 \times 9 \times 9 = \mathbf{729}$ booleans

5	3			7				
6								
							8	
		4						
				9				

Constraint 1 : Only one digit in one cell

No two digits in this cell are the same

$$\neg(g_{ijk} \wedge g_{ijk'}) \quad \text{for all } k \neq k'$$

At least one digit sits in this cell

$$\bigvee_{k=1}^{9} g_{ijk}$$

Constraint 2 : No 2 digits in the same row should be the same

P1 : No duplicate digit in a row

$$\neg(g_{ijk} \wedge g_{ij'k}) \quad \text{for all } j \neq j'$$

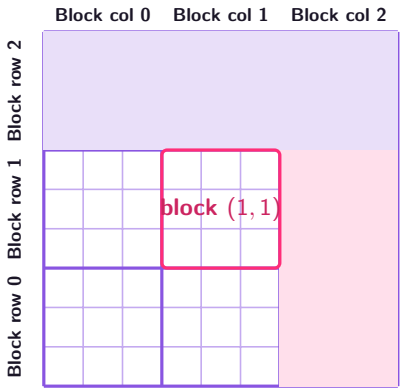
Constraint 3: No two digits in the same column be the same

P1 : No duplicate digit in a column

$$\neg(g_{ijk} \wedge g_{i'jk}) \quad \text{for all } i \neq i'$$

Blocks, block rows and block columns

- A block is a 3×3 subgrid.
- The three blocks in the same horizontal strip form a block row.
- The three blocks in the same vertical strip form a block column.
- We enforce that each digit appears exactly once inside every block.



Constraint 4: Each block doesn't have 2 digits of the same kind

P2 : No duplicate digit in the same block

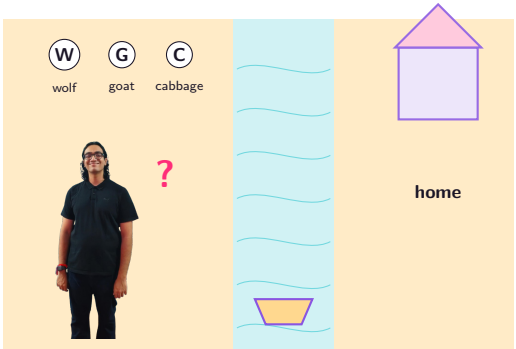
$$\neg(g_{ijk} \wedge g_{i'j'k}) \quad \text{for all } (i,j) \neq (i',j') \text{ in } B_{br,bc}$$

Exercise - 3

Exercise - 3 : Shivansh crosses the river

Farmer Shivansh is back from the market with a **Wolf**, a **Goat** and a **Cabbage**. His house is on the far bank. He must ferry all three across — and himself.

- The boat holds **Shivansh + at most one** item.
- Only Shivansh rows: he is in the boat *every* trip.
- Leave the **Wolf & Goat** alone ⇒ goat eaten.
- Leave the **Goat & Cabbage** alone ⇒ cabbage eaten.
- Safety must hold at **every** instant.



Exercise - 3 : the encoding you must produce

Two families of Booleans

- $L_{e,t}$: entity e is on the **left** bank at time t (right bank $= \neg L$)
- $M_{e,t}$: cargo e rides the boat on step $t \to t{+}1$

$e \in \{F, W, G, C\}, \quad t = 0 \ldots N.$

Five things the formula says

1. start: all on the left at $t{=}0$
2. goal: all on the right at $t{=}N$
3. boat: farmer flips banks; ≤ 1 cargo; boards from farmer's bank, ends across
4. frame: not carried $\Rightarrow$ bank unchanged
5. safety: no unwatched W/G or G/C pair, ever

Horizon $N = 7$ is satisfiable. Set $N = 6$ and Z3 *proves* no plan exists.

Run the independent checker

From the `river_crossing` directory: `python3 cross_river.py` then `python3 checker.py cross_river.py`

POP QUIZ !!

The Great Golgappa Challenge

The department is running a golgappa-eating contest: can you finish **30 golgappas** in one sitting? Five TAs entered — but rumour says not everyone has the stomach. Each Boolean is `True` iff that TA can eat 30. **Encode the clues and return the unique model.**

Var	TA
A	Tamanna
B	Shishir
C	Shivansh
D	Anshul
E	Shayna

1. `A` **iff** `E`. `(==)`
2. **At least one** of `A`, `E`. `(Or)`
3. **Exactly one** of `B`, `C`. `(Xor)`
4. **If** `C` **then not** `D`. `(Implies, Not)`
5. **Exactly one** of `C`, `D`. `(Xor)`
6. `C`, **or not** `D`. `(Or, Not)`
7. **Not both** `B` and `D`. `(Not, And)`

NOTE: do not change the variable names.

Fill `get_model()` in `pop/pop.py`, then run `python check.py` from that folder.

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 12: Syntax of Predicate Logic

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax and Semantics
   - Questions of interest: Satisfiability, Validity
2. Formal proof systems
   - Natural Deduction, Soundness and Completeness
3. Normal Forms
   - Negation, Conjunctive and Disjunctive Normal Forms (NNF, CNF, DNF)
   - Tseitin's Encoding

Module 2: The Problem of Satisfiability

1. Propositional Resolution - soundness, termination and completeness
2. The DPLL algorithm
3. Clause learning, the CDCL algorithm, and modern SAT solvers
4. A tool demo of SAT/SMT solvers in practice

Next Module: Predicate Logic

Propositional Logic was about simple statements and connectives

- e.g., If Osim is a student and Rowena is a teacher, then Osim is younger than Rowena.

Predicate logic goes beyond and talks about quantifiers and predicates

- e.g., Every student is younger than some teacher.
- What are the constituents of this sentence?
  - Being a student/teacher : these are called unary predicates.
    - e.g., S(Osim) will denote Osim is a student, T(Rowena) will denote Rowena is a teacher.
  - Being younger : these are called binary predicates.
    - e.g., Y(Osim, Rowena) will denote Osim is younger than Rowena
  - Every/some – these are quantifiers
  - Variables – to be able to say "$\forall\ x\ S(x)$" instead of writing each student out...
- e.g., $\forall x(S(x) \rightarrow \exists y(T(y) \wedge Y(x,y)))$ For all x, if x is a student, then there is some y which is a teacher such that x is younger than y.

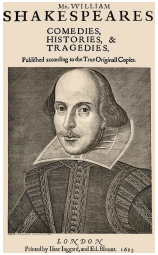
Constituents of Predicate logic

Exercise 12.1 Convert the following sentence into Predicate logic.

Not all that glitters is Gold.

- Choose predicates: $G(x)$: x glitters, $A(y)$: y is gold.
- $\neg(\forall x(G(x) \rightarrow A(x)))$
- $\exists x(G(x) \wedge \neg A(x))$

What is the difference between these two statements? Are they "equivalent"?



Can we show they are equivalent?

1. Need to define semantics, and semantical equivalence.
2. Can we also show this using proof rules? Need Natural deduction system!
3. First, what is the syntax!?

Fun fact: this proverb is a common misquote of Shakespeare's The Merchant of Venice – the original line is "All that glisters is not gold."

Topic 12.1

Syntax of Predicate Logic

Another difference from Propositional Logic

Exercise 12.2 Encode into Predicate logic.

Asif and Lavanya have the same grandfather.

- Use "variables" a for "Asif", ℓ for "Lavanya"
- Binary predicate F for "being father", i.e., $F(x, y)$ if y is father of x.

Function Symbols: Suppose we had a function symbol f(x) to mean father of x.

- Then we get an easy encoding! $f(f(a)) = f(f(\ell))$
- Function symbols are very useful to get compact encodings.
- Only applicable when no ambiguity, e.g., father of. Can't use for "brother of". (why?)
- Can take multiple or zero arguments. Functions with zero arguments are called constants.
- Thus, a and ℓ above are actually constants, not variables!

Important note: Equality $=$ is a special binary predicate symbol. Always assumed to be present!

Our Alphabet

Variable names: x, y, z

Vocabulary

- Function symbols: $f, g, h...$
  - Arity of function is number of arguments.
  - 0-ary functions are constants, we will use $a, b, c...$
- Relation (Predicate) symbols: $P, Q, R, ...$ (including $=$)
  - Hence, called Predicate logic or calculus.
  - Arity of predicate is number of arguments.

Fixed symbols

- All symbols from prop. logic: $\wedge, \vee, \neg, \rightarrow, (,)$
- New ones: $\exists, \forall$ (quantifiers)

Syntax of Predicate Logic: Terms

- Strings over the alphabet defined, but not any strings, only those with special structure!
- Two classes of syntactic objects: terms and formulas.

Terms

- every variable and constant is a term
- if f is an n-ary function and $t_1, \ldots t_n$ are terms, then so is $f(t_1, \ldots, t_n)$.

$$t ::= x \mid c \mid f(t, \ldots, t)$$

where x is over variables, c over constants and f over other function symbols.

Examples (recall Exercise 12.2)

- a (Asif), ℓ (Lavanya) are terms – they are constants.
- $f(a)$ (father of Asif), $f(f(a))$ (grandfather of Asif) are terms too!
- Careful: S(Osim) (recall the earlier example!) is *not* a term!
  - f is a function symbol: maps objects to objects, so $f(a)$ *denotes an object* – a term.
  - S is a predicate symbol: maps objects to truth values, so S(Osim) is True or False – not a term!

Syntax of Predicate Logic: Formulas

An expression that (informally) *evaluates to a truth value* is called a formula!

Formulas

- If R is an m-ary predicate, $t_1, \ldots t_m$ are terms then $R(t_1, \ldots t_m)$ is a(n atomic) formula
- Boolean connectives give formulas: $\neg\varphi$, $\varphi_1 \wedge \varphi_2$ etc.
- Quantifiers give formulas: if φ is a formula, x a variable, then so is $\exists x\varphi$, $\forall x\varphi$.

$$\varphi ::= P(t_1, \ldots t_n) \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi) \mid (\forall x\varphi) \mid (\exists x\varphi)$$

Examples

- $f(f(a)) = f(f(\ell))$ is an (atomic) formula – Asif and Lavanya have the same grandfather!
- $\forall x(S(x) \rightarrow \exists y(T(y) \wedge Y(x, y)))$ is a formula – every student is younger than some teacher.

Why "First Order"?

Called First order logic because quantification is only over individual variables...

Binding and Parse trees

Binding rules as expected!

$$\{\neg, \exists x, \forall x\} \quad > \quad \{\vee, \wedge\} \quad > \quad \{\rightarrow\}$$

- $\rightarrow$ is also right-associative: $\varphi_1 \rightarrow \varphi_2 \rightarrow \varphi_3$ means $\varphi_1 \rightarrow (\varphi_2 \rightarrow \varphi_3)$.
- Unlike $\wedge, \vee$ (where bracketing never changes the meaning), bracketing genuinely changes the truth value for $\rightarrow$!

Formulas can be represented as Parse trees as well! We omit brackets when they do not introduce ambiguity.

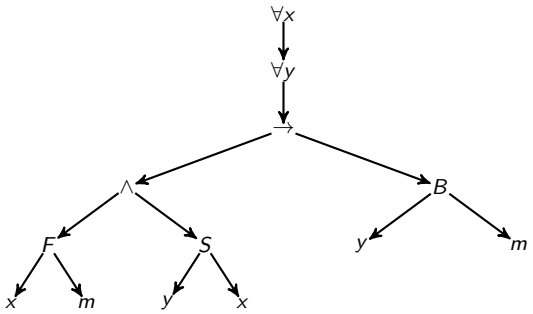
Exercise 12.3

1. Draw the Parse tree for $\forall x((P(x) \rightarrow Q(x)) \wedge S(x, y))$.
2. Encode into Predicate logic and draw its Parse tree: every son of my father is my brother.

Encoding "every son of my father is my brother"

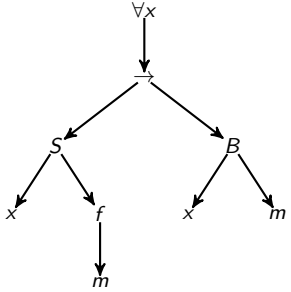
Using Predicates Only:
$F(x, y)$: x father of y; $S(x, y)$: x son of y; $B(x, y)$: x brother of y; m: "me".

$$\forall x \forall y (F(x, m) \wedge S(y, x) \rightarrow B(y, m))$$



Using a Function Symbol (reuse f from Ex 12.2!):
$S(x, y)$: x son of y; $B(x, y)$: x brother of y; m: "me".

$$\forall x (S(x, f(m)) \rightarrow B(x, m))$$



Much more compact than the predicates-only version – this is exactly why function symbols are useful!

Free variables, Bound variables and Sentences

Free variables are those that are not quantified in a formula, denoted $free(\varphi)$.

A recursive definition

- if φ is an atomic formula, then $free(\varphi) = \{x \mid x \text{ occurs in } \varphi\}$.
- if $\varphi = \neg\psi$, then $free(\varphi) = free(\psi)$.
- if $\varphi = \psi_1 \circ \psi_2$ for $\circ \in \{\wedge, \vee, \rightarrow\}$, then $free(\varphi) = free(\psi_1) \cup free(\psi_2)$.
- if $\varphi = Qx\ \psi$ for $Q \in \{\exists, \forall\}$, then $free(\varphi) = free(\psi) \setminus \{x\}$

If φ has free variables $\{x, y\}$, then we write $\varphi(x, y)$.
A formula with no free variables is called a sentence, e.g., $\exists x \forall y\ f(x) = y$.

Exercise 12.4

1. What is free in $\psi := ((\exists x\ P(x, y)) \wedge (\forall y\ Q(x, y)))$? Is ψ a sentence?
2. A variable is bound if it is quantified in a formula. Give a recursive definition of $bnd(\varphi)$.
3. Are free and bound variables complements? Why or why not?

Free and Bound Variables: Completing the Definition

Recursive definition of $bnd(\varphi)$, alongside $free(\varphi)$

- if φ is an atomic formula, then $bnd(\varphi) = \emptyset$
- if $\varphi = \neg\psi$, then $bnd(\varphi) = bnd(\psi)$
- if $\varphi = \psi_1 \circ \psi_2$ for $\circ \in \{\wedge, \vee, \rightarrow\}$, then $bnd(\varphi) = bnd(\psi_1) \cup bnd(\psi_2)$
- if $\varphi = Qx\ \psi$ for $Q \in \{\exists, \forall\}$, then $bnd(\varphi) = bnd(\psi) \cup \{x\}$

Free and Bound variables are not complements of each other!

Consider again $\psi = ((\exists x\ P(x,y)) \wedge (\forall y\ Q(x,y)))$.

$$bnd(\psi) = bnd(\exists x\ P(x,y)) \cup bnd(\forall y\ Q(x,y))$$
$$= (bnd(P(x,y)) \cup \{x\}) \cup (bnd(Q(x,y)) \cup \{y\})$$
$$= (\emptyset \cup \{x\}) \cup (\emptyset \cup \{y\}) = \{x, y\}$$

$free(\psi) = bnd(\psi) = \{x, y\}$

Substitution in FOL

Variables are placeholders; so we must be able to replace them with concrete terms: substitution.

- Suppose $x \in \mathit{free}(\varphi)$ and t is any term.
- Can we replace every free occurrence of x in φ with t?
- For example, consider $\psi = \exists y R(x, y) \vee \forall x R(z, x)$.
- If we naively try substituting $t = f(y, x)$ for x in ψ, we get $\exists y R(f(y, x), y) \vee \forall x R(z, x)$
- Problem: the y from t is now captured by $\exists y$ – it's no longer free like it was in t!

Definition

Term t is free for x in φ if no free occurrence of x in φ is in the "scope" of $\forall y$ or $\exists y$ for any var y occurring in t. e.g., $f(z, x)$ is free for x in ψ, but $f(y, x)$ is not.

Substitution

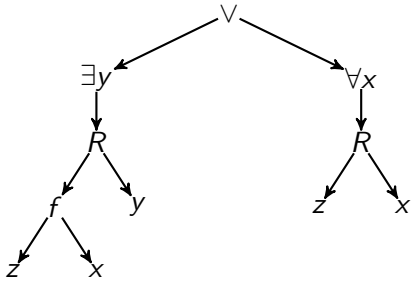
$\varphi[t/x]$ is formula obtained by replacing each free occurrence of x in φ by t, if t is free for x in φ.

For ψ defined above: $\psi[f(z, x)/x] := \exists y R(f(z, x), y) \vee \forall x R(z, x)$

Substitution in FOL: Correct vs. Captured

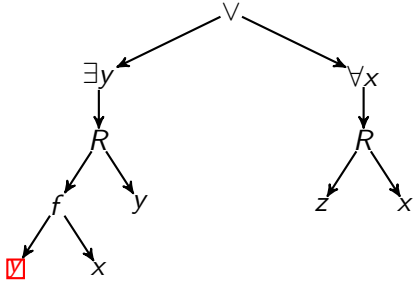
Correct: $\psi[f(z,x)/x]$

$\exists y\, R(f(z,x),y) \vee \forall x\, R(z,x)$



Wrong: $\psi[f(y,x)/x]$

$\exists y\, R(f(y,x),y) \vee \forall x\, R(z,x)$



The y inside $f(y,x)$ (boxed in red) is now captured by $\exists y$ – it's a different leaf than the term's original free y, but shares its scope!

- Fix: rename the bound y to a fresh y' first
- Rewrite ψ as $\exists y'\, R(x,y') \vee \forall x\, R(z,x)$ (same meaning, y is just a placeholder), *then* substitute.

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 13: Semantics of Predicate Logic

Instructor: S. Akshay

IIT Bombay, India

Logistics

Syllabus for Mid-Semester Exam

- Module 1: Propositional Logic
- Module 2: The Problem of Satisfiability
- All lectures till and including Lecture 11

Mid-Semester Exam

- Venue: LA 201, LA 002; PwD students at KR 125
- Seating arrangement will be shared on Piazza
- 1 A4 size paper allowed as cheatsheet, but MUST be handwritten with blue ink – no photocopy, etc.

Topics Covered till date

Module 1: Propositional Logic

1. Syntax and Semantics ause
2. Formal proof systems
3. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution - soundness, termination and completeness
2. The DPLL algorithm
3. Clause learning, the CDCL algorithm, and modern SAT solvers
4. A tool demo of SAT/SMT solvers in practice

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Today: semantics of predicate logic

Quick Recap: Syntax of Predicate Logic

- Terms: $t ::= x \mid c \mid f(t, \ldots, t)$
  where x is a variable, c a constant, f a function symbol
- Formulas: $\varphi ::= P(t_1, \ldots, t_n) \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \rightarrow \varphi \mid \forall x\, \varphi \mid \exists x\, \varphi$
  where P is a predicate symbol
- Free/bound variables: $\mathit{free}(\varphi)$, $\mathit{bnd}(\varphi)$ defined recursively – not complements of each other!
- Substitution: $\varphi[t/x]$ replaces free occurrences of x by t – but only if t is free for x in φ (no variable of t gets captured by a quantifier in φ)!

Today: what do these formulas actually *mean*?

Topic 13.1

Semantics of Predicate Logic

Semantics of FOL

Example 13.1 $\psi_1 := \forall x \forall y (P(x,y) \rightarrow \exists z (\neg(z = x) \wedge \neg(z = y) \wedge P(x,z) \wedge P(z,y)))$

1. What does this mean? English meaning: For every x,y if $P(x,y)$ holds, then we can find z different from x, y such that $P(x,z), P(z,y)$ hold.
2. Is the statement true?
   - Case 1: Suppose,
     - variables take values from real numbers.
     - $P(x,y)$ represents $x < y$
     - Then, ψ_1 is true, i.e., real numbers are dense!
   - Case 2: Instead suppose,
     - variables take values from real numbers, but $P(x,y)$ represents $x \leq y$
     - Then, ψ_1 is false!
   - Case 3: Finally, suppose,
     - variables take values from natural numbers, with $P(x,y)$ representing $x < y$
     - Then, ψ_1 is false, since natural numbers are not dense.

Truth value of any formula φ depends on (i) underlying set from which variables take values and (ii) interpretation of constants, functions and predicates, i.e., Vocabulary!

Semantics of FOL

- Vocabulary $\mathcal{V}$. E.g., $\mathcal{V} = \{a, f, =, P\}$
- $\mathcal{V}$-formula. E.g., $\psi_2 := \exists x(P(x, f(y, a)) \wedge \neg(x = a))$, $var(\psi_2) = \{x, y\}$, $free(\psi_2) = \{y\}$.

Truth of φ depends on

1. Universe U from which variables take values, e.g., $\mathbb{N}$
2. Interpretation of $\mathcal{V}$ on U
   - Each constant to an element of U, e.g., $a \mapsto 0$
   - Each n-ary function symbol to a function from U^n to U, e.g., $f(u, v) = u + v$.
   - Each m-ary predicate symbol to a subset of U^m, e.g.,
     - Interpret for $=$: $\{(c, c) \mid c \in \mathbb{N}\}$ - fixed interpretation!
     - Interpret for P : $\{(c, d) \mid c, d \in U, c < d\}$

   This defines a $\mathcal{V}$-structure (or Model) $M = (U^M, (a^M, f^M, P^M))$, e.g., $(\mathbb{N}, (0, +, <))$
3. We also need an environment $\alpha : var(\varphi) \to U$, e.g., $\alpha(y) = 2, \alpha(x) = 1$.
   - We will also use $\alpha[x \mapsto u]$ to denote the env which maps x to u (other vars are mapped according to α).

Semantics of FOL: An Example First

- **Example 13.2** Consider again $\psi_2 := \exists x(P(x, f(y, a)) \wedge \neg(x = a))$ over Vocabulary $\mathcal{V} = \{a, f, =, P\}$
- Also, let $M = (\mathbb{N}, (0, +, <))$ be a $\mathcal{V}$-structure, and $\alpha(x) = 0, \alpha(y) = 2$ be the env.
- Does M, α satisfy ψ_2?
  - First compute $f(y, a) = y + a = 2 + 0 = 2$ (using $\alpha(y) = 2, a^M = 0$).
  - Is $P(x, f(y, a))$ true, i.e. is $x < 2$?
  - Is $\neg(x = a)$ true, i.e. is $x \neq 0$?
  - Is their conjunction true, i.e. is $x < 2$ and $x \neq 0$?
  - Finally, is there a value of x from the universe $\mathbb{N}$ that makes their conjunction true?
    - With $x = 0$ it is not satisfied.
    - But, $x = 1$ works! And that is all we need.
    - In other words, the env α, where we set $x = 1$ (ignoring the earlier value) satisfies the formula, since $(1 < 2 \wedge 1 \neq 0)$ is true!
  - Thus, we can conclude $M, \alpha \models \psi_2$!

Now, let's make this definition precise and generic!

Semantics of FOL: Formal Definition

Given $\mathcal{V}$-structure/model M and env α, we define $M, \alpha \models \varphi$ (satisfaction relation)

By structural induction! First we extend α to $\overline{\alpha} : \mathit{Terms}(\varphi) \to U^M$, by

- If t is a variable x, then $\overline{\alpha}(t) = \alpha(x)$
- If t is $f(t_1, \ldots, t_m)$, then $\overline{\alpha}(t) = f^M(\overline{\alpha}(t_1), \ldots, \overline{\alpha}(t_m))$
  In Example 13.2, we have $\overline{\alpha}(f(y, a)) = f^M(\alpha(y), a^M) = 2 + 0 = 2$

- $M, \alpha \models P(t_1, \ldots t_m)$ iff $(\overline{\alpha}(t_1), \ldots \overline{\alpha}(t_m)) \in P^M$ (atomic formulas)
- $M, \alpha \models \neg\varphi$ iff $M, \alpha \not\models \varphi$.
- $M, \alpha \models \varphi_1 \wedge \varphi_2$ iff $M, \alpha \models \varphi_1$ and $M, \alpha \models \varphi_2$.
- $M, \alpha \models \exists x\varphi$ iff there exists some $u \in U^M$ s.t., $M, \alpha[x \mapsto u] \models \varphi$.
- $M, \alpha \models \forall x\varphi$ iff for all $u \in U^M$, $M, \alpha[x \mapsto u] \models \varphi$.

We omit $\vee, \to, \Leftrightarrow$ here since they reduce to $\wedge, \neg$ in the usual way (e.g. $\varphi_1 \to \varphi_2 \equiv \neg\varphi_1 \vee \varphi_2$) – same treatment as $\wedge$ above. **Exercise 13.1** In the e.g., what if we chose (i) $\alpha(y) = 0$? (ii) $\alpha(x) = 5$? (iii) $\alpha(x) = 1$?

Satisfaction only depends on the "free" environment

Exercise 13.2

Show that if α, α' are identical on the set of free variables, then for any formula φ and any $\mathcal{V}$-structure M, $M, \alpha \models \varphi$ iff $M, \alpha' \models \varphi$.

hint: use structural induction!

- Thus, if φ has no free variables, i.e., it is a sentence, then α does not matter!

For a sentence φ, we just write $M \models \varphi$.

Recall: Truth Depends on the Structure

In Example 13.1 $\psi_1 := \forall x \forall y (P(x,y) \rightarrow \exists z (\neg(z = x) \wedge \neg(z = y) \wedge P(x,z) \wedge P(z,y)))$

- For structure $M_1 = (\mathbb{R}, <)$: $M_1 \models \psi_1$ (density!)
- For structure $M_2 = (\mathbb{R}, \leq)$: $M_2 \not\models \psi_1$
- For structure $M_3 = (\mathbb{N}, <)$: $M_3 \not\models \psi_1$

What is the vocab here?
Why is there no α?

In Example 13.2 $\psi_2 := \exists x (P(x, f(y,a)) \wedge \neg(x = a))$ over $\mathcal{V} = \{a, f, =, P\}$

- For $M_4 = (\mathbb{N}, (0, +, <))$ with $\alpha(x) = 0, \alpha(y) = 2$: $M_4, \alpha \models \psi_2$ (witness $x = 1$)
- For $M_4 = (\mathbb{N}, (0, +, <))$ with $\alpha'(x) = 0, \alpha'(y) = 0$: $M_4, \alpha' \not\models \psi_2$ (need $x < 0$, no natural number works!)

Exercise 13.3 Find a *different* structure M' (and/or environment α'') for formula ψ_2, and determine whether $M', \alpha'' \models \psi_2$.
What happens when $M' = (\mathbb{N}, (1, \times, <))$? Is there an env which makes the formula true/false?

Revisiting substitution

Exercise 13.4 Let $\psi_3(x) := \exists y\,(y > x)$ over $\mathcal{V} = \{>\}$, with $M = (\mathbb{N}, >)$.

1. What is *free*(ψ_3)? For any α, does $M, \alpha \models \psi_3$ hold?
2. Is $t = y$ free for x in ψ_3? If we substitute anyway, what is $\psi_3[y/x]$? What does this tell us about the substitution definition?

free$(\psi_3) = \{x\}$. For *any* α, is there $u \in \mathbb{N}$ with $u > \alpha(x)$? Yes, always! Numbers are unbounded above, so $M, \alpha \models \psi_3$ for every α.

Recall Substitution in FOL: $t = y$ is not free for x in ψ_3: (y occurs in t, and the free x is under the scope of $\exists y$)

- Substituting anyway, step by step:

$$\exists y\,(y > x) \quad \longrightarrow \quad \exists y\,(y > y)$$

- So $\psi_3[y/x] = \exists y\,(y > y)$
- Is *this* true? No – always false! No number is bigger than itself.

The truth value flipped. This is exactly why substitution is restricted to terms that are free for x.

Semantic entailment

Once we fix the vocab $\mathcal{V}$, we just write structure instead of $\mathcal{V}$-structure.

Let Γ be a (possibly infinite) set of formulas and let ψ be a formula in predicate logic.

- **Semantic entailment** $\Gamma \models \psi$ holds iff for all M and α, whenever $M, \alpha \models \varphi$ holds for all $\varphi \in \Gamma$, then $M, \alpha \models \psi$ holds as well.
- ψ is **satisfiable** iff there is some M and α such that $M, \alpha \models \psi$ holds.
- ψ is **valid** iff $M, \alpha \models \psi$ holds for all M and α.
- Γ is **consistent** iff there is at least one M and α such that $M, \alpha \models \varphi$ for all $\varphi \in \Gamma$.

Note: M ranges over *all* universes and *all* interpretations of $\mathcal{V}$ on them – not a fixed universe!

Exercise 13.5 Give 2 examples of valid and 2 of satisfiable (but not valid) formulas with the vocab given in Example 13.2.

Exercise 13.5 Solution

Vocab: constant a, unary function f, binary predicate P (plus always-present $=$).

Valid (true for every M and α, regardless of interpretation)

1. $a = a$ – equality has a *fixed* interpretation (always literal identity), so this holds no matter how a is interpreted.
2. $\forall x\,(P(x, f(a)) \to P(x, f(a)))$ – an instance of the propositional tautology $\varphi \to \varphi$, lifted into predicate logic; true regardless of how P, f, a are interpreted.

Satisfiable but *not* valid

1. $P(a, a)$ – true if we pick M with $P^M = U \times U$ (P always holds); false in our familiar $M_4 = (\mathbb{N}, (0, +, <))$ since $0 < 0$ is false. Satisfiable, not valid.
2. $\exists x\, P(x, a)$ – true if $U = \mathbb{Z}$, P interpreted as "$<$", $a = 0$ (witness $x = -1$); false in $M_4 = (\mathbb{N}, (0, +, <))$ since no natural number is < 0. Satisfiable, not valid.

Semantic equivalences

Definition

$\varphi \equiv \psi$ iff $\{\varphi\} \models \psi$ and $\{\psi\} \models \varphi$.

Exercise 13.6 Prove the following: Quantifier equivalences

- $\forall\, x\varphi \equiv \neg\exists\, x\neg\varphi$
- $\forall x \forall y \varphi \equiv \forall y \forall x \varphi$
- $\forall x(\varphi_1 \wedge \varphi_2) \equiv (\forall x \varphi_1) \wedge (\forall x \varphi_2)$
- $\exists x(\varphi_1 \vee \varphi_2) \equiv (\exists x \varphi_1) \vee (\exists x \varphi_2)$

Validity and Satisfiability for predicate logic

- For propositional logic, check validity just required to write down the truth table.
- Can we do something similar for predicate logic, in particular FOL?

No! Checking validity is undecidable!

Undecidable means: there is *no algorithm* which, given any formula, always correctly answers "valid" or "not valid" in finite time.

- In fact this is the case for a vocabulary with 1 constant, 2 unary functions and 1 binary predicate! (Church, 1936)
- Encoding of Post's correspondence problem.



What about satisfiability? Also undecidable!

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 14: Natural Deduction for First-Order Logic

Instructor: S. Akshay

IIT Bombay, India

Welcome back after the Midsem

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Semantics of predicate logic (Lecture 13)
3. Today: a formal proof system for predicate logic (natural deduction)

Let's warm up with a quick recap before diving into today's new material.

Predicate Logic: Alphabet and Syntax

Alphabet consists of

- Variable names: x, y, z
- Vocabulary
  - Function symbols: $f, g, h...$
  - Constants: a, b, c are just 0-ary function symbols.
  - Relation (Predicate) symbols: $P, Q, R, ...$
- Fixed symbols: $\wedge, \vee, \neg, \rightarrow, (,), \exists, \forall$

Syntax of First Order Logic (quantification over variables)

Two classes of syntactic objects: terms and formulas defined over the alphabet.

- Terms $$t ::= x \mid c \mid f(t, \ldots, t)$$

  where x is over variables, c over constants and f over other function symbols.

- Formulas $$\varphi ::= P(t_1, \ldots t_n) \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi) \mid (\forall x\varphi) \mid (\exists x\varphi)$$

  where P is over set of predicates, $t_1, \ldots t_n$ are terms and x is a variable.

Substitution in FOL

Variables are placeholders; so we must be able to replace them with concrete terms: substitution.

- Suppose $x \in \mathit{free}(\varphi)$ and t is any term.

- For example, consider $\psi = \exists y R(x, y) \vee \forall x R(z, x)$.

Definition

Term t is free for x in φ if no free occurrence of x in φ is in the "scope" of $\forall y$ or $\exists y$ for any var y occurring in t. e.g., $f(z, x)$ is free for x in ψ, but $f(y, x)$ is not.

Substitution

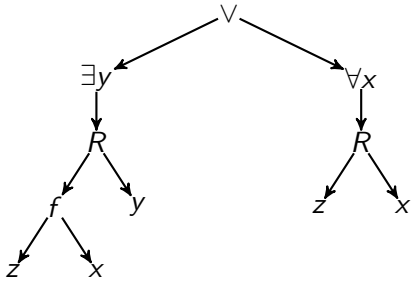
$\varphi[t/x]$ is formula obtained by replacing each free occurrence of x in φ by t, if t is free for x in φ.

For ψ defined above: $\psi[f(z, x)/x] := \exists y R(f(z, x), y) \vee \forall x R(z, x)$

Substitution in FOL: Correct vs. Captured

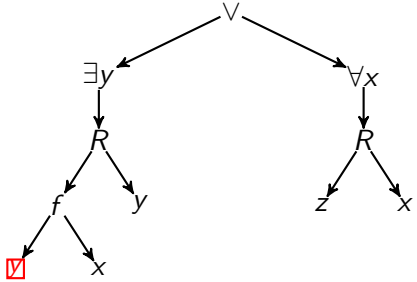
Correct: $\psi[f(z,x)/x]$

$\exists y\, R(f(z,x),y) \vee \forall x\, R(z,x)$



Wrong: $\psi[f(y,x)/x]$

$\exists y\, R(f(y,x),y) \vee \forall x\, R(z,x)$



The y inside $f(y,x)$ (boxed in red) is now captured by $\exists y$ – it's a different leaf than the term's original free y, but shares its scope!

- Fix: rename the bound y to a fresh y' first
- Rewrite ψ as $\exists y'\, R(x,y') \vee \forall x\, R(z,x)$ (same meaning, y is just a placeholder), *then* substitute.

Quick Recap: Semantics of Predicate Logic

- Vocabulary $\mathcal{V}$ + universe U + interpretation of constants/functions/predicates on $U \Rightarrow$ a $\mathcal{V}$-structure (model) M
- Environment $\alpha : var(\varphi) \to U$ (defined on all variables, but only its values on free variables affect satisfaction)
- Satisfaction relation $M, \alpha \models \varphi$ defined by structural induction on φ (atomic formulas via interpretation; $\wedge, \neg$ as usual; $\exists x\, \varphi$ iff some $u \in U$ with $M, \alpha[x \mapsto u] \models \varphi$; similarly for $\forall$)
- For a sentence (no free variables), we just write $M \models \varphi$ – the environment doesn't matter
- Semantic entailment $\Gamma \models \psi$, validity, and satisfiability are defined just as in propositional logic, with M, α in place of a truth assignment

Unlike propositional logic, both validity and satisfiability for FOL are undecidable – no algorithm can always decide them in finite time.

Today: can we nevertheless *prove* things about FOL formulas syntactically?

Topic 14.1

Natural Deduction for FOL

A Proof System for FOL

- All proof rules for Propositional Logic carry over and remain sound for FOL.
- In addition, we need new rules for:
  - Equality ($=$)
  - Quantifiers ($\forall, \exists$)

Equality Rules

- A rose is a rose!
- If two terms are equal, then we can replace one by the other in any formula.
- If $y * (w + 2) = y * w + y * 2$; then $z \geq y * (w + 2)$ implies $z \geq y * w + y * 2$ (and vice versa)

$$\frac{}{t = t} =_i \qquad \frac{t_1 = t_2 \qquad \varphi[t_1/x]}{\varphi[t_2/x]} =_e$$

- Axiom for equality, no premises
- Only on terms NOT formulas

- Of course, this is applicable ONLY if t_1, t_2 are free for x in φ.
- Eliminates $=$ by replacing all t_1 in $\varphi[t_1/x]$ by t_2.

Exercise 14.1 Show that the following sequents are valid.

1. $t_1 = t_2 \vdash t_2 = t_1$
2. $t_1 = t_2, t_2 = t_3 \vdash t_1 = t_3$

Exercise 14.1: Solution

1. $t_1 = t_2 \vdash t_2 = t_1$

1.	$t_1 = t_2$	premise
2.	$t_1 = t_1$	$=_i$
3.	$t_2 = t_1$	$=_e$ 1,2

where φ is $x = t_1$, so line 2 is $\varphi[t_1/x]$ and line 3 is $\varphi[t_2/x]$.

2. $t_1 = t_2, t_2 = t_3 \vdash t_1 = t_3$

1.	$t_1 = t_2$	premise
2.	$t_2 = t_3$	premise
3.	$t_1 = t_3$	$=_e$ 2,1

where φ is $t_1 = x$, so line 1 is $\varphi[t_2/x]$ and line 3 is $\varphi[t_3/x]$.

Proof Rules for Universal Quantification

$$\frac{\forall x \varphi}{\varphi[t/x]}\ \forall x\ e$$

$$\frac{\boxed{\begin{array}{c} x_0 \\ \vdots \\ \varphi[x_0/x] \end{array}}}{\forall x \varphi}\ \forall x\ i$$

- All students did PS1 $\Rightarrow$ so did YOU!
- If $\forall x \varphi$ holds, then we can replace x in φ by any term t.
- Imp: side condition t is free for x in φ.
  - Consider $\varphi = \exists y(x < y)$ with vocab reals with usual smaller than relation.
  - $\forall x \varphi$ means for all real n, there exists real m bigger than n, true!
  - But $\varphi[y/x] = \exists y(y < y)$, which says there must exist a no. bigger than itself, false!
  - We want only sound rules!
  - Soln: replace variable y by fresh var z

- Say I threw this tennis ball $\Rightarrow$ I can throw ANY tennis ball!
- If from a fresh var x_0, we can prove $\varphi[x_0/x]$, then $\forall x \varphi$ holds.
- Side condn: x_0 does not occur outside the box.
- In proving φ, we didn't make an assumption on x.

Examples

Exercise 14.2 Prove that the following sequents are valid.

1. $\forall x(P(x) \rightarrow Q(x)), \forall x P(x) \vdash \forall x Q(x)$
2. $P(t), \forall x(P(x) \rightarrow \neg Q(x)) \vdash \neg Q(t)$ for any term t.

Exercise 14.2: Solution

1. $\forall x(P(x) \to Q(x)), \forall x P(x) \vdash \forall x Q(x)$

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\forall x P(x)$	premise
3.	$P(x_0) \to Q(x_0)$	$\forall x$ e 1
4.	$P(x_0)$	$\forall x$ e 2
5.	$Q(x_0)$	MP 3,4
6.	$\forall x Q(x)$	$\forall x$ i 3-5

(box scoped by fresh x_0)

2. $P(t), \forall x(P(x) \to \neg Q(x)) \vdash \neg Q(t)$

1.	$P(t)$	premise
2.	$\forall x(P(x) \to \neg Q(x))$	premise
3.	$P(t) \to \neg Q(t)$	$\forall x$ e 2
4.	$\neg Q(t)$	MP 3,1

An Interesting Analogy

Intuition: $\forall$ "generalizes" $\wedge$

- Just like in $\wedge$-e, we could get F or G from $F \wedge G$, we can get $\varphi[t/x]$ for any term t (modulo side-condn!).
- Similarly $\wedge$-i had two premises, here $\forall x\ i$ has premise for any x_0.

Proof rules for Existential Quantifiers

- Can we lift this analogy and do the same for $\exists$ and $\vee$?
- For $\vee$, $\vee$-i was easy! So what should be the rule for $\exists$-i? What about $\exists$-e?

$$\frac{\varphi[t/x]}{\exists x \varphi}\ \exists x\ i$$

$$\frac{\exists x \varphi \qquad \boxed{\begin{array}{l} x_0\ \ \varphi[x_0/x] \\ \vdots \\ \chi \end{array}}}{\chi}\ \exists x\ e$$

- To use $\exists x \varphi$, you should have a proof from any possible value for x.
- The box controls scope of x_0 and scope of assumption also.

Examples

Exercise 14.3 Show that the following sequents are valid.

1. $\forall x \varphi \vdash \exists x \varphi$
2. $\forall x(P(x) \to Q(x)), \exists x P(x) \vdash \exists x Q(x)$
3. $\exists x P(x), \forall x \forall y(P(x) \to Q(y)) \vdash \forall y Q(y)$

Note: Be very careful with scope of x_0... see examples in Huth & Ryan for "false" proofs that one may derive by misusing the scope.

Many more examples in Huth & Ryan... Do try them out!

Solution for 1. $\forall x \varphi \vdash \exists x \varphi$

1.	$\forall x \varphi$	premise	
2.	φ	$\forall x\, e\, 1$	(choose $t = x$ for both steps: $\varphi[x/x] = \varphi$, so this is legal)
3.	$\exists x \varphi$	$\exists x\, i\, 2$	

Exercise 14.3.2: Correct vs. Wrong

Correct

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\exists x P(x)$	premise
3.	$P(x_0)$	assumption
4.	$P(x_0) \to Q(x_0)$	$\forall x$ *e* 1
5.	$Q(x_0)$	MP 4,3
6.	$\exists x Q(x)$	$\exists x$ *i* 5
7.	$\exists x Q(x)$	$\exists x$ *e* 2, 3-6

Wrong!

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\exists x P(x)$	premise
3.	$P(x_0)$	assumption
4.	$P(x_0) \to Q(x_0)$	$\forall x$ *e* 1
5.	$Q(x_0)$	MP 4,3
6.	$Q(x_0)$	$\exists x$ *e* 2, 3-5
7.	$\exists x Q(x)$	$\exists x$ *i* 6

The box on the right closes one line too early: line 6 ($Q(x_0)$) lets the fresh x_0 escape its own box before $\exists x$ *i* is applied outside – but χ in $\exists x$ *e* must not mention x_0!

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 15: Soundness, Completeness, and Structures in FOL

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Semantics of predicate logic (Lecture 13)
3. Natural deduction for FOL (Lecture 14)
4. Today: is our proof system sound and complete, and just how powerful is FOL?

Quick Recap: Natural Deduction for FOL

- All proof rules for Propositional Logic carry over and remain sound for FOL.

$$\frac{}{t = t}\ =_i \qquad \frac{t_1 = t_2 \qquad \varphi[t_1/x]}{\varphi[t_2/x]}\ =_e$$

Equality: reflexivity axiom, and substitute equals for equals (terms only, and only where free for x).

$$\frac{\forall x \varphi}{\varphi[t/x]}\ \forall x\ e \qquad \frac{\begin{array}{|l} x_0 \\ \vdots \\ \varphi[x_0/x] \end{array}}{\forall x \varphi}\ \forall x\ i$$

Universal quantifier: instantiate to any term t (free for x); or prove $\varphi[x_0/x]$ for a fresh x_0 inside a box.

$$\frac{\varphi[t/x]}{\exists x \varphi}\ \exists x\ i \qquad \frac{\exists x \varphi \qquad \begin{array}{|ll} x_0 & \varphi[x_0/x] \\ & \vdots \\ & \chi \end{array}}{\chi}\ \exists x\ e$$

Existential quantifier: generalize $\varphi[t/x]$ to $\exists x \varphi$; or from $\exists x \varphi$ and a boxed proof of χ from fresh $x_0, \varphi[x_0/x]$, conclude χ (provided χ doesn't mention x_0).

Side conditions on freshness/scope are important – violate them and rules become unsound!

Quick Recap: (Ungraded) Pop quiz!

Recall the exercise from last lecture: Prove the following sequents using ND for FOL:

1. $\forall x \varphi \vdash \exists x \varphi$
2. $\forall x(P(x) \to Q(x)), \exists x P(x) \vdash \exists x Q(x)$
3. $\exists x P(x), \forall x \forall y (P(x) \to Q(y)) \vdash \forall y Q(y)$
   Example: A factory's policy is "if any laptop in a batch has a faulty battery, recall every laptop in the batch." We're told some laptop has a faulty battery – so every laptop gets recalled!
   ($P(x)$: x has a faulty battery; $Q(y)$: y gets recalled.)
4. "Every prime number greater than 2 is odd. There exists a prime number greater than 2. So, there exists an odd number." First formalize this in FOL, then prove it using ND!

Which rules would you use first? When do you need a fresh variable and when do you need a box?

First Question: $\forall x \varphi \vdash \exists x \varphi$

1. $\forall x \varphi \vdash \exists x \varphi$

1.	$\forall x \varphi$	premise
2.	φ	$\forall x\ e\ 1$
3.	$\exists x \varphi$	$\exists x\ i\ 2$

(choose $t = x$ for both steps: $\varphi[x/x] = \varphi$, so this is legal)

Second Question: $\forall x(P(x) \to Q(x)), \exists x P(x) \vdash \exists x Q(x)$

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\exists x P(x)$	premise
3.	x_0 $P(x_0)$	assumption
4.	$P(x_0) \to Q(x_0)$	$\forall x$ *e* 1
5.	$Q(x_0)$	MP 4,3
6.	$\exists x Q(x)$	$\exists x$ *i* 5
7.	$\exists x Q(x)$	$\exists x$ *e* 2, 3-6

Wrong!

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\exists x P(x)$	premise
3.	x_0 $P(x_0)$	assumption
4.	$P(x_0) \to Q(x_0)$	$\forall x$ *e* 1
5.	$Q(x_0)$	MP 4,3
6.	$Q(x_0)$	$\exists x$ *e* 2, 3-5
7.	$\exists x Q(x)$	$\exists x$ *i* 6

The box on the right closes one line too early: line 6 ($Q(x_0)$) lets the fresh x_0 escape its own box before $\exists x$ *i* is applied outside – but χ in $\exists x$ *e* must not mention x_0!

Third Question: $\exists x P(x), \forall x \forall y (P(x) \rightarrow Q(y)) \vdash \forall y Q(y)$

3. $\exists x P(x), \forall x \forall y (P(x) \rightarrow Q(y)) \vdash \forall y Q(y)$

1.		$\exists x P(x)$	premise
2.		$\forall x \forall y (P(x) \rightarrow Q(y))$	premise
3.	y_0		
4.	x_0	$P(x_0)$	assumption
5.		$\forall y (P(x_0) \rightarrow Q(y))$	$\forall x$ e 2
6.		$P(x_0) \rightarrow Q(y_0)$	$\forall y$ e 5
7.		$Q(y_0)$	MP 6,4
8.		$Q(y_0)$	$\exists x$ e 1, 4-7
9.		$\forall y Q(y)$	$\forall y$ i 3-8

(outer box scoped by fresh y_0; inner box scoped by fresh x_0, assuming $P(x_0)$)

Fourth Question: Formalization & Proof

Every prime number greater than 2 is odd. There exists a prime number greater than 2. So, there exists an odd number.

Formalization

Let $P(x)$: "x is a prime number greater than 2"; $Q(x)$: "x is odd."

$\forall x(P(x) \to Q(x)), \exists x P(x) \vdash \exists x Q(x)$

1.		$\forall x(P(x) \to Q(x))$	premise
2.		$\exists x P(x)$	premise
3.	x_0	$P(x_0)$	assumption
4.		$P(x_0) \to Q(x_0)$	$\forall x\ e$ 1
5.		$Q(x_0)$	MP 4,3
6.		$\exists x Q(x)$	$\exists x\ i$ 5
7.		$\exists x Q(x)$	$\exists x\ e$ 2, 3-6

Topic 15.1

Soundness, Completeness, and the Power of FOL

Soundness and completeness

Soundness: If $\varphi \vdash \psi$, then $\varphi \models \psi$

- We can only derive semantically correct proofs!
- Intuitively easy to see, again by structural induction.

Is this proof system also complete?

Completeness: If $\varphi \models \psi$, then $\varphi \vdash \psi$

- Is anything that is true, derivable?
- Yes, but we won't stop to prove it here!

So even checking whether $\vdash \varphi$ holds is undecidable – because validity itself ($\models \varphi$) is undecidable, and completeness tells us $\vdash$ and $\models$ coincide!

How powerful is First-order logic?

Even basic properties are undecidable.

Can it express everything of interest?

How can we use it if it is so powerful?

Food for thought!

Topic 15.2

Structures and Expressiveness of FOL

Quick Recap: Semantics of Predicate Logic (Lecture 13)

- Vocabulary $\mathcal{V}$: the constants, functions, and predicates available (e.g. $\mathcal{V} = \{a, f, =, P\}$).
- $\mathcal{V}$-structure (or Model) $M = (U^M, (a^M, f^M, P^M))$: a universe U^M plus an interpretation of every symbol in $\mathcal{V}$ over it (e.g. $M = (\mathbb{N}, (0, +, <))$).
- Environment α: maps free variables to elements of U^M (needed only when φ has free variables).
- Satisfaction $M, \alpha \models \varphi$: defined by structural induction on φ.

First-order Definable Structures

- Recall that, if φ is a sentence (no free variables), no environment is necessary for evaluating truth of φ
  - So given $\mathcal{V}$-structure M, we can ask if $M \models \varphi$
  - Class of $\mathcal{V}$-structures defined by φ is just $\{M \mid M \models \varphi\}$
  - Such classes of structures that are defined by an FOL sentence are called FO-definable.
- We will see three commonly occurring examples of structures:
  - Numbers (we saw this already!)
  - Graphs
  - Databases

Number systems as FO structures

Natural/reals with add, mult, lin order and constants 0,1

- $\mathfrak{N} = \{(\mathbb{N}, 0, 1, \times, +, <, =)\}$
- $\mathfrak{R} = \{(\mathbb{R}, 0, 1, \times, +, <, =)\}$

Examples of properties expressible in FOL

- Every number has a cube root
- $\mathfrak{R} \models \forall x \exists y (x = ((y \times y) \times y))$
- $\mathfrak{N} \not\models \forall x \exists y (x = ((y \times y) \times y))$
- There are infinitely many prime natural numbers. **Exercise 15.1**

Maybe we can check validity, but given the model/structure can we check if it satisfies a given formula? Model checking!

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 16: Expressiveness and the Limits of FOL

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Semantics of predicate logic (Lecture 13)
3. Natural deduction for FOL (Lecture 14)
4. Soundness, completeness, and structures in FOL (Lecture 15)
5. Today: Expressiveness and the limits of FOL

Quick Recap: First-order Definable Structures

- Recall: for a sentence φ (no free variables), $\{M \mid M \models \varphi\}$ – the class of structures it defines – is called FO-definable.
- Already saw: Numbers as FO structures, $\mathfrak{N} = (\mathbb{N}, 0, 1, \times, +, <, =)$ and $\mathfrak{R} = (\mathbb{R}, 0, 1, \times, +, <, =)$.
  - E.g.for $\psi = \forall x \exists y (x = y \times y \times y)$, $\mathfrak{R} \models \psi$ (every real has a cube root), but $\mathfrak{N} \not\models \psi$; and infinitely many primes (Exercise 15.1).
- Today: Databases and Graphs as FO structures, then just how far this expressive power actually goes.

Topic 16.1

Structures and Expressiveness of FOL

Relational Databases as FO structures

A school's database has tables like `DOB` (a student's date of birth), `Class` (which class a student is in), and `Primary` (which classes are primary-school classes)

DOB	
Student	Year
s_1	2010
s_2	2018

Class	
Student	Class
s_1	c_2
s_2	c_1

Primary
c_1

Relational Database D (a $\mathcal{V}$-structure/model)

- U^D: set of (possibly differently typed) data items
- Vocab $\mathcal{V}$: $\{P_1, \ldots P_k, =\}$, where P_i is k_i-ary pred corr to i^{th} table in database with k_i cols.
- Interpret: For $a_1, \ldots a_{k_i} \in U^D$, $P_i(a_1, \ldots a_{k_i}) = \top$ iff $(a_1, \ldots a_{k_i})$ is a row of ith table.

Example of class of databases definable in FOL:

$\forall x \forall y \forall z \, (Record(x, y, z) \leftrightarrow (DOB(x, y) \wedge Class(x, z)))$: Record is the join of DOB and Class.

Example Query: $\varphi(x) := \exists y \exists z (DOB(x, y) \wedge After(y, \text{“2016”}) \wedge Class(x, z) \wedge Primary(z))$

Defines set of students born after 2016, and studying in a primary class.

Model checking: Does a given Database satisfy a given property?

Words as FO structures

A word structure W (a $\mathcal{V}$-structure/model)

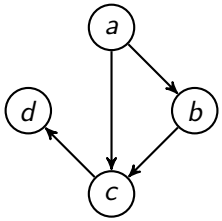
- U^W: set of all finite binary words (strings) over $\{0,1\}$, including the empty word ϵ.
- Vocab $\mathcal{V} : \{\cdot, \leq, =\}$, where $\cdot$ is a binary function symbol and $\leq$ a binary predicate symbol.
- Interpret $\cdot^W$: concatenation, e.g. $0110 \cdot^W 1110 = 01101110$; in general $a_1 \ldots a_k \cdot^W b_1 \ldots b_n = a_1 \ldots a_k b_1 \ldots b_n$.
- Interpret $\leq^W$: the prefix ordering – $s_1 \leq^W s_2$ iff $\exists s_3$ such that $s_1 \cdot^W s_3 = s_2$

E.g. 011 is a prefix of 011001 and of 011, but 010 is neither.

Pop Quiz. For each formula φ below, write its English meaning and argue whether it is true in W – i.e., does $W \models \varphi$ hold?

- **Q1.** $\exists y \, \forall x \, (y \leq x)$
- **Q2.** $\forall x \, \exists y \, (y \leq x)$
- **Q3.** $\forall x \, \forall y \, \forall z \, \big((x \leq y) \rightarrow (x \cdot z \leq y \cdot z)\big)$
- **Q4.** $\neg \exists x \, \forall y \, \big((x \leq y) \rightarrow (y \leq x)\big)$

Graphs as FO structures



A graph G (a $\mathcal{V}$-structure/model)

- U^G is a set of vertices
- Vocab $\mathcal{V} : \{E, =\}$ where E is the (binary) edge relation
- Interpret: For $a, b \in U^G$, $E^G(a, b) = \top$ iff there is a(n directed) edge from vertex a to vertex b in G.

Examples of graphs in FOL

- What does $\forall x \forall y(\neg(x = y) \rightarrow E(x, y))$ represent? (Infinite) Class of cliques!
- **Exercise 15.2** Give a FOL formula to define class of all graphs with no cycles of length 3.
  - A directed 3-cycle is $x \rightarrow y \rightarrow z \rightarrow x$ for three *distinct* vertices:

$$\exists x \exists y \exists z \left(\neg(x = y) \wedge \neg(y = z) \wedge \neg(x = z) \wedge E(x, y) \wedge E(y, z) \wedge E(z, x)\right)$$

  - So "no 3-cycles" is the negation of that:

$$\forall x \forall y \forall z \left(\neg(x = y) \wedge \neg(y = z) \wedge \neg(x = z) \wedge E(x, y) \wedge E(y, z) \rightarrow \neg E(z, x)\right)$$

Power of FOL

- FOL seems all powerful!
- Let us turn to a simpler question:
  - Given u, v in a directed graph, does there exist a finite path from u to v?
  - i.e., Give a FOL formula φ, which has u, v as free variables and E as the binary edge relation, such that
    - $G \models \varphi$ iff there is a path in G from u to v
    - So every graph G (with u, v assigned to specific vertices) satisfying φ must have a directed path from u to v
    - $u = v \vee E(u, v) \vee \exists x(E(u, x) \wedge E(x, v)) \vee \ldots$ is infinite! So not correct...

Surprise! There is no FOL formula that can express this!

Compactness of FOL

Turns out there is a very interesting property of FOL that implies this and many more results...

Compactness theorem

Let Γ be a (possibly infinite) set of FOL sentences. If all finite subsets of Γ are satisfiable/consistent, then so is Γ.

Corollary: If Γ is unsatisfiable/inconsistent, then there is a finite subset of Γ that is already unsat!
Proof of Theorem follows from completeness of FOL!

- If Γ is unsatisfiable, then $\Gamma \models \bot$ holds (why?)
- By completeness, $\Gamma \vdash \bot$ is valid, in other words there is a proof in ND for this.
- This proof is a finite set of lines, so uses only finitely many premises Δ from Γ.
- But then $\Delta \vdash \bot$ is also valid.
- By soundness, we get $\Delta \models \bot$.. thus we have a finite subset of Γ that is already unsat, as required!

Reachability is not expressible in FOL

Theorem: Reach is not expressible in FOL

- Suppose not. Then let ψ be the formula and let c, c' be consts.
- Let ψ_n be the formula expressing that there is a path of length n from c to c' **Exercise 15.3**
- Let $\Delta = \{\neg\psi_i \mid i \geq 0\} \cup \{\psi[c/u][c'/v]\}$.
- All formulas in Δ are sentences and it is unsatisfiable. (why?)
- But every finite subset of Δ is satisfiable!(why?)
- By compactness, Δ is satisfiable! Contradiction.

So how do we increase expressivity?
Quantify over sets!– higher order logic!!

Another corollary

Using compactness to show the following theorem.

Exercise 15.4

Let ψ be a FOL sentence s.t. $\forall n \geq 1$, there is a $\mathcal{V}$-structure M_n with $\geq n$ elements in its universe, such that $M_n \models \psi$, then there exists a $\mathcal{V}$-structure M with infinitely many elements in universe s.t. $M \models \psi$.

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 17: Normal Forms for FOL

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic (Lecture 12)
2. Semantics of predicate logic (Lecture 13)
3. Natural deduction for FOL (Lecture 14)
4. Soundness, completeness, and structures in FOL (Lecture 15)
5. Expressiveness and the limits of FOL (Lecture 16)
6. Today: Normal forms for FOL

Pop Quiz Yet Again!

1. Let m be a constant, f a function symbol of arity 1, and S, B predicate symbols of arity 2. Which of the following are well-formed formulas? Which of them are atomic?

i.	$S(m, x)$	well-formed – atomic
ii.	$B(m, f(m))$	well-formed – atomic
iii.	$f(m)$	not a formula (a term)
iv.	$B(B(m, x), y)$	ill-formed ($B(m, x)$ isn't a term)
v.	$S(B(m), z)$	ill-formed (B wrong arity, and not a term)
vi.	$(B(x, y) \to (\exists z\, S(z, y)))$	well-formed – not atomic
vii.	$(S(x, y) \to S(y, f(f(x))))$	well-formed – not atomic
viii.	$(B(x) \to B(B(x)))$	ill-formed (wrong arity)

2. Let Q be a predicate symbol of arity 3, g a function symbol of arity 2, c a constant.
   - $\varphi_1 := \forall x \forall y\, Q(g(x, y), g(y, y), c)$ $\quad$ $\varphi_2 := \forall x \forall y\, Q(g(x, y), g(y, y), z)$
   - $\varphi_3 := \forall x \forall y \forall z\, Q(g(x, y), g(y, y), z)$ $\quad$ $\varphi_4 := \forall x \forall y \exists z\, Q(g(x, y), g(y, y), z)$

   Under $M = (\mathbb{N}, g^M, Q^M, c^M)$, with $g^M(x, y) := 0$ for all x, y; $Q^M := \{(a, b, d) \mid d = 0\}$; $c^M := 0$, what is the truth value of $\varphi_1, \varphi_2, \varphi_3, \varphi_4$?

Pop Quiz Contd.

Note first: $Q(g(x,y), g(y,y), \cdot)$ collapses to "$(\cdot = 0)$", independent of x, y.

(a) φ_1 (constant c, c^M=0): true – M alone decides it, since c^M is already fixed at 0.

(b) φ_2 (free z): cannot be determined from M alone!
z is free, so we also need an environment α

- e.g. $\alpha(z)$:=0 gives $M, \alpha \models \varphi_2$, but $\alpha'(z)$:=1 gives $M, \alpha' \not\models \varphi_2$. Same M, two different answers.

(c) φ_3 (z bound by $\forall$): false – $\forall z\,(z = 0)$ fails in $\mathbb{N}$ (e.g. $z = 1$). M alone decides it.

(d) φ_4 (z bound by $\exists$): true – $\exists z\,(z = 0)$ holds (witness $z = 0$). M alone decides it.

Moral: $\varphi_1, \varphi_3, \varphi_4$ are all sentences – M alone fixes their truth value, no environment needed.
Only φ_2 has a free variable, so it genuinely needs an environment too.

Topic 17.1

Normal Forms for FOL

Why Normal Forms, Again?

- Recall (Module 1): propositional resolution and DPLL both needed the formula in CNF first.
- FOL has the same story – but quantifiers complicate things: $\forall, \exists$ can be nested, mixed, and their variables reused.
- Goal today: strip away the quantifiers until what's left is propositional-shaped again –
  - Prenex form: push all quantifiers out front;
  - Skolemization: eliminate the $\exists$'s;
  - Clausal form: what remains is a set of clauses, just like propositional CNF.
- This is exactly the machinery we need for:
  1. FOL resolution
  2. DPLL extensions to handle FOL (SMT solvers)

Prenex Normal Form

Example: $\forall w \forall y \left(R(f(w), y) \wedge (\neg R(w, c) \vee \neg P(w))\right)$ – all quantifiers pulled out front, matrix left quantifier-free.

Prenex form

A sentence is in prenex form if it looks like $Q_1 x_1 \ldots Q_n x_n \varphi$, where each $Q_i \in \{\forall, \exists\}$ and the quantifier-free φ is called the matrix.

We do this in two steps

1. Rectify – rename apart, so no quantifier clashes.
2. Pull quantifiers to the front (via NNF)

Rectified Formulas

$\neg\big(\exists x \forall y\, R(x,y) \to \forall y \exists x\, (R(x,y) \wedge P(x))\big)$: x and y are each reused by two *different* quantifiers!
Fully bracketed, to make scopes explicit: $\neg\Big(\big(\exists x\, (\forall y\, R(x,y))\big) \to \big(\forall y\, (\exists x\, (R(x,y) \wedge P(x)))\big)\Big)$ – the two scopes never overlap, they just share names.

Rectified

A formula is rectified ("renamed apart") if no two quantifiers use the same variable, and no variable occurs both bound and free.

Renamed apart: $\neg\big(\exists x \forall y\, R(x,y) \to \forall z \exists w\, (R(w,z) \wedge P(w))\big)$

- Why OK? $\forall x.F(x) \equiv \forall y.F(y)$ whenever y is fresh (similarly for $\exists$) – renaming bound variables never changes meaning.
- Why do it? Without it, pulling a quantifier to the front can capture a variable wrongly!
  - E.g. $P(x) \wedge \forall x\, Q(x)$ – here x is *free* in $P(x)$.
  - Naively pulling $\forall x$ to the front: $\forall x\, (P(x) \wedge Q(x))$ – now x in $P(x)$ is captured, turning "some fixed x" into "for all x"!
  - Correct: rename first ($\forall x\, Q(x) \equiv \forall x'\, Q(x')$), *then* pull: $\forall x'\, (P(x) \wedge Q(x')) \equiv P(x) \wedge \forall x\, Q(x)$

Renaming Lemma

Renaming Lemma

If y does not occur in $\varphi(x)$, then $\forall x\, \varphi(x) \equiv \forall y\, \varphi(y)$ (similarly for $\exists$).

Exercise 17.1 Prove it using ND! (Recall $\forall e, \forall i$ from Lecture 14.) One direction:

1.	$\forall x\, \varphi(x)$	premise
2.	$y_0\ \varphi(y_0)$	$\forall x\, e$, 1
3.	$\forall y\, \varphi(y)$	$\forall y\, i$, 2

(box scoped by fresh y_0 – the side conditions for both rules hold precisely because y_0 is fresh. The converse runs symmetrically.)

- **Exercise 17.2** adapt this to prove the $\exists$ case.

Substitution Needs Renaming Too

Recall (PS6, Q2d): $\varphi := \exists x\,(P(y,z) \wedge \forall y\,(\neg Q(y,x) \vee P(y,z)))$.

- Substituting t for a variable is only safe when t is free for that variable in φ
- Fix, when it's not: rename the offending bound variable first (Renaming Lemma!), *then* substitute.

$\varphi[f(x)/y]$: free y sits inside $\exists x$'s scope, and $f(x)$ contains x

- Naive: $\exists x\,(P(f(x),z) \wedge \forall y(\neg Q(y,x) \vee P(y,z)))$ – the x in $f(x)$ gets captured by $\exists x$!
- Rename first: $\exists x(\ldots) \equiv \exists x'\,(P(y,z) \wedge \forall y(\neg Q(y,x') \vee P(y,z)))$
- Now substitute safely: $\exists x'\,(P(f(x),z) \wedge \forall y(\neg Q(y,x') \vee P(y,z)))$ – x in $f(x)$ stays free.

$\varphi[g(y,z)/z]$: 2nd free z sits inside $\forall y$'s scope, and $g(y,z)$ contains y

- Naive: $\exists x\,(P(y,g(y,z)) \wedge \forall y(\neg Q(y,x) \vee P(y,g(y,z))))$ – the *2nd* $g(y,z)$'s y gets captured by that very $\forall y$!
- Rename first: $\forall y(\ldots) \equiv \forall y'\,(\neg Q(y',x) \vee P(y',z))$
- Now substitute safely: $\exists x\,(P(y,g(y,z)) \wedge \forall y'(\neg Q(y',x) \vee P(y',g(y,z))))$.

More Equivalences

Recall (Lecture 13): $\varphi \equiv \psi$ iff $\{\varphi\} \models \psi$ and $\{\psi\} \models \varphi$.

- Quantifier duals: $\neg\forall x\,\varphi \equiv \exists x\,\neg\varphi$, and $\neg\exists x\,\varphi \equiv \forall x\,\neg\varphi$
- Distributivity (*same* x on both sides): $\forall x\,(\varphi \wedge \psi) \equiv (\forall x\,\varphi) \wedge (\forall x\,\psi)$, and $\exists x\,(\varphi \vee \psi) \equiv (\exists x\,\varphi) \vee (\exists x\,\psi)$

Exercise 17.3 Prove the following too – *(i) directly* (semantically), *and (ii) using ND*:

1. Pulling out of $\wedge$: $(Qx\,\varphi) \wedge \psi \equiv Qx\,(\varphi \wedge \psi)$, for $Q \in \{\forall, \exists\}$, provided $x \notin \mathit{free}(\psi)$
2. Pulling out of $\vee$: $(Qx\,\varphi) \vee \psi \equiv Qx\,(\varphi \vee \psi)$, for $Q \in \{\forall, \exists\}$, provided $x \notin \mathit{free}(\psi)$

Careful: in practice we only ever apply this to already-rectified formulas, where x doesn't occur in ψ *at all*.

Rectified Prenex Form (RPF)

A sentence is in Rectified Prenex Form if it is in prenex form $(Q_1 x_1 \ldots Q_n x_n \, \varphi)$ and additionally rectified: all the x_i are distinct, and none occurs free in φ.

- To convert to RPF: push negations to the atoms (NNF), then pull all quantifiers to the front.
- Note: NNF is *not* part of RPF definition – it's just a consequence of the method to pull quantifiers out safely.

Exercise 17.4 Convert to RPF

$\neg\big(\exists x \forall y \, R(x,y) \to \forall z \exists w \, (R(w,z) \wedge P(w))\big)$

$\equiv \; \exists x \forall y \, R(x,y) \;\wedge\; \neg \forall z \exists w \, (R(w,z) \wedge P(w))$ $\quad (\neg(A \to B) \equiv A \wedge \neg B)$

$\equiv \; \exists x \forall y \, R(x,y) \;\wedge\; \exists z \forall w \, \neg(R(w,z) \wedge P(w))$ $\quad$ (push $\neg$ through $\forall, \exists$: $\neg \forall z \, \varphi \equiv \exists z \, \neg \varphi$, $\neg \exists w \, \varphi \equiv \forall w \, \neg \varphi$)

$\equiv \; \exists x \forall y \, R(x,y) \;\wedge\; \exists z \forall w \, (\neg R(w,z) \vee \neg P(w))$ $\quad$ (De Morgan: $\neg(A \wedge B) \equiv \neg A \vee \neg B$)

$\equiv \; \exists z \, \forall w \, \exists x \, \forall y \, \big(R(x,y) \wedge (\neg R(w,z) \vee \neg P(w))\big)$ $\quad$ (pull quantifiers to the front one at a time – safe since rectified)

Skolemization

Example: Let $M(x)$: "x is a man", $W(x)$: "x is a woman". "For every man, there is a woman he likes": $\forall m \exists w\,(M(m) \rightarrow (W(w) \wedge \mathit{Likes}(m,w)))$. Introduce a function f picking, for each man m, one such woman: $\forall m\,(M(m) \rightarrow (W(f(m)) \wedge \mathit{Likes}(m, f(m))))$.

The Skolemization Procedure

- In a prenex sentence, replace each $\exists x$ by a fresh function of all $\forall$-variables preceding it (just a fresh constant if none precede it), substituting it for x in the matrix, then drop the $\exists$.
- Such a function/constant is called a Skolem function/Skolem constant.
- Work outermost $\exists$ first.

Continuing Exercise 17.4's RPF: $\exists z \forall w \exists x \forall y\,(R(x,y) \wedge (\neg R(w,z) \vee \neg P(w)))$.

- No $\forall$ before $\exists z$: introduce constant c: $\forall w \exists x \forall y\,(R(x,y) \wedge (\neg R(w,c) \vee \neg P(w)))$
- $\forall w$ precedes $\exists x$: introduce $f(w)$: $\forall w \forall y\,(R(f(w),y) \wedge (\neg R(w,c) \vee \neg P(w)))$

Skolem Normal Form

- Skolemization results in sentence with only universal quantifiers: Skolem Normal Form (SNF)
- SNF is equisatisfiable with the original formula, though generally *not* equivalent to it.

Clausal Form (for Resolution)

Example: continuing $\forall w \forall y \left(R(f(w), y) \wedge (\neg R(w, c) \vee \neg P(w))\right)$ from before.

- Finally, make the matrix into CNF – distribute $\vee$ over $\wedge$ wherever needed!
  - Here $R(f(w), y) \wedge (\neg R(w, c) \vee \neg P(w))$ is *already* a conjunction of clauses – nothing to do!
- Since $\forall$ distributes over $\wedge$:

$$(\forall w \forall y\, R(f(w), y)) \;\wedge\; (\forall w \forall y\, (\neg R(w, c) \vee \neg P(w)))$$

- Dropping the (now implicit) universal quantifiers, we get a set of clauses:

$$\{\, R(f(w), y), \quad \neg R(w, c) \vee \neg P(w) \,\}$$

Clausal form

A sentence is in clausal form if written as a set of clauses $C_1, \ldots, C_k$ with *all* variables implicitly universally quantified, and no quantifier symbols remain.

Careful: different clauses never share variables, even if they reuse a name – e.g. the w in $R(f(w), y)$ is not the same w as in $\neg R(w, c) \vee \neg P(w)$.

- This is exactly the input format for FOL resolution, even if we won't go into that!

CS228 : Logic for Computer Science

Module 3: Predicate Logic

Lecture 18: DPLL(T), SMT, and Moving on to Hoare Logic

Instructor: S. Akshay

IIT Bombay, India

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

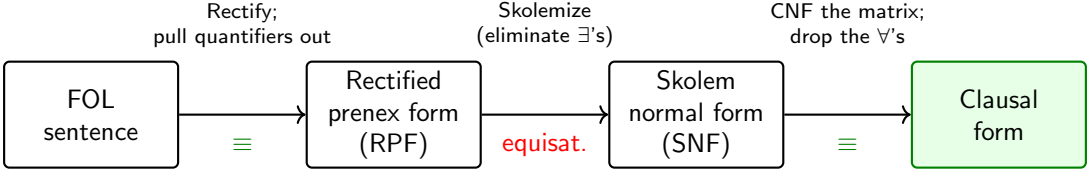
1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic
2. Semantics of predicate logic
3. Natural deduction for FOL
4. Structures, expressiveness, and limits of FOL
5. Normal forms for FOL
6. Today: DPLL(T) and SMT solving

Quick Recap: Normal Forms So Far

- Convert FOL formulas to equisatisfiable clausal form, ready for automated reasoning.



Consider: every non-negative number has a non-negative half.

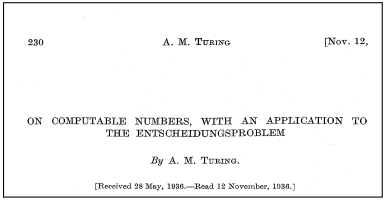
1. Write this in FOL $\varphi := \forall x \left(x \geq 0 \rightarrow \exists y \left(y \geq 0 \wedge y + y = x\right)\right)$
2. Write the RPF (one alternation) $\forall x \, \exists y \left(\neg(x \geq 0) \vee \left(y \geq 0 \wedge y + y = x\right)\right)$
3. Write the SNF (f is *new*) $\sigma := \forall x \left(\neg(x \geq 0) \vee \left(f(x) \geq 0 \wedge f(x) + f(x) = x\right)\right)$
4. Write the clausal form $\{\, \neg(x \geq 0) \vee f(x) \geq 0, \quad \neg(x \geq 0) \vee f(x) + f(x) = x \,\}$
5. What should f be in $(\mathbb{R}, 0, +, \geq, =)$? Both clauses must hold for *every* $x \in \mathbb{R}$. $f(x) := x/2$
   What if the structure is $(\mathbb{N}, 0, +, \geq, =)$? Any other f? For $x = 1$ we must have $f(1) + f(1) = 1$, but $2f(1)$ is even, so *no* f works. In fact, φ is false in $\mathbb{N}$. This does not change equisatisfiability!

Topic 18.1

The Problem of Satisfiability for FOL

The Problem of Satisfiability... for FOL

- Recall: given a propositional formula, is it satisfiable?
  - Resolution gives UNSAT proofs; DPLL/CDCL always halts with SAT/UNSAT.
- Now the same question, but for FOL: given a FOL formula, is it satisfiable?



230 A. M. TURING [Nov. 12,

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

Why is this harder?

- FOL validity and satisfiability are undecidable – no algorithm always halts with yes/no.
- Quantifiers range over infinite domains ($\mathbb{N}, \mathbb{R}, \ldots$), and FOL can encode computation – no finite truth table to brute-force.
- In fact, Turing introduced Turing machines while proving this for "Functional calculus K" (the old name for FOL)!

So do we give up? **No!**

- restrict to a fixed vocabulary and structure (a "theory") where satisfiability *is* decidable!

Theories: Two Examples

A theory T fixes a vocabulary $\mathcal{V}_T$ and a single intended $\mathcal{V}_T$-structure.

Example 1: LRA (Linear Real Arithmetic)

- Universe $\mathbb{R}$, $\mathcal{V}_T = \{0, 1, +, \leq, =\}$ (only linear comb, no $x \times y$), interpreted as usual.
- e.g, There are real numbers x, y such that $x \geq 5$ and $2x - y = 3$

Example 2: BV (fixed-width Bit-Vectors)

- A bit-vector of width n is simply a tuple $(b_{n-1}, \ldots, b_0) \in \{0,1\}^n$.
- Universe $\{0,1\}^n$, $\mathcal{V}_T = \{\&, |, \oplus, +, \leq_u, \ldots\}$ (bitwise ops applied per-bit, plus arithmetic mod 2^n reading the tuple as a number)
- e.g., For a 4-bit register w, doubling it *can overflow* – and become (unsigned) smaller than w itself (e.g. $w = 1100$, i.e. 12, doubled is 11000, truncated to 4 bits it's 1000, i.e. 8).
  $w + w <_u w$

Used in hardware/crypto/systems verification.

Satisfiability Modulo Theories

The T-satisfiability problem a.k.a. Satisfiability Modulo Theories (SMT)

Given a quantifier-free $\mathcal{V}_T$-formula, is there an assignment of *values in T's universe* that makes it true?

- For both theories we considered, T-satisfiability is decidable!
  - That is, there is an algorithm that always halts with yes/no.
  - (Not every theory qualifies – e.g. general nonlinear arithmetic over the integers doesn't.)
  - LRA: It suffices to check if a conjunction of linear (in)equalities has a solution
    - How would you check if you only had equality constraints? Gaussian elimination!
    - Use "Simplex Algorithm" to check feasibility of corresponding Linear Program.
  - BV: each variable has a finite domain (2^n values) – bit-blast into an equisatisfiable propositional formula, then run usual DPLL/CDCL !

replace each n-bit variable by n Bool variables, and each op by a circuit (AND/OR/XOR gates) over those bits.

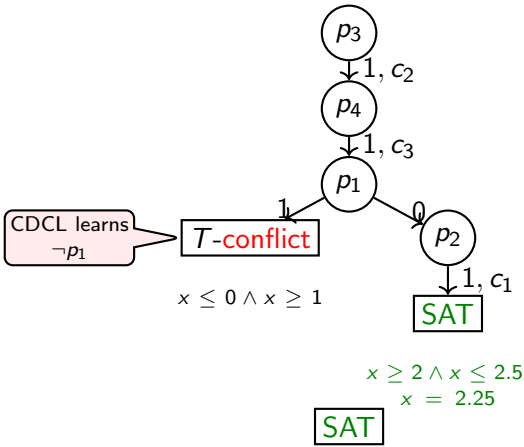
Extending DPLL to DPLL(T)

So, can we extend DPLL/CDCL to handle theories? Yes!

- Atomic $\mathcal{V}_T$-formulas – e.g. $x \leq 5$ or $x + y = 3$ – are called theory atoms.
- Take the quantifier-free $\mathcal{V}_T$-formula in CNF and replace each distinct theory atom by a fresh Boolean variable – the Boolean skeleton. $(x \leq 5 \vee x + y = 3) \wedge \neg(x \leq 5) \rightsquigarrow (p \vee q) \wedge \neg p$
- Run DPLL/CDCL on the skeleton to find a candidate assignment.
- Ask an underlying T-solver: is this *conjunction of theory literals* satisfiable in T's structure?
- If yes: SAT, done. If no: (learn a conflict clause and) keep searching.

DPLL(T): A Worked Example (LRA)

- Input: A quantifier-free LRA-formula in CNF $\varphi' := (x \leq 0 \vee x \geq 2) \wedge (x \geq 1) \wedge (x \leq 2.5)$.
- Theory atoms $\rightarrow$ Bool variables: $p_1{:=}(x \leq 0)$, $p_2{:=}(x \geq 2)$, $p_3{:=}(x \geq 1)$, $p_4{:=}(x \leq 2.5)$.
- Boolean skeleton: $c_1 \wedge c_2 \wedge c_3$, where $c_1{:=}(p_1 \vee p_2)$, $c_2{:=}p_3$, $c_3{:=}p_4$.



- p_1 was the only real decision – both values lead to a theory conflict, so φ is UNSAT.
- What if $c_3 := (x \leq 2.5)$ instead? Only the p_2 branch changes
- T-solver check: $\{x \geq 1, x \leq 2.5, x > 0, x \geq 2\}$ – SAT! witness $x = 2.25$.
- So φ' is satisfiable – the $p_1{=}0, p_2{=}1$ branch succeeded.

Adding Quantifiers

- In the modified, satisfiable version: $\varphi' := (x \leq 0 \vee x \geq 2) \wedge (x \geq 1) \wedge (x \leq 2.5)$.
- $\exists x\, \varphi'(x)$: true – the witness $x = 2.25$ comes for free: it is the model the T-solver already found!
- $\forall x\, \varphi'(x)$ is true iff $\neg\varphi'(x)$ is UNSAT – again just DPLL(T)!
  - $\neg\varphi'(x) \equiv (x > 0 \wedge x < 2) \vee x < 1 \vee x > 2.5$
  - It is SAT, e.g. x=0 $(0 < 1)$.
  - So $\forall x\, \varphi'(x)$ is false.
- Bottomline: one block of $\exists$ or of $\forall$ over a quantifier-free formula is just SAT/UNSAT. The trouble starts when quantifiers alternate!

Quantifier Alternation is Hard

$\varphi'(x) := (x \leq 0 \vee x \geq 2) \wedge (x \geq 1) \wedge (x \leq 2.5)$.

Pure existential

- $\exists x\, \varphi'(x)$: true, witness $x = 2.25$ – just the SAT problem DPLL(T) already solved!

Now alternate: $\forall y\, \exists x$

- Let $\psi(x, y) := (x \leq y \vee x \geq y{+}2) \wedge (x \geq y{+}1) \wedge (x \leq y{+}2.5)$ -same shape as φ', shifted by y.
- $\forall y\, \exists x\, \psi(x, y)$: for *every* y, need a witness x – but now it can (must!) depend on y. A single fixed x won't work for all y at once.
- Here $x := y + 2.25$ works for *every* y (the old witness, shifted) – that dependency is exactly a Skolem function!
- In general, *finding* that function (or enough instantiations to pin it down) is the hard part – that's where the real search/work in quantifier instantiation lives.

Summary

- Satisfiability for FOL is undecidable – so fix a theory T (LRA, BV, ...) where it is decidable: SMT.
- DPLL(T) = CDCL on the Boolean skeleton + a T-solver for conjunctions of theory literals.
- *Quantifier-free* is somewhat easy, depending on the theory; so is even one block of quantifiers (just SAT/UNSAT in T) – but alternating quantifiers are hard!

That wraps up our module on first-order logic

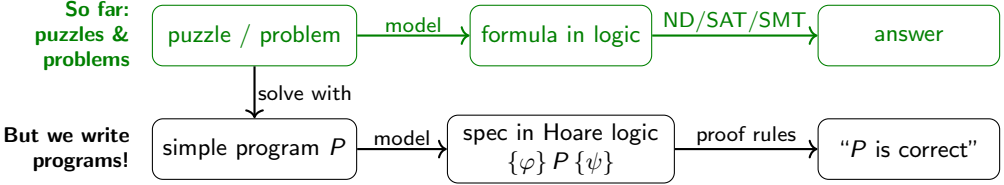
- Payoff: SMT solvers such as Z3 put much of the power of FOL to work – with decision procedures inside the theories (and heuristics beyond).
- Optional Z3 demo soon!

Topic 18.2

Introduction to Program Verification

Next topic alert!

An introduction to Program Verification



- Can we use logic to reason about programs, too?
- How? A logic of programs, Hoare logic, built on top of FOL!
- In this course: for *simple* imperative programs, we'll see how to *state* properties in logic (pre/postconditions) and *prove* them using proof rules.
- Real program verifiers (e.g., Dafny) largely automate the proof step using SMT solvers!

Remember Ariane 5? (Lecture 0)



June 4, 1996 – Ariane 5, Flight 501

- Ariane 4 code, reused in Ariane 5, overflowed – the rocket was lost within a minute of lift-off.
- On Ariane 4 that value never got so large: the code was correct under Ariane 4's assumptions, but not under Ariane 5's.
- A violated precondition!

What does "correct" mean?

"Correct" only makes sense relative to an assumed precondition and a promised postcondition

How do we formalize this in logic?

CS228 : Logic for Computer Science

Module 4: Hoare Logic and Program Verification

Lecture 19: Introduction to Hoare Logic

Instructor: S. Akshay

IIT Bombay, India

Logistics

Quiz 2

- **Friday, Oct 23rd, 6pm**
- Syllabus:
  - Module 3 (Predicate Logic/FOL): all of Lectures 12–18
  - Module 4 (Hoare logic): from Lecture 19 onwards, covered till Thursday, Oct 15th

SMT solver demo

- Postponed to next week.

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT/SMT tool demo

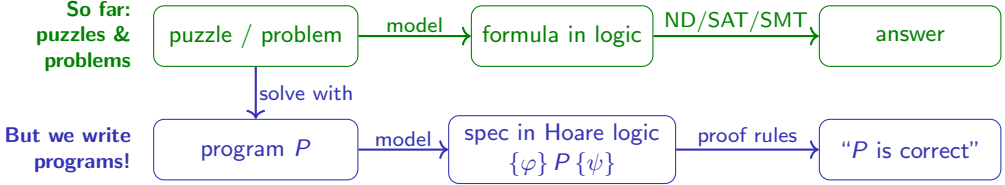
Module 3: Predicate Logic

1. Syntax of predicate logic
2. Semantics of predicate logic
3. Natural deduction for FOL
4. Structures, expressiveness, and limits of FOL
5. Normal forms for FOL
6. DPLL(T) and SMT solving

Module 4: Hoare Logic and Program Verification

Today: Introduction to Hoare logic

Recap: Hoare Logic and Program Verification



- A logic to reason about programs
- A logic of programs, Hoare logic, built on top of FOL!
- In this module: for *simple* imperative programs, we'll see how to *state* properties in logic (pre/postconditions) and *prove* them using proof rules.
- Real program verifiers (e.g., Dafny) largely automate the proof step using SMT solvers!

Consider a simple program

What does this program do? Take x as input and y as output.

	$y := 1;$
	$z := 0;$
Fac$_1$	**while** $(z \neq x)\{$
inp x	$\quad z := z + 1;$
out y	$\quad y := y * z;$
	$\}$

Trace for $x = 3$:

after	z	y	$z \neq x$?
start	0	1	yes
iter. 1	1	1	yes
iter. 2	2	2	yes
iter. 3	3	6	no (stop)

Let's call this **Fac**$_1$.

- **Exercise 19.1** What is y when the loop stops for input (i) $x = 3$ and (ii) $x = 0$? $y = 6 = 3!, y = 1 = 0!$
- The program is meant to compute the factorial of x and to store that in y.
- But does it do that? How do we show that it really computes $x!$ in y?
- What happens if we start with $x = -2$? It never stops: $z = 1, 2, 3, \ldots$ never equals -2!
- In fact, if $x \geq 0$, then **Fac**$_1$ computes $x!$ and stores it in y. Our goal is to prove this!

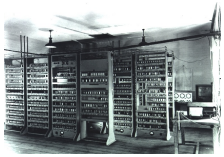
Proving programs correct

1. If $\mathbf{Fac}_1$ is run with $x \geq 0$, it terminates with $y = x!$.
2. Whenever $\mathbf{Fac}_1$ terminates, we have $y = x!$.

Our goal is to prove BOTH these statements!

- Proof goals mix a program with facts about its input/output values,
- so we treat both as mathematical/logical objects.

We explore a paradigm that can do this: Hoare Logic



EDSAC (1948)

In 1949, at a Cambridge conference, Turing asked *"How can one check a routine in the sense of making sure that it is right?"* His example was a factorial program, with assertions for correctness and a quantity that decreases and vanishes for termination – exactly our two statements!



Alan Turing

To reason about programs, we must first write them formally

- To prove things about programs, programs must be mathematical objects, just as formulas were.
- So, as for logic, we need
  - Syntax: precisely which strings are programs? (a grammar)
  - Semantics: precisely what does a program do? (how it changes the state)
- Where have we done this before? Propositional logic (Lectures 1–2) and predicate logic (Lectures 12–13)!

Topic 19.1

A Simple Programming Language

A simple (imperative) programming language

Integer expressions

$$E ::= n \mid x \mid (-E) \mid (E + E) \mid (E - E) \mid (E * E)$$

Boolean expressions

$$B ::= \top \mid \bot \mid (\neg B) \mid (B \wedge B) \mid (E \leq E)$$

Commands, i.e., programs

$$C ::= (x := E) \mid C; C \mid \textbf{if } B \ \{C\} \textbf{ else } \{C\} \mid \textbf{while } B \ \{C\}$$

where { and } mark the start and end of blocks of code

Syntax and semantics of a simple program

Fix variables $x, y, z, \ldots$, numerals/integers $n \in \{0, 1, 2, \ldots, -1, -2, \ldots\}$

$$E ::= n \mid x \mid (-E) \mid (E + E) \mid (E - E) \mid (E * E)$$
$$B ::= \top \mid \bot \mid (\neg B) \mid (B \wedge B) \mid (E \leq E)$$
$$C ::= (x := E) \mid C; C \mid \textbf{if } B \ \{C\} \textbf{ else } \{C\} \mid \textbf{while } B \ \{C\}$$

The state of the program is determined by the current value of all its variables. As the program executes, its state changes.

- $(x := E)$ is the usual assignment statement.
- $C; C$ is sequential composition.
- **if** $B \ \{C_1\}$ **else** $\{C_2\}$ evaluates boolean expression B in current state and if that result is true, then C_1 is executed, else C_2 is executed.
- **while** $B \ \{C\}$ evaluates B at current state. If B evaluates false the command terminates. Else, C is executed. When C terminates we re-evaluate B with updated state and continue. Thus, we repeatedly evaluate C as long as B is true.

The simple programming language

Exercise 19.4

1. Write a program in the above syntax to
   1.1 compute 2^n from n.
   1.2 find greatest y whose square is less than x
2. What features of a common imperative (i.e., state based) programming language like C, java, are not captured? Give 4 examples. Is it easy to modify the language to have these features?

Topic 19.2

Specifying Programs: Hoare Triples

Hoare Logic, sometimes called Floyd–Hoare Logic

Main idea: Use logic to capture what must be true before and after a command!

In general, a **Hoare triple** $\{\varphi\}\, P\, \{\psi\}$ is a formula of Hoare logic:

- φ and ψ are formulas in a base logic, e.g., predicate logic, Presburger arithmetic, etc. In our case, we will use first order logic over integers.
  E.g., for $\mathbf{Fac}_1$: $\{x \geq 0\}\ \mathbf{Fac}_1\ \{y = x!\}$
- P is an imperative program in our language
- φ is called the precondition of P and ψ the postcondition.



Robert Floyd



Tony Hoare

Floyd 1967, Hoare 1969

Turing Awards: 1978, 1980

States and formulas

States and Formulas

- A state of the program is an environment, i.e., a function α that assigns to each variable x an integer $\alpha(x)$ (as in Lecture 13).
- Assertions φ, ψ are formulas of first order logic over the vocabulary $\mathcal{V}$ with the numerals as constants, function symbols $-$ (unary) and $+, *, -$ (binary), and predicate symbols $\leq$ and $=$.
- Let $M_{\mathbb{Z}} = (\mathbb{Z}, (\ldots, -1, 0, 1, \ldots, -, +, *, \leq))$ be the $\mathcal{V}$-structure in which every symbol has its usual meaning. We write $\alpha \models \varphi$ for $M_{\mathbb{Z}}, \alpha \models \varphi$.

Exercise 19.5

Let α be a state where $\alpha(x) = -2, \alpha(y) = 5, \alpha(z) = -1$. Which of the following hold?

(a) $\alpha \models \neg(x + y < z)$

(b) $\alpha \models \forall u\,(y < u \rightarrow y * z < u * z)$

(a) holds: $x + y = 3$ is not less than $z = -1$. (b) fails: for $u = 6$, $y * z = -5 < -6 = u * z$ is false.

Examples of Hoare Triples

Some examples of correct Hoare triples

- $\{(n \geq 0) \wedge (n * n > 20)\}\ \mathrm{m} := \mathrm{n} + 1; \mathrm{m} := \mathrm{m} * \mathrm{m}\ \{\neg(m = 50)\}$ quantifier-free first order logic over integers.
- $\{\exists z\,(z + z = x)\}\ \mathrm{y} := \mathrm{x} + 2\ \{\exists z\,(z + z = y)\}$ general first order logic over integers: "if x is even, so is y".

An incorrect Hoare triple

$\{n \geq 0\}\ \mathrm{m} := \mathrm{n} + 1; \mathrm{m} := \mathrm{m} * \mathrm{m}\ \{m \geq 4\}$ – fails for $n = 0$: then $m = 1$.

Exercise 19.6

Give 3 "correct" and 3 "incorrect" Hoare triples.

How do we formalize the notion of correctness?

Correctness and semantics of Hoare Triples

Partial correctness

A triple $\{\varphi\}P\{\psi\}$ is satisfied under partial correctness, denoted $\models_{par} \{\varphi\}P\{\psi\}$ if, for all states that satisfy φ, the state resulting from P's execution satisfies the postcondition ψ, provided P actually terminates.

Total correctness

A triple $\{\varphi\}P\{\psi\}$ is satisfied under total correctness, denoted $\models_{tot} \{\varphi\}P\{\psi\}$ if, for all states that satisfy φ, P when executed is guaranteed to terminate and the resulting state satisfies the postcondition ψ.

Examples: Recall the program $\mathbf{Fac}_1$ from earlier,

- $\{\top\}\ \mathbf{Fac}_1\ \{y = x!\}$ satisfies partial correctness, but not total correctness.
- $\{x \geq 0\}\ \mathbf{Fac}_1\ \{y = x!\}$ satisfies partial AND total correctness.

Examples

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

Unk$_1$ *inp* x *out* y	$a := x + 1;$ **if** $(a - 1 = 0)\{$ $\quad y := 1;$ $\}$ **else** $\{$ $\quad y := a;$ $\}$

- $\{\top\}$ **Fac$_1$** $\{y = x!\}$ satisfies partial correctness, but NOT total correctness.
- $\{x \geq 0\}$ **Fac$_1$** $\{y = x!\}$ satisfies partial AND total correctness.

Exercise 19.7

1. What does Unk_1 do? Take x as input and y as output.
2. Write a Hoare triple that captures the spec. Does it hold under partial/total correctness?

Program variables and logical variables

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

Fac$_2$ *inp* x *out* y	$y := 1;$ **while** $(x \neq 0)\{$ $\quad y := y * x;$ $\quad x := x - 1;$ $\}$

- **Fac**$_2$ is another program that computes the factorial.
- $\{x \geq 0\}$ **Fac**$_1$ $\{y = x!\}$ satisfies partial and total correctness.
- Qn: What about $\{x \geq 0\}$ **Fac**$_2$ $\{y = x!\}$?
- How do we correct this? What is the correct specification? Need to remember initial value!
- we obtain $\{x = x_0 \wedge x \geq 0\}$ **Fac**$_2$ $\{y = x_0!\}$ x_0 is called a logical variable.

CS228 : Logic for Computer Science

Module 4: Hoare Logic and Program Verification

Lecture 20: Proof Rules for Hoare Logic

Instructor: S. Akshay

IIT Bombay, India

Logistics

Recall: Quiz 2

- **Friday, Oct 23rd, 6pm**
- Syllabus:
  - Module 3 (Predicate Logic/FOL): all of Lectures 12–18
  - Module 4 (Hoare logic): from Lecture 19 up to and including Thursday's lecture (Oct 15th)

Optional SMT solver demo

- **Wednesday, Oct 14th, 3:30pm**
- Venue: CC 103

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic
2. Semantics of predicate logic
3. Natural deduction for FOL
4. Structures, power and limits of FOL
5. Normal forms, DPLL(T) and SMT

Module 4: Hoare Logic and Program Verification

1. Introduction to Hoare logic
2. Today: Proof rules for Hoare logic

Recap: Hoare triples and correctness

A Hoare triple $\{\varphi\}\ P\ \{\psi\}$: precondition, program, postcondition

- Partial correctness ($\models_{par}$): *if* P terminates when started in a state satisfying φ, then ψ holds in the final state.
- Total correctness ($\models_{tot}$): started in any state satisfying φ, P *does terminate*, and ψ holds in the final state.

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

Qn: Recall **Fac**$_1$. Which of these hold?

	partial	total
$\{\top\}$ **Fac**$_1$ $\{y = x!\}$	✓	×
$\{x \geq 0\}$ **Fac**$_1$ $\{y = x!\}$	✓	✓

- The two statements from Lecture 19: statement 1 (terminates with $y = x!$ if $x \geq 0$) is the *total* entry for $x \geq 0$; statement 2 (whenever it terminates, $y = x!$) is the *partial* entry for $\top$.

total correctness = partial correctness + termination

Recap: Partial correctness and non-termination

Exercise 20.1 (a): $\mathbf{Fac}_1$ never stops when $x = -2$.

- Why does $\{\top\}$ $\mathbf{Fac}_1$ $\{y = x!\}$ still hold under partial correctness?
- Vacuous! No terminating run starts with $x < 0$, so no run can violate $y = x!$.

Exercise 20.1 (b): Let P be **while** $\top$ $\{x := 0\}$.

- Does $\{\top\}$ P $\{\bot\}$ hold under partial correctness? Under total correctness?
  - Note: the $\top$ in the guard is part of the *program* (a Boolean expression, like **while**); the plain $\top$ and $\bot$ in the braces are *assertions*, i.e., formulas of the logic (you can think of them as abbreviations of, e.g., $0 \leq 0$ and $\neg(0 \leq 0)$).
- Partial: yes (it never terminates, so again vacuous). Total: no.

A partially correct program may do nothing useful: partial correctness says nothing about termination!

Program variables and logical variables

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

Fac$_2$ *inp* x *out* y	$y := 1;$ **while** $(x \neq 0)\{$ $\quad y := y * x;$ $\quad x := x - 1;$ $\}$

- **Fac**$_2$ is another program that computes the factorial.
- $\{x \geq 0\}$ **Fac**$_1$ $\{y = x!\}$ satisfies partial and total correctness.
- Qn: What about $\{x \geq 0\}$ **Fac**$_2$ $\{y = x!\}$?
- How do we correct this? What is the correct specification? Need to remember initial value!
- We obtain $\{x = x_0 \wedge x \geq 0\}$ **Fac**$_2$ $\{y = x_0!\}$ x_0 is called a logical variable.

For a Hoare triple $\{\varphi\}P\{\psi\}$, its logical variables are those that are free in φ or ψ and do not occur in P.

Topic 20.1

Proving Programs: Proof Rules

A proof calculus for program correctness

Rule1: Sequential Composition

$$\frac{\{\varphi\}\ C_1\ \{\eta\} \qquad \{\eta\}\ C_2\ \{\psi\}}{\{\varphi\}\ C_1; C_2\ \{\psi\}}\ \text{Seq. Composition}$$

- Easier to read the rules bottom-up!
- To show that ψ holds on running $C_1; C_2$ starting from φ, we have to find η and show that η holds after running C_1 from φ and from η running C_2 results in ψ holding.
- η is called a midcondition.

Example:

$$\frac{\{y+z>4\}\ y := y+z-1\ \{y>3\} \qquad \{y>3\}\ x := y+4\ \{x>7\}}{\{y+z>4\}\ y := y+z-1; x := y+4\ \{x>7\}}\ \text{Seq. Composition}$$

Hoare Logic: Proof rule for Assignment

Rule 2: Assignment

$$\frac{}{\{\psi[E/x]\}\ x := E\ \{\psi\}}\ \text{Assignment}$$

- This rule is an axiom.
- $\psi[E/x]$ refers to replacing every *free* occurrence of x in ψ by E.
- Example: $\{x + 1 + 5 = y\}\ x := x + 1\ \{x + 5 = y\}$
- Why are we doing this "backward"? Would forward work?
- No! $\{\varphi\}\ x := E\ \{\varphi[E/x]\}$ does not hold in general. **Exercise 20.2**: Show this by giving a counter-example.

Exercise 20.3: Let P be $x := 2$. Are these true? (i) $\{2 = 2\}\ P\ \{x = 2\}$, (ii) $\{x = 2\}\ P\ \{2 = 2\}$, (iii) $\{x = 3\}\ P\ \{x = 3\}$, (iv) $\{2 = 4\}\ P\ \{x = 4\}$. Sol: Only (i), (ii), (iv) are true.

Hoare Logic: Proof rule for If-then-Else

Rule 3: If-then-Else

$$\frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

- In the *then*-branch we know B held; in the *else*-branch we know B failed. We use this information in the sub-proofs!

Example

$$\frac{\{x > 4 \wedge z > 0\}\ x := x + z\ \{x > 3\} \qquad \{x > 4 \wedge \neg(z > 0)\}\ x := x - 1\ \{x > 3\}}{\{x > 4\}\ \textbf{if}\ (z > 0)\ \{x := x + z\}\ \textbf{else}\ \{x := x - 1\}\ \{x > 3\}}\ \text{If-then-Else}$$

(Each premise needs to be proved (using other rules).)

Hoare Logic: Strengthening Preconditions, Weakening Postconditions

Rule 4: Strengthen the precondition, weaken the postcondition

$$\frac{\vdash_{AR} \varphi \to \varphi' \qquad \{\varphi'\}\ C\ \{\psi'\} \qquad \vdash_{AR} \psi' \to \psi}{\{\varphi\}\ C\ \{\psi\}}\ \text{Implied}$$

- The rule links Hoare logic to the base logic: $\vdash_{AR} \varphi \to \varphi'$ means the implication is provable in natural deduction for first order logic, taking facts about arithmetic (e.g., $\forall x\,(x = x + 0)$) as premises.
- Strengthen the precondition = assume more than needed; weaken the postcondition = promise less.
- Note that this works regardless of C!

Example

$$\frac{\vdash_{AR} x > 9 \to x > 7 \qquad \{x > 7\}\ x := x - 4\ \{x > 3\} \qquad \vdash_{AR} x > 3 \to x > 2}{\{x > 9\}\ x := x - 4\ \{x > 2\}}\ \text{Implied}$$

Hoare Logic: Proof rule for partial correctness of While Loop

Rule 5: partial-while

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Partial-while}$$

- η is a loop-invariant: it holds before the loop and is preserved by every iteration (that is what the premise says).
- Partial correctness!
  - If loop does not terminate, Hoare triple is vacuously satisfied.
  - If it does terminate, then $\eta \wedge \neg B$ must be satisfied at termination.
- Example

$$\frac{\{y = x + z \wedge (z \neq 0)\}\ x := x + 1; z := z - 1\ \{y = x + z\}}{\{y = x + z\}\ \textbf{while}\ (z \neq 0)\ \{x := x + 1; z := z - 1\}\ \{y = x + z \wedge (z = 0)\}}$$

Topic 20.2

Using the Proof Rules: Proof Outlines and Loop Invariants

A proof calculus for program correctness

$$\frac{\{\varphi\}\ C_1\ \{\eta\} \qquad \{\eta\}\ C_2\ \{\psi\}}{\{\varphi\}\ C_1; C_2\ \{\psi\}}\ \text{Seq. Comp.}$$

$$\frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

$$\frac{}{\{\psi[E/x]\}\ x := E\ \{\psi\}}\ \text{Assignment}$$

$$\frac{\vdash_{AR} \varphi \rightarrow \varphi' \qquad \{\varphi'\}\ C\ \{\psi'\} \qquad \vdash_{AR} \psi' \rightarrow \psi}{\{\varphi\}\ C\ \{\psi\}}\ \text{Implied}$$

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Partial-while}$$

We say sequent $\vdash_{par} \{\varphi\}\ P\ \{\psi\}$ is valid if ψ can be derived from φ by applying these rules!

This proof system is sound so we can use this to infer partial correctness, i.e., $\models_{par}$.

Using the proof rules in practice!

Writing trees is unwieldy (and difficult in latex!). So we use a different convention...

Example 1: Show that $\vdash_{par} \{y < 5\}\ x := y + 1\ \{x < 6\}$

$$
\begin{array}{ll}
\{y < 5\} & \\
\{y + 1 < 6\} & \text{Implied} \\
x := y + 1 & \\
\{x < 6\} & \text{Assignment}
\end{array}
$$

1. Start by writing down the program
2. Write down the pre- and post-conditions before and after
3. Work backwards using rule corresponding to prog. statement to get a mid condition
4. At any midcondition, you can go further up by using other rules

Another example

Consider program P, which is $z := x; z := z + y; u := z$. **Exercise 20.4**. Write a Hoare triple!

Example 2: Show that $\vdash_{par} \{\top\}\ P\ \{u = x + y\}$

$\{\top\}$	
$\{x + y = x + y\}$	Implied
$z := x;$	
$\{x + y = z + y\}$	Assignment
$z := z + y;$	
$\{x + y = z\}$	Assignment
$u := z;$	
$\{u = x + y\}$	Assignment

- Though you constructed the proof from bottom to up, you can read it top to bottom!
- Implied rule always refers to immediate preceding line.
- Weakest pre-condition

Using the If-then-Else rule

$$\frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

$$\frac{\{\varphi_1\}\ C_1\ \{\psi\} \qquad \{\varphi_2\}\ C_2\ \{\psi\}}{\{(B \to \varphi_1) \wedge (\neg B \to \varphi_2)\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

We will use a slight variant of the rule in practice...

Exercise 20.5

- Show that any instance of the modified rule in a proof can be replaced by an instance of the original rule and instances of rule Implied.
- Is the converse true?

CS228 : Logic for Computer Science

Module 4: Hoare Logic and Program Verification

Lecture 21: Proving Loops Correct: Invariants and Variants

Instructor: S. Akshay

IIT Bombay, India

Logistics

Recall: Quiz 2

- **Friday, Oct 23rd, 6pm**
- Venue: CC 101/103/105 (*to be confirmed, will be announced on Piazza*)
- Syllabus:
  - Module 3 (Predicate Logic/FOL): all of Lectures 12–18
  - Module 4 (Hoare logic): from Lecture 19 up to and including today's lecture (Thursday, Oct 15th)

Topics Covered till date

Module 1: Propositional Logic

1. Syntax
2. Semantics
3. Formal proof systems
4. Normal Forms

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL
3. CDCL
4. SAT tool demo

Module 3: Predicate Logic

1. Syntax of predicate logic
2. Semantics of predicate logic
3. Natural deduction for FOL
4. Structures, power and limits of FOL
5. Normal forms, DPLL(T) and SMT

Module 4: Hoare Logic and Program Verification

1. Introduction to Hoare logic
2. Proof rules for partial correctness
3. Today: Loop invariants and variants

Recap: A proof calculus for program correctness

$$\frac{\{\varphi\}\ C_1\ \{\eta\} \qquad \{\eta\}\ C_2\ \{\psi\}}{\{\varphi\}\ C_1; C_2\ \{\psi\}}\ \text{Seq. Comp.} \qquad \frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

$$\frac{}{\{\psi[E/x]\}\ x := E\ \{\psi\}}\ \text{Assignment} \qquad \frac{\vdash_{AR} \varphi \rightarrow \varphi' \qquad \{\varphi'\}\ C\ \{\psi'\} \qquad \vdash_{AR} \psi' \rightarrow \psi}{\{\varphi\}\ C\ \{\psi\}}\ \text{Implied}$$

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Partial-while}$$

We say sequent $\vdash_{par} \{\varphi\}\ P\ \{\psi\}$ is valid if ψ can be derived from φ by applying these rules!

This proof system is sound so we can use this to infer partial correctness, i.e., $\models_{par}$.

The dreaded 8am Pop quiz!

Consider program **Swap** defined as $t := x;\ x := y;\ y := t$, which is meant to swap x and y.

Look at the Hoare triple $\{x = x_0 \wedge y = y_0\}$ **Swap** $\{x = y_0 \wedge y = x_0\}$.

1. What are the pre and post conditions of this Hoare triple?
2. What are the program variables and what are the logical variables?
3. Show that this Hoare triple is valid, i.e., $\vdash_{par} \{x = x_0 \wedge y = y_0\}$ **Swap** $\{x = y_0 \wedge y = x_0\}$.

Solution

$\{x = x_0 \wedge y = y_0\}$

$\{y = y_0 \wedge x = x_0\}$ Implied

$t := x;$

$\{y = y_0 \wedge t = x_0\}$ Assignment

$x := y;$

$\{x = y_0 \wedge t = x_0\}$ Assignment

$y := t;$

$\{x = y_0 \wedge y = x_0\}$ Assignment

- x_0 and y_0 are logical variables: they remember the initial values (recall **Fac**$_2$)!

Exercise 21.1: Swap without a temporary

Show the same triple for the program $x := x + y;\ y := x - y;\ x := x - y$.

Topic 21.1

Proof Outlines and Loop Invariants

Recall: the If-then-Else rule and its variant

The rule

$$\frac{\{\varphi \wedge B\}\ C_1\ \{\psi\} \qquad \{\varphi \wedge \neg B\}\ C_2\ \{\psi\}}{\{\varphi\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else}$$

The variant

$$\frac{\{\varphi_1\}\ C_1\ \{\psi\} \qquad \{\varphi_2\}\ C_2\ \{\psi\}}{\{(B \to \varphi_1) \wedge (\neg B \to \varphi_2)\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{If-then-Else (variant)}$$

- **Quick question:** we know ψ and the two branches, but not φ. Which rule helps us *compute* a precondition?
- The variant: push ψ through C_1 and C_2 to get φ_1 and φ_2, then combine them as above.
- Together with Implied, the two rules are equivalent (**Exercise 20.5**).

Using the If-then-Else rule. E.g., $\vdash_{par} \{\top\}$ **Succ** $\{y = x + 1\}$ holds

$\{\top\}$	
$\{((x = 0) \rightarrow (0 = x)) \wedge (\neg(x = 0) \rightarrow (x + 1 = x + 1))\}$	Implied
$\{((x + 1 - 1 = 0) \rightarrow (1 = x + 1)) \wedge$	
$(\neg(x + 1 - 1 = 0) \rightarrow (x + 1 = x + 1))\}$	Implied
$a := x + 1;$	
$\{((a - 1 = 0) \rightarrow (1 = x + 1)) \wedge (\neg(a - 1 = 0) \rightarrow (a = x + 1))\}$	Assignment
if $(a - 1 = 0)\{$	
$\{1 = x + 1\}$	If-then-Else
$y := 1;$	
$\{y = x + 1\}$	Assignment
$\}$ **else** $\{$	
$\{a = x + 1\}$	If-then-Else
$y := a;$	
$\{y = x + 1\}$	Assignment
$\}$	
$\{y = x + 1\}$	If-then-Else

- **Quick question:** is $(B \rightarrow \varphi_1) \wedge (\neg B \rightarrow \varphi_2)$ the same as $(B \wedge \varphi_1) \vee (\neg B \wedge \varphi_2)$?
- Pushing ψ backwards like this is Dijkstra's weakest precondition calculus (1975): φ is $wp(\textbf{if } \ldots, \psi)$.



Edsger W. Dijkstra (1930–2002)

Are these still relevant 50 years later?



DOI:10.1145/3777544

Formal verification of AI workflows.

BY ERIK MEIJER

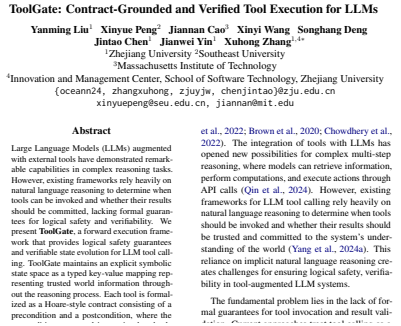
Guardians of the Agents

AGENTIC APPLICATIONS—AI SYSTEMS empowered to take autonomous actions by calling external tools—are the current rage in software development. They promise efficiency, convenience, and reduced human intervention. Giving autonomous agents access to that these risks of ceding control to autonomous agents are sufficiently threatening that their use should be entirely prohibited.[a]

To mitigate the risks inherent in agentic applications, we propose a safety paradigm rooted in mathematical proof verification. In this design pattern, the AI agent is required to generate formal proofs demonstrating the safety of planned actions before being authorized to execute them. This approach parallels existing real-world practices (for example, credit card checksums, banknote security features), where complexity lies in production, but verification remains simple and efficient. Moreover, it extends the mechanism used by widely adopted computing platforms such as Java and .NET, where code undergoes a process of bytecode verification prior to execution.[b] Bytecode verification ensures crucial safety guarantees—notably memory safety, type correctness, and proper access control. Our proposal naturally extends this established principle, adapting it effectively to the emerging domain of agentic computations, thus providing robust assurances against harmful or unintended agent behavior.

The current state of the art in AI safety is to rely on evaluations (evals), which aim to identify potential risks, vulnerabilities, or harmful behaviors

E. Meijer, *Guardians of the Agents*, CACM 69(1), Jan. 2026



ToolGate: Contract-Grounded and Verified Tool Execution for LLMs

Yanming Liu[1] Xinyue Peng[2] Jiannan Cao[3] Xinyi Wang Songhang Deng
Jintao Chen[1] Jianwei Yin[1] Xuhong Zhang[1,4*]
[1]Zhejiang University [2]Southeast University
[3]Massachusetts Institute of Technology
[4]Innovation and Management Center, School of Software Technology, Zhejiang University
{oceann24, zhangxuhong, zjuyjw, chenjintao}@zju.edu.cn
xinyuepeng@seu.edu.cn, jiannan@mit.edu

Abstract

Large Language Models (LLMs) augmented with external tools have demonstrated remarkable capabilities in complex reasoning tasks. However, existing frameworks rely heavily on natural language reasoning to determine when tools can be invoked and whether their results should be committed, lacking formal guarantees for logical safety and verifiability. We present **ToolGate**, a forward execution framework that provides logical safety guarantees and verifiable state evolution for LLM tool calling. ToolGate maintains an explicit symbolic state space as a typed key-value mapping representing trusted world information throughout the reasoning process. Each tool is formalized as a Hoare-style contract consisting of a precondition and a postcondition, where the

et al., 2022; Brown et al., 2020; Chowdhery et al., 2022). The integration of tools with LLMs has opened new possibilities for complex multi-step reasoning, where models can retrieve information, perform computations, and execute actions through API calls (Qin et al., 2024). However, existing frameworks for LLM tool calling rely heavily on natural language reasoning to determine when tools should be invoked and whether their results should be trusted and committed to the system's understanding of the world (Yang et al., 2024a). This reliance on implicit natural language reasoning creates challenges for ensuring logical safety, verifiability in tool-augmented LLM systems.

The fundamental problem lies in the lack of formal guarantees for tool invocation and result vali-

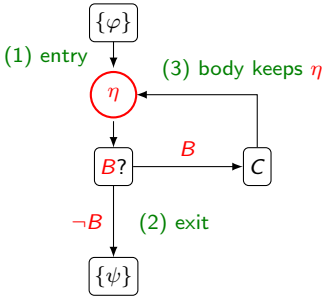
Liu et al., *ToolGate*, Findings of ACL 2026

> verification process relies on preconditions, postconditions, and invariants, ensuring only mathematically safe workflows proceed to execution.

CACM 2026, p. 51

Tackling the partial while!

- An invariant η holds before and after every iteration, even if not during! It is an induction hypothesis on the number of iterations: (1) is the base case, (3) the inductive step.
- **Exercise 21.2.** Give an example of a trivial invariant for any loop! Why does that not suffice?



The factorial example revisited

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

- What is the invariant here?
- **Exercise 21.3**
  - Starting with $x = 6$, write the values of x, y, z before the loop and after each iteration.
- Notice that $y = z!$ holds every time the guard is tested!
- But for this to be our loop inv candidate we should check 2 other things.

- (1) Pre-condn of while loop implies it: $\vdash_{AR} (y = 1 \wedge z = 0) \rightarrow (y = z!)$ is valid.
- (2) Together with neg of guard, implies post-condn: $\vdash_{AR} (y = z! \wedge x = z) \rightarrow (y = x!)$ is valid.

So, let's write the full proof for the factorial program now!

The factorial example solved!

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}$$

Partial-while applied, with $\eta := y = z!$ and $B := z \neq x$

(1) entry, (2) exit, (3) body keeps η

$\{\top\}$	
$\{1 = 0!\}$	Implied
$y := 1;$	
$\{y = 0!\}$	Assignment
$z := 0;$	
(1)$\{y = z!\}$	Assignment
while $(z \neq x)\{$	
(3)$\{y = z! \wedge z \neq x\}$	Inv Hyp. and Guard
$\{y \cdot (z + 1) = (z + 1)!\}$	Implied
$\quad z := z + 1$	
$\{y \cdot z = z!\}$	Assignment
$\quad y := y * z$	
$\{y = z!\}$	Assignment
$\}$	
(2)$\{y = z! \wedge \neg(z \neq x)\}$	Partial-while
$\{y = x!\}$	Implied

Another Factorial Example

Fac$_2$	$y := 1;$
inp x	**while** $(x \neq 0)\{$
out y	$\quad y := y * x;$
	$\quad x := x - 1;$
	$\}$

Exercise 21.4 Consider program **Fac**$_2$

1. Does the same invariant (as for **Fac**$_1$) work for this? Why or why not?
2. Find a correct invariant (hint: use x_0) and show that
   $\vdash_{par} \{x = x_0 \wedge x \geq 0\}$ **Fac**$_2$ $\{y = x_0!\}$

In practice, use SMT Solvers to automate the checks.

- Proof steps like these can be handed to an SMT solver such as Z3 (a wrong invariant, e.g. $y = x!$, then gets a counterexample). Which theory is needed?
- Can it *synthesize* invariants? Not always! You (or an LLM?) have to provide it.
- Optional demo exercise with Z3 in next problem sheet!

Topic 21.2

Total Correctness

Total correctness

- **Quick question:** what happens if we run $\mathbf{Fac}_1$ with $x = -1$?
- The invariant $y = z!$ keeps holding, but z never reaches x: the loop *never exits*! So $\{\top\}\ \mathbf{Fac}_1\ \{y = x!\}$ is only partially correct.
- The partial correctness rules developed till now do not say anything if a program P loops indefinitely.
- Can we extend the proof calculus to show that programs also terminate?
- The only rule that causes looping is the while condn, so the only thing to change is the while rule!

Total correctness: the core idea

Core idea!

- for partial correctness of while loop, we have an invariant.
- for termination of while loop, we need something that varies but only a bounded no. of times.
  - In fact, we look for an integer expression that decreases every loop iteration, but always remains non-negative.
  - hence, it can decrease only finitely many times and hence the loop terminates.
  - Such an expression is called a variant.

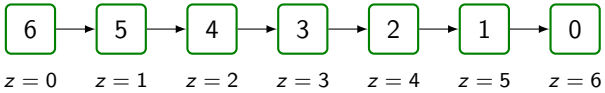


Revisiting the Factorial example

	$y := 1;$
	$z := 0;$
Fac$_1$	**while** $(z \neq x)\{$
inp x	$\quad z := z + 1;$
out y	$\quad y := y * z;$
	$\}$

- What is the variant here?
- Can you find an integer expression whose value decreases at each loop iteration?
- easy! the expression $x - z$.
- Note that it is always nonnegative (as long as $x \geq 0$ initially).
- When it is 0, the loop terminates!

$x - z$ (for $x = 6$):



Proof rule for total correctness of while loops

For while loops

Total correctness = Partial correctness + Variant

$$\frac{\{\eta \wedge B \wedge 0 \leq E = E_0\}\ C\ \{\eta \wedge 0 \leq E < E_0\}}{\{\eta \wedge 0 \leq E\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Total-while}$$

- Variant E is the integer expression
  - whose value is equal to the logical variable E_0 before the loop body is executed
  - and after one execution of the loop body, the value is strictly less than E_0 but still non-negative.
- η is the invariant as before
- For $\mathbf{Fac}_1$: $E = x - z$

CS228 : Logic for Computer Science

Module 4: Hoare Logic and Program Verification

Lecture 22: Total Correctness and Wrapping up

Instructor: S. Akshay

IIT Bombay, India

Logistics

Quiz 2

- **Tomorrow, Friday, Oct 23rd, 6pm**
- Venue:
  - CC 101: Roll No. 901–950
  - CC 103: Roll No. 951–1050
  - CC 105: Roll No. 1051–1100
  - PwD students who wish to avail compensatory time: KR 125
- No cheatsheets allowed

Syllabus

- Module 3 (Predicate Logic/FOL): all of Lectures 12–18
- Module 4 (Hoare logic): Lectures 19–21, up to partial correctness and loop invariants; *variants and total correctness are not included*

Recap: A proof calculus for program correctness

$$\frac{\{\varphi\}\ C_1\ \{\eta\} \qquad \{\eta\}\ C_2\ \{\psi\}}{\{\varphi\}\ C_1; C_2\ \{\psi\}}\ \text{Seq. Comp.}$$

$$\frac{\{\varphi_1\}\ C_1\ \{\psi\} \qquad \{\varphi_2\}\ C_2\ \{\psi\}}{\{(B \to \varphi_1) \wedge (\neg B \to \varphi_2)\}\ \textbf{if}\ B\ \{C_1\}\ \textbf{else}\ \{C_2\}\ \{\psi\}}\ \text{ITE}$$

$$\frac{}{\{\psi[E/x]\}\ x := E\ \{\psi\}}\ \text{Assignment}$$

$$\frac{\vdash_{AR} \varphi \to \varphi' \qquad \{\varphi'\}\ C\ \{\psi'\} \qquad \vdash_{AR} \psi' \to \psi}{\{\varphi\}\ C\ \{\psi\}}\ \text{Implied}$$

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Partial-while}$$

$$\frac{\{\eta \wedge B \wedge 0 \leq E = E_0\}\ C\ \{\eta \wedge 0 \leq E < E_0\}}{\{\eta \wedge 0 \leq E\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Total-while}$$

$\vdash_{par} \{\varphi\}\ P\ \{\psi\}$: derivable with Partial-while; $\vdash_{tot} \{\varphi\}\ P\ \{\psi\}$: with Total-while instead

These proof systems are sound: $\vdash_{par}$ implies $\models_{par}$, and $\vdash_{tot}$ implies $\models_{tot}$.

Recall: Partial-while on $\mathbf{Fac}_1$

$\{y = z!\}$

while $(z \neq x)$ $\{$

$\{y = z! \wedge z \neq x\}$	Inv Hyp. and Guard
$\{y \cdot (z+1) = (z+1)!\}$	Implied
$z := z + 1$	
$\{y \cdot z = z!\}$	Assignment
$y := y * z$	
$\{y = z!\}$	Assignment

$\}$

$\{y = z! \wedge \neg(z \neq x)\}$ Partial-while

$$\frac{\{\eta \wedge B\}\ C\ \{\eta\}}{\{\eta\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}$$

- Premise = the body: η is $y = z!$, B is $(z \neq x)$, C is $z := z + 1$; $y := y * z$
- Conclusion = the whole loop

Topic 22.1

Total Correctness

From partial to total correctness of while loops

For while loops

Total correctness = Partial correctness + Variant

$$\frac{\{\eta \wedge B \wedge 0 \leq E = E_0\}\ C\ \{\eta \wedge 0 \leq E < E_0\}}{\{\eta \wedge 0 \leq E\}\ \textbf{while}\ B\ \{C\}\ \{\eta \wedge \neg B\}}\ \text{Total-while}$$

- Variant E is the integer expression
  - whose value is equal to the logical variable E_0 before the loop body is executed
  - and after one execution of the loop body, the value is strictly less than E_0 but still non-negative.
- η is an invariant as before
- For **Fac**$_1$: $E = x - z$
- **Quick recall:** the variant must decrease in every iteration. Why must it also stay non-negative?
- Otherwise it could decrease forever and the loop need not terminate.

Fac$_1$ *inp* x *out* y	$y := 1;$ $z := 0;$ **while** $(z \neq x)\{$ $\quad z := z + 1;$ $\quad y := y * z;$ $\}$

Finally: Total correctness for the factorial example

	$y := 1;$
	$z := 0;$
Fac$_1$	**while** $(z \neq x)\{$
inp x	$\quad z := z + 1;$
out y	$\quad y := y * z;$
	$\}$

Show that

$\vdash_{tot} \{x \geq 0\}$ **Fac**$_1$ $\{y = x!\}$

Step 1: We guess

- Invariant: $y = z!$
- Variant $x - z$

(1) entry, (2) exit, (3) body keeps η and decreases E

$\{x \geq 0\}$

$\{1 = 0! \wedge 0 \leq x - 0\}$ Implied

$y := 1;$

$\{y = 0! \wedge 0 \leq x - 0\}$ Assignment

$z := 0;$

(1)$\{y = z! \wedge (0 \leq x - z)\}$ Assignment

while $(z \neq x)\{$

(3)$\{y = z! \wedge z \neq x \wedge 0 \leq x - z = E_0\}$ Inv Hyp. and Guard

$\{y \cdot (z + 1) = (z + 1)! \wedge 0 \leq x - (z + 1) < E_0\}$ Implied

$\quad z := z + 1$

$\{y \cdot z = z! \wedge 0 \leq x - z < E_0\}$ Assignment

$\quad y := y * z$

$\{y = z! \wedge 0 \leq x - z < E_0\}$ Assignment

$\}$

(2)$\{y = z! \wedge \neg(z \neq x)\}$ Total-while

$\{y = x!\}$ Implied

Some exercises

Fac$_2$ *inp* x *out* y	$y := 1;$ **while** $(x \neq 0)\{$ $\quad y := y * x;$ $\quad x := x - 1;$ $\}$

Mult *inp* x, y *out* z	$a := 0;$ $z := 0;$ **while** $(a \neq y)\{$ $\quad z := z + x;$ $\quad a := a + 1;$ $\}$

Unk$_2$ *inp* x, y *out* d	$r := x;$ $d := 0;$ **while** $(r \geq y)\{$ $\quad r := r - y;$ $\quad d := d + 1;$ $\}$

Exercise 22.1

1. What is a variant for **Fac**$_2$?
2. Show that
   $\vdash_{tot} \{x = x_0 \wedge x \geq 0\}$ **Fac**$_2$ $\{y = x_0!\}$

Exercise 22.2

1. What is a variant for **Mult**?
2. Show that
   $\vdash_{tot} \{y \geq 0\}$ **Mult** $\{(z = x \cdot y)\}$

Exercise 22.3

1. What is a variant for **Unk**$_2$?
2. Is $\vdash_{tot} \{y \neq 0\}$ **Unk**$_2$ $\{(x = d \cdot y + r) \wedge (r < y)\}$? If yes, prove it, else correct it.

Finding a variant can be hard: Collatz

Collatz *inp* x	$c := x$; **while** $(c \neq 1)\{$ **if** $(c \bmod 2 = 0)$ $c := c/2$; **else** $c := 3 * c + 1$; $\}$

- $x = 5$: $c = 5, 16, 8, 4, 2, 1$
- $x = 172$: 32 iterations; $x = 123456789$: 177 iterations
- **Quick question:** is c a variant? Is there one? c is not: it grows when c is odd!
- Nobody knows whether it always terminates (the Collatz conjecture)
- **Therefore** nobody knows whether a *variant* exists for this program either!

A few points to note and conclude

We "guessed" the invariant and variant!

- too weak *or* too strong: the proof fails. No algorithm finds the right invariant or variant in general (undecidable)!
- humans, search or an LLM can guess; a solver checks

Annotations help programmers

- programmer gives the "intended" invariant / variant
- checking is largely automated by SMT solvers, e.g. Z3; the other steps are mechanical

Tools infer midconditions

- KeY-Hoare, Dafny, Why3, Frama-C/WP: weakest preconditions + SMT solver
- from annotations or from code; code alone is not always enough!

Beyond: heaps, dynamic data, pointers

- *Separation logic* (Reynolds, O'Hearn)
- Meta's Infer is built on it

Topics Covered till date

Module 1: Propositional Logic

1. Syntax and semantics
2. Natural deduction
3. Normal Forms

Applications: logic puzzles, checking arguments

Module 2: The Problem of Satisfiability

1. Propositional Resolution
2. DPLL and CDCL
3. SAT solver tool demo

Applications: HW/SW verification, planning

Module 3: Predicate Logic

1. Syntax and semantics of predicate logic
2. Natural deduction for FOL
3. Structures, power and limits of FOL
4. Normal forms and DPLL(T)

Applications: databases, SMT solvers

Module 4: Hoare Logic and Program Verification

1. Hoare triples; partial vs. total correctness
2. Proof rules for partial correctness
3. Loop invariants
4. Variants and total correctness

Applications: verifying code

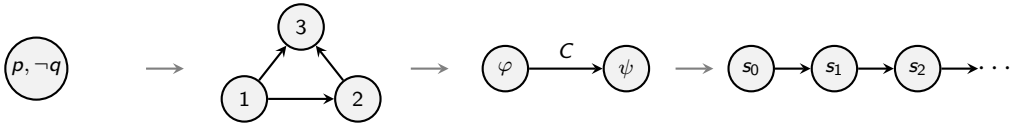
Next: Module 5, Temporal Logic — LTL, CTL and model checking

Topic 22.2

Module 5: Temporal Logic

The story so far

Propositional logic **Predicate logic** **Hoare logic** **Temporal logic**



a *static snapshot*: which facts are true?

a *richer* snapshot: objects $1, 2, 3$ and the relation $<$; formulas like $\forall x \exists y \ x < y$

two snapshots: before $\rightarrow$ after, then it halts

a whole *film*: infinitely many snapshots!

But why would a program need infinitely many snapshots?

Beyond programs that stop

- Hoare logic: a program starts, *halts*, and we compare before and after
- But an operating system, a lift controller, a pacemaker should *never* stop!
- We want to say: "every request is eventually granted", "the lift never moves with its doors open"
- **Quick question:** can we write these as Hoare triples? Not really! These programs have no "end"!

Amir Pnueli, 1977

- introduced temporal logic to reason about programs ("The temporal logic of programs", FOCS 1977)
- Turing Award, 1996, "for seminal work introducing temporal logic into computing science"



Amir Pnueli

Temporal Logics

In propositional and predicate (first-order) logic

- Models or assignments were static
- A formula is either true or false in a given model
- So we could for e.g., say $m \models \varphi$.

Idea of Temporal logics

- What if models are richer and dynamic.
- i.e., Models have states and transitions.
- A formula's truth may change depending on the state of a given model!
- So we could for e.g., say $M, s \models \varphi$.

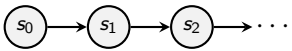
Quick recall: what was a *model* in propositional logic? In first-order logic? A valuation; a structure. Now: a graph of states and transitions!

Notions of Time

Temporal logics are dynamic

- Truth of a formula depends on which state within the model you are in currently.
- So you have to talk about how the formula evolves over time.

Linear-time → LTL

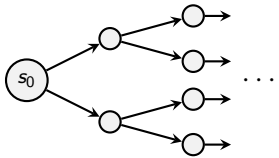


view time as a path (or a set of paths):
one future at each moment



Loki: the Sacred Timeline,
the TVA prunes every branch

Branching-time → CTL



view time as a tree, rooted at present:
many possible futures



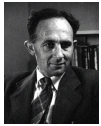
Doctor Strange: millions of futures,
only one where we win

Time can be modelled in other ways too

Not the only way to distinguish:

- You can also differentiate discrete and continuous time
- There are logics for each one!
- But we will focus on two of them: LTL and CTL.

A little history



Arthur Prior
late 1950s:
tense logic



Amir Pnueli
1977: temporal logic
for programs



Edmund Clarke
1981: CTL



E. Allen Emerson
1981: CTL



Joseph Sifakis
early 1980s:
model checking

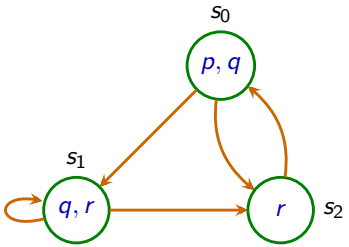
Clarke, Emerson and Sifakis shared the 2007 Turing Award for founding *model checking*.

Still relevant today? Model checking where failure is expensive

- Space: NASA's *FRET*: structured English $\rightarrow$ temporal logic (e.g. grasping spent rocket stages)
- Blockchain: Ethereum's 3SF consensus checked with *TLA+* and Apalache (2025)
- Industry: *TLA+ Foundation* (2023: AWS, Oracle, Microsoft); AWS's *Kani* found Firecracker bugs

Specify in temporal logic, check by model checking: wherever failure is expensive!

Transition systems as models



A directed graph: each node (state) contains the atomic propositions true in that state

L is like a valuation: in propositional logic, $m \subseteq AP$ assigned truth values to all propositions; now each state has its own $L(s) \subseteq AP$.

A Transition system (also called a *Kripke structure*) is a triple $M = (S, \rightarrow, L)$ where,

- S is a set of states
- $\rightarrow$ is a binary relation on S called transition relation s.t., $\forall s \in S, \exists s' \in S$ with $s \rightarrow s'$.
- $L : S \rightarrow \mathcal{P}(AP)$ where
  - AP is a set of atomic propositions: statements about system, e.g., *Printer is busy*
  - 2^{AP} is its power set: the set of all subsets.

CS228 : Logic for Computer Science

Module 5: Temporal Logic

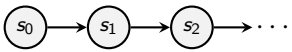
Lecture 23: Linear-time Temporal Logic (LTL)

Instructor: S. Akshay

IIT Bombay, India

Recap: Why temporal logic?

Linear-time → LTL

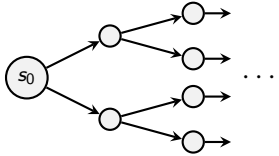


look along *one path*:
one future at each moment



Loki: the Sacred Timeline,
the TVA prunes every branch

Branching-time → CTL



view time as a tree, rooted at present:
many possible futures

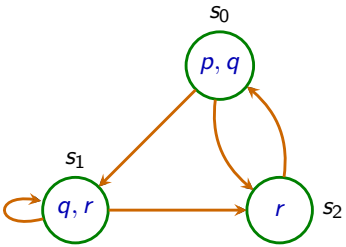


Doctor Strange: millions of futures,
only one where we win

Topic 23.1

Transition Systems

Transition systems as models



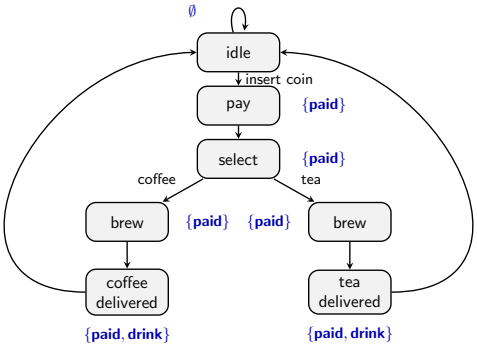
A directed graph: each node (state) contains the atomic propositions true in that state

Exercise 23.1

1. Let $AP = \{p, q, r\}$. List all elements of $\mathcal{P}(AP)$.
2. Write the formal description of M in the diagram.
3. Can M have a deadlocked state, a state from which there is no outgoing transition?
4. If we remove the deadlock condition from the definition, does it matter?

A real transition system: a vending machine

"A vending machine takes a coin, brews coffee or tea, then delivers it."



Atomic propositions $AP = \{\textbf{paid}, \textbf{drink}\}$: the user has paid; a drink is delivered.

Quick questions

1. Which states can follow *select*? two different states, both called *brew*: one for coffee, one for tea!
2. What is the label of *idle*? $\emptyset$: no atomic proposition is true there, a legal label.

Which properties of this machine would we like to state?

"A drink is never delivered before it is paid for."

"Once it is paid for, a drink is eventually delivered."

"Once a drink is delivered, user has to pay before getting a drink again."

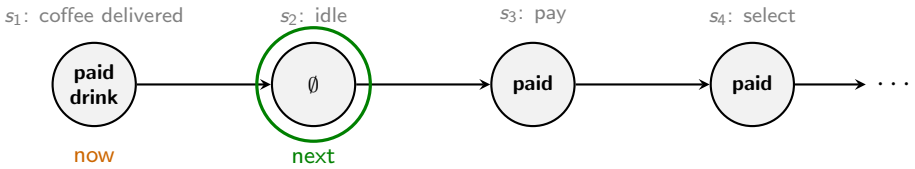
Each one talks about how states follow one another over time: how can we write this in logic?

Topic 23.2

Temporal Connectives

Time by example: next (X)

"If a drink is delivered now, the user has to pay again in the *next* state."



drink $\rightarrow X$ $\neg$**paid**

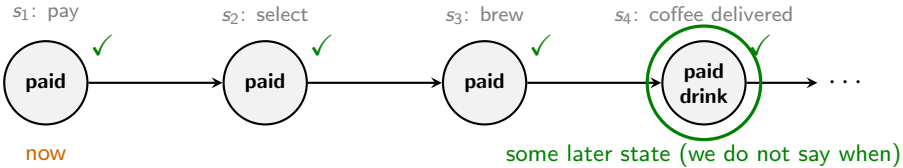
$X\varphi$: φ holds in the "neXt state"

Exercise 23.2 Does the above path, at the current state s_1 (marked *now*), satisfy

1. **drink** $\rightarrow X$ **paid**? No: **paid** does not hold at s_2.
2. **paid** $\rightarrow X$ **paid**? No: **paid** holds at s_1 but not at s_2.
3. **drink** $\rightarrow X\,X$ **paid**? Yes: **paid** holds at s_3.

Time by example: eventually (F)

"The machine *eventually* delivers a drink (**drink**)."



F **drink**

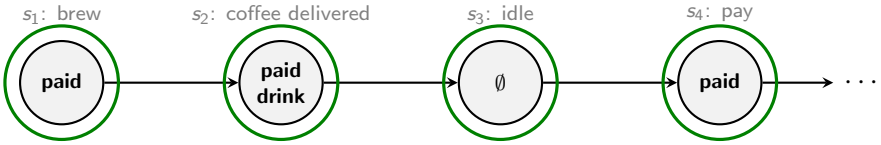
$F\varphi$: φ holds in "some Future state" (possibly the current one)

Exercise 23.3

1. At which states of this path does F **drink** hold? At $s_1, \ldots, s_4$: every state upto and including drink.
2. Does F **drink** hold if **drink** already holds in the current state? Yes: the current state counts.
3. If **paid** holds now, is a drink eventually delivered on this path?
   That is, does **paid** $\rightarrow F$ **drink** hold on this path? Yes: **paid** holds at s_1, and so does F **drink**.

Time by example: always (G)

"Whenever a drink is delivered (**drink**), it has been paid for (**paid**)."



drink $\rightarrow$ **paid** holds in every state, from now on

$$G\,(\textbf{drink} \rightarrow \textbf{paid})$$

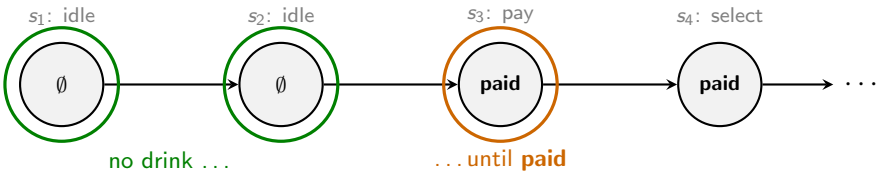
$G\varphi$: φ holds "Globally": in the current state and all later ones

Exercise 23.4

1. Does G **paid** hold on this path? No: **paid** fails at s_3 (idle).
2. Does $G(\textbf{paid} \rightarrow \textbf{drink})$ hold on this path? No: **paid** holds at s_1 but **drink** does not.
3. What would a path look like on which $G(\textbf{drink} \rightarrow \textbf{paid})$ fails? A state where **drink** holds but **paid** does not: a free drink!
4. Write in LTL: "every time the user has paid, a drink is eventually delivered". $G(\textbf{paid} \rightarrow F\,\textbf{drink})$

Time by example: until (U)

"No drink is delivered *until* the user has paid."



¬drink U paid

$\varphi\, U\, \psi$: φ holds "Until" ψ holds (and ψ does hold)

Exercise 23.5

1. Does ¬**drink** U **paid** hold at s_2? Yes: **paid** holds at s_3, and ¬**drink** holds at s_2.
2. Does it hold at s_3, where **paid** already holds? Yes: ψ holds immediately, so φ is not needed.
3. If the user never pays (idle forever), does it hold? No: ψ must eventually hold.
4. Which weaker property says "no drink unless the user pays", allowing that the user never pays? ¬**drink** W **paid**, the weak until (as we will see later).

Topic 23.3

LTL Syntax

Linear-time Temporal Logic (LTL): Syntax

The syntax of LTL in Backus–Naur form (BNF):

$$\varphi ::= \top \mid \bot \mid p \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi)$$
$$\mid (X\varphi) \mid (F\varphi) \mid (G\varphi) \mid (\varphi \ U \ \varphi)$$

where p is an atomic proposition from set AP.

- $\top$, $\bot$ and the usual propositional connectives $\neg, \wedge, \vee, \rightarrow$ are as in propositional logic.
- In addition we have temporal connectives:
  - $X\varphi$: φ holds in the "neXt state"
  - $F\varphi$: φ holds in "some Future state" (possibly the current one)
  - $G\varphi$: φ holds "Globally": in the current state and all later ones
  - $\varphi \, U \, \psi$: φ holds "Until" ψ holds (and ψ does hold)
- Brackets can be dropped using binding rules ("$>$" reads "binds more tightly than"):

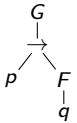
$$\text{Unary } (\neg, X, F, G) \ > \ U \ > \ \vee, \wedge \ > \ \rightarrow$$

Parsing LTL formulae

$$\varphi ::= \top \mid \bot \mid p \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi)$$
$$\mid (X\varphi) \mid (F\varphi) \mid (G\varphi) \mid (\varphi U \varphi)$$

Examples

1. $G\ (p \rightarrow F\ q)$



2. $F\ (p \rightarrow G\ r) \vee \neg q\ U\ p$
3. $F\ p \rightarrow G\ r \vee \neg q\ U\ p$

Parse trees

- As usual we can write Parse trees for the formulas
- A subformula of an LTL formula φ is any formula ψ whose parse tree is a subtree of φ's parse tree.

Exercise 23.6

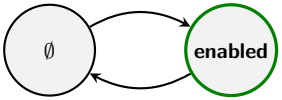
1. Which of these are LTL formulas?
   1.1 $(p\ G\ q)$
   1.2 $(p\ U\ q)\ U\ r \rightarrow\ F\ p$
   1.3 $(p \wedge q) \rightarrow (U\ r) \vee G\ p$
2. Write the parse trees of Examples 2 and 3 and of the formula in part 1 that is an LTL formula.
3. Write all the subformulas of Example 1 above.

Putting it together: writing specifications

What can you express in this logic?

Consider atomic propositions: **req**, **grant**, **enabled**, **running**.

1. Response: In any state, if there is a **request** (of some resource), then it will be eventually **granted**.
   $G(\textbf{req} \rightarrow F\ \textbf{grant})$
2. Some process is **enabled** infinitely often in every computation path. $G\ F\ \textbf{enabled}$
3. If a process is **enabled** infinitely often, then it **runs** infinitely often. **Exercise 23.7!**



every path keeps coming back to **enabled**

Exercise 23.8 Consider the specification: An upwards moving lift at second floor does not change its direction if someone has pressed button for 5th floor. Come up with atomic propositions and write an LTL formula for this spec.

safety: nothing bad ever happens, e.g., $G(\textbf{drink} \rightarrow \textbf{paid})$

liveness: something good eventually happens, e.g., $F\ \textbf{drink}$

Topic 23.4

LTL Semantics

Semantics of LTL

$$\varphi ::= \top \mid \bot \mid p \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi)$$
$$\mid (X\varphi) \mid (F\varphi) \mid (G\varphi) \mid (\varphi U \varphi)$$

- A path in $M = (S, \rightarrow, L)$ is an infinite sequence of states $s_1, s_2, \ldots$ in S s.t., $\forall i \geq 1, s_i \rightarrow s_{i+1}$. Write $\pi = s_1 \rightarrow s_2 \rightarrow \ldots$ and denote by π^i the suffix of π starting at s_i.

Given LTL formula φ, model $M = (S, \rightarrow, L)$ and path $\pi = s_1 \rightarrow \ldots$ in M, the satisfaction relation $\pi \models \varphi$, is defined inductively:

1. $\pi \models \top, \pi \not\models \bot$
2. $\pi \models p$ iff $p \in L(s_1)$
3. $\neg, \wedge, \vee, \rightarrow$ are as usual!
4. $\pi \models X\ \varphi$ iff $\pi^2 \models \varphi$.
5. $\pi \models F\ \varphi$ iff $\exists i \geq 1$, s.t. $\pi^i \models \varphi$.
6. $\pi \models G\ \varphi$ iff $\forall i \geq 1$, $\pi^i \models \varphi$.
7. $\pi \models \varphi\ U\ \psi$ iff $\exists i \geq 1$, s.t. $\pi^i \models \psi$ and $\forall j \in \{1, \ldots, i-1\}$, $\pi^j \models \varphi$.

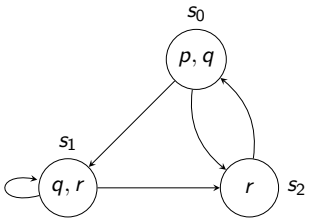
Exercise 23.9 If $\pi = s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow \cdots$, what is π^3? Unfold $\pi \models X\,X\,p$ by the definition. $\pi^3 = s_3 \rightarrow s_4 \rightarrow \cdots$; $\pi \models X\,X\,p$ iff $\pi^2 \models X\,p$ iff $\pi^3 \models p$.

Semantics of LTL continued

- We defined a satisfaction relation between paths and LTL formulas.
- But to verify systems, we would like to say the whole model satisfies a formula.

Given LTL formula φ, model $M = (S, \to, L)$ and $s \in S$, we define $M, s \models \varphi$ if, for every path π of M starting from s, we have $\pi \models \varphi$.

Let's see an example.



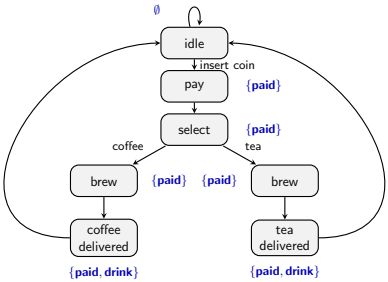
Consider M (left).

1. $M, s_0 \models (p \wedge q)$ ✓
2. $M, s_0 \models \neg r$ ✓
3. $M, s_0 \models X\ r$
4. $M, s_0 \models X(q \wedge r)$ ×
5. $M, s_0 \models G\neg(p \wedge r)$

Which of the following hold?

6. $M, s_2 \models G\ r$
7. $\forall s, M, s \models (F(\neg q \wedge r) \to F\ G\ r)$
8. $M, s_0 \models G\ F\ p$ ×
9. $M, s_0 \models (G\ F\ p \to G\ F\ r)$

Satisfaction on the vending machine



Exercise 23.10 Which of these hold in the vending machine M (with the labels shown)?

1. $M, idle \models F\,\textbf{drink}$? No: the path idle, idle, idle, ... never delivers a drink.
2. $M, pay \models F\,\textbf{drink}$? Yes: every path from pay reaches a delivered state.
3. $M, idle \models G(\textbf{drink} \rightarrow \textbf{paid})$? Yes: **drink** holds only in the delivered states, and they have **paid**.
4. $M, idle \models (\neg\textbf{drink}\ U\ \textbf{paid})$? No: on the path idle, idle, ..., **paid** never holds. (The weak until W would hold.)

CS228 : Logic for Computer Science

Module 5: Temporal Logic

Lecture 24: LTL Equivalences, and on to CTL

Instructor: S. Akshay

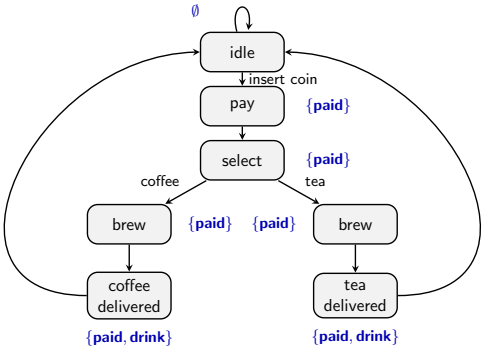
IIT Bombay, India

Recap: LTL syntax

$$\varphi ::= \top \mid \bot \mid p \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi)$$
$$\mid (X\varphi) \mid (F\varphi) \mid (G\varphi) \mid (\varphi U \varphi)$$

Examples (atomic propositions **paid** and **drink**):

- "If a drink is delivered now, the user has to pay again in the *next* state."
  drink $\rightarrow X \neg$ **paid**
- "The machine *eventually* delivers a drink."
  F **drink**
- "Whenever a drink is delivered, it has been paid for."
  G (**drink** $\rightarrow$ **paid**)
- "No drink is delivered *until* the user has paid."
  $\neg$**drink** U **paid**



Recap: LTL semantics

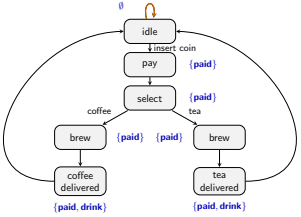
- A path in a model $M = (S, \to, L)$ is an infinite sequence $\pi = s_1 \to s_2 \to \ldots$ of states with $s_i \to s_{i+1}$ for all $i \geq 1$; π^i is the suffix of π starting at s_i.
- $\pi \models \varphi$ is defined by induction on φ:

1. $\pi \models \top, \pi \not\models \bot$
2. $\pi \models p$ iff $p \in L(s_1)$
3. $\pi \models \neg\varphi$ iff $\pi \not\models \varphi$; $\wedge, \vee, \to$ are as usual!
4. $\pi \models X\ \varphi$ iff $\pi^2 \models \varphi$.
5. $\pi \models F\ \varphi$ iff $\exists i \geq 1$, s.t. $\pi^i \models \varphi$.
6. $\pi \models G\ \varphi$ iff $\forall i \geq 1$, $\pi^i \models \varphi$.
7. $\pi \models \varphi\ U\ \psi$ iff $\exists i \geq 1$, s.t. $\pi^i \models \psi$ and $\forall j \in \{1, \ldots, i-1\}$, $\pi^j \models \varphi$.

For a model M and a state s: $M, s \models \varphi$ if *every* path π of M starting from s satisfies φ.

Exercise 24.1 (continued)

3. In the vending machine, does $M, idle \models F$ **drink** hold? No: the path idle, idle, ... never delivers a drink.



Two ways to say the same thing?

"A drink is never delivered unless it is paid for."

Two ways to write it:

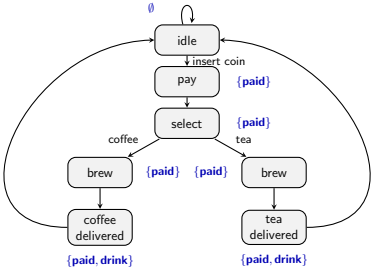
(a) $G(\textbf{drink} \rightarrow \textbf{paid})$

(b) $\neg F(\textbf{drink} \wedge \neg \textbf{paid})$

Exercise 24.2 Do (a) and (b) agree (both true or both false) on

1. the orange path π? Both true: no state has **drink** without **paid**.
2. the path idle, idle, ...? Both true: no state has **drink**.
3. a path of a faulty machine that delivers a free drink (a state with **drink** but not **paid**)? Both false: (a) fails at that state, and $F(\textbf{drink} \wedge \neg \textbf{paid})$ holds, so (b) fails too.

Same answer on every path of every model? That is what we call *equivalence*.



Equivalence and properties

Equivalence

LTL formulas φ and ψ are (semantically) equivalent, denoted $\varphi \equiv \psi$ if for all models M and all paths π in M we have:

$$\pi \models \varphi \text{ iff } \pi \models \psi$$

- As in prop logic, if two formulae φ, ψ are equivalent and if φ occurs as a subformula of some formula χ, then in χ we can replace φ by ψ without changing its meaning.
- All propositional equivalences carry over to LTL, e.g., $\neg\neg\varphi \equiv \varphi$ and $\varphi \to \psi \equiv \neg\varphi \vee \psi$. **Why?** On a path, $\neg, \wedge, \vee, \to$ mean exactly what they mean in propositional logic.

Duals

- $\wedge, \vee$ are duals of each other: $\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi$, $\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi$,
- So are F, G! $\neg G\varphi \equiv F\neg\varphi$, $\neg F\varphi \equiv G\neg\varphi$. What is the dual of X? X is its own dual: $\neg X\varphi \equiv X\neg\varphi$.

Proving equivalences

Claim: $\neg G\varphi \equiv F\neg\varphi$. Proof for an arbitrary model M and path π in M:

$$
\begin{aligned}
& \pi \models \neg G\varphi && \\
\iff\ & \pi \not\models G\varphi && \text{semantics of } \neg \\
\iff\ & \text{not } (\forall i \geq 1,\ \pi^i \models \varphi) && \text{semantics of } G \\
\iff\ & \exists i \geq 1,\ \pi^i \not\models \varphi && \text{negating } \forall \\
\iff\ & \exists i \geq 1,\ \pi^i \models \neg\varphi && \text{semantics of } \neg \\
\iff\ & \pi \models F\neg\varphi && \text{semantics of } F
\end{aligned}
$$

Exercise 24.3 Prove the other duals in the same way:

1. $\neg F\varphi \equiv G\neg\varphi$
2. $\neg X\varphi \equiv X\neg\varphi$

Using equivalences to rewrite a specification

Exercise 24.4 Show that (a) $G(\textbf{drink} \rightarrow \textbf{paid})$ and (b) $\neg F(\textbf{drink} \wedge \neg\textbf{paid})$ from the earlier slide are equivalent, using the duals and propositional equivalences.

Solution: start from (b) and push the negation inwards

$$
\begin{aligned}
& \neg F(\textbf{drink} \wedge \neg\textbf{paid}) && \\
\equiv\ & G\,\neg(\textbf{drink} \wedge \neg\textbf{paid}) && \text{dual: } \neg F\varphi \equiv G\neg\varphi \\
\equiv\ & G\,(\neg\textbf{drink} \vee \neg\neg\textbf{paid}) && \text{De Morgan, inside } G \\
\equiv\ & G\,(\neg\textbf{drink} \vee \textbf{paid}) && \neg\neg\varphi \equiv \varphi \\
\equiv\ & G\,(\textbf{drink} \rightarrow \textbf{paid}) && \varphi \rightarrow \psi \equiv \neg\varphi \vee \psi
\end{aligned}
$$

Each step replaces a subformula by an equivalent one, which is allowed by the substitution remark on the definition slide.

Distributivity

Exercise 24.5 Which of these equivalences hold? Prove the ones that do, unfolding the semantics as on the previous slides.

1. $F(\varphi \vee \psi) \equiv F\varphi \vee F\psi$ ✓
2. $F(\varphi \wedge \psi) \equiv F\varphi \wedge F\psi$ ×
3. $G(\varphi \wedge \psi) \equiv G\varphi \wedge G\psi$ ✓
4. $G(\varphi \vee \psi) \equiv G\varphi \vee G\psi$ ×

- F distributes over $\vee$ and G over $\wedge$.

Exercise 24.6 How do we show that (2) and (4) do *not* hold?

How do we show a non-equivalence?

The same attempt for $F(\varphi \wedge \psi) \equiv F\varphi \wedge F\psi$

$\pi \models F(\varphi \wedge \psi)$

$\iff$ $\exists i \geq 1,\ (\pi^i \models \varphi$ and $\pi^i \models \psi)$

$\implies$ $(\exists i \geq 1,\ \pi^i \models \varphi)$ and $(\exists j \geq 1,\ \pi^j \models \psi)$

$\iff$ $\pi \models F\varphi \wedge F\psi$

$\impliedby$ fails: the two witnesses i and j can be different

Counterexample: φ holds only at s_1 and ψ only at s_2.

This is not a proof that $F(\varphi \wedge \psi) \not\equiv F\varphi \wedge F\psi$: it only shows that *this* attempt at proving the equivalence failed.

To show a non-equivalence $\varphi \not\equiv \psi$ we need to exhibit

- one model M and one path π in M,
- on which they differ: $\pi \models \varphi$ but $\pi \not\models \psi$ (or the other way round)!

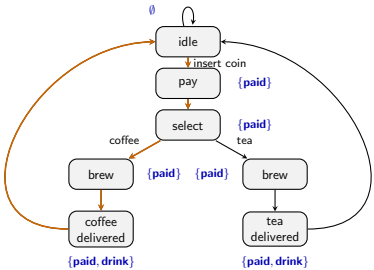
Solution to Exercise 24.6

$F(\varphi \wedge \psi) \not\equiv F\varphi \wedge F\psi$: take the vending machine, $\varphi = \neg$paid, $\psi =$ drink

$\pi =$ idle, pay, select, brew, coffee, idle, ...

- $\pi^1 =$ idle $\models \neg$**paid**, so $\pi \models F\neg$**paid**
- $\pi^5 =$ coffee delivered $\models$ **drink**, so $\pi \models F$ **drink**
- no state has $\neg$**paid** and **drink** (a delivered state has **paid**), so $\pi \not\models F(\neg$**paid** $\wedge$ **drink**$)$

Hence $\pi \models F\varphi \wedge F\psi$ but $\pi \not\models F(\varphi \wedge \psi)$.



Adapt the above idea to show the non-equivalence (4) as well: $G(\varphi \vee \psi) \not\equiv G\varphi \vee G\psi$.

Do we need all these connectives?

- $G\varphi \equiv \neg F \neg \varphi$: G can be written with F and $\neg$.
- $\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi)$: $\vee$ can be written with $\neg$ and $\wedge$.
- $\varphi \rightarrow \psi \equiv \neg\varphi \vee \psi$: so can $\rightarrow$.

Definition: adequate (called *universal* in ProblemSheet1)

A set of connectives is adequate/universal for a logic, if all other connectives in that logic can be expressed by combinations within this set. E.g., for prop logic $\{\neg, \vee\}$ and $\{\top, \neg, \wedge\}$ are adequate.

Exercise 24.7 Show that $\{\top, \neg, \wedge, X, U\}$ is adequate for LTL.

- First observe that all non-temporal connectives are done.
- Then, for F, write it in terms of U. Finally, write G in terms of F.

We still write the others in the logic for simplicity. Sometimes we use even more!

- Release: the dual of Until $\varphi \; R \; \psi := \neg(\neg\varphi \; U \; \neg\psi)$
- Weak-Until: $\varphi \; W \; \psi := \varphi \; U \; \psi \vee \; G \; \varphi$. E.g., $\neg$**drink** W **paid**, "no drink unless the user pays": **paid** here does not need to hold.

Limitations of linear-time logic: no branching!

- LTL is evaluated on paths. A state of a system satisfies an LTL formula if all paths from the state satisfy it.
- Implicitly quantifies universally over paths.
- Problem: LTL cannot assert existence of path.
- Soln: To check if there exists a path satisfying φ, just check if all paths satisfy $\neg\varphi$.
- But: What if you mix universal and existential path quantifiers? then just taking negation won't work!
  - E.g., "from every state, we can reach a state where a drink is delivered": for *all* states, *there exists* a path to **drink**.

Computation Tree Logic (CTL)

Introduce two quantifiers A to express 'all paths' and E for 'exists a path'.

- There exists a reachable state in which q holds: $E\ F\ q$
- Whenever a state satisfying p is reached, there is a path where q holds continuously after that: $A\ G\ (p \rightarrow E\ G\ q)$

CTL Syntax

$$\varphi ::= \bot \mid \top \mid p \mid (\neg\varphi) \mid (\varphi \vee \varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \rightarrow \varphi) \mid$$
$$AX\varphi \mid EX\varphi \mid AF\varphi \mid EF\varphi \mid AG\varphi \mid EG\varphi \mid A[\varphi\ U\ \varphi] \mid E[\varphi\ U\ \varphi]$$

where $p \in AP$ atomic propositions.

- $A\psi$ means along all paths the temporal formula ψ holds.
- $E\psi$ means there exists a path in which the temporal formula ψ holds.
- All temporal formula must have A or E in front and all occurrences of A or E must be followed by temporal formulas
- Similar binding as for LTL: unary connectives ($\neg$, AX, EX, AF, EF, AG, EG) bind most tightly

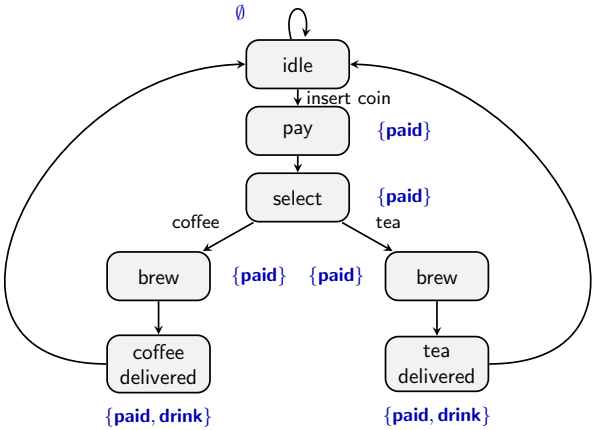
Exercise 24.9 Which of the following are well-formed CTL formulas? Draw their parse tree.

1. $AG\ (q \rightarrow EG\ r)$ ✓
2. $EFG\ r$ ×
3. $F[r\ Uq]$ ×
4. $A[p\ U\ A[q\ U\ r]]$ ✓
5. $A[(r\ U\ q) \wedge (p\ U\ r)]$ ×
6. $A[(r\ \wedge\ q)\ U\ (\neg p \rightarrow\ r)]$ ✓

CTL on the vending machine

Exercise 24.10 In the vending machine, which of these hold at *idle*? (E: some path from the state, A: all paths from the state.)

1. EF **drink**? ✓ some path reaches a delivered state.
2. AF **drink**? × the path idle, idle, . . . never delivers one.
3. $EG\,\neg$**drink**? ✓ the same path, idle, idle, . . . , has no drink.
4. $AG\,EF$ **drink**? ✓ from every state, we can reach a state where a drink is delivered.



Semantics of CTL

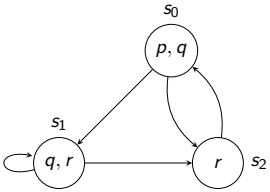
Let $M = (S, \rightarrow, L)$ be a model, $s \in S$, φ a CTL formula. Then the satisfaction relation $M, s \models \varphi$ is defined inductively, as follows,

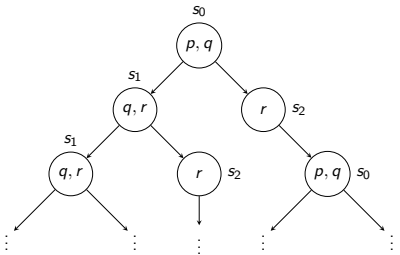
- $M, s \models p$ iff $p \in L(s)$
- $M, s \models AX\varphi$ iff $\forall s_1, s \rightarrow s_1$, we have $M, s_1 \models \varphi$.
- $M, s \models AG\varphi$ iff $\forall$ paths $s = s_1 \rightarrow s_2 \ldots$, $\forall s_i$, we have $M, s_i \models \varphi$.
- $M, s \models AF\varphi$ iff $\forall$ paths $s = s_1 \rightarrow s_2 \ldots$, $\exists s_i$ in path with $M, s_i \models \varphi$.
- $M, s \models A[\varphi\ U \psi]$ iff $\forall$ paths $s = s_1 \rightarrow s_2 \ldots$, $\exists s_i$ such that $M, s_i \models \psi$ and $\forall j < i$, $M, s_j \models \varphi$.
- $\top, \bot, \neg, \vee, \wedge, \rightarrow$ defined as usual
- $M, s \models EX\varphi$ iff $\exists s_1, s \rightarrow s_1$, we have $M, s_1 \models \varphi$.
- $M, s \models EG\varphi$ iff $\exists$ path $s = s_1 \rightarrow s_2 \ldots$, $\forall s_i$, we have $M, s_i \models \varphi$.
- $M, s \models EF\varphi$ iff $\exists$ path $s = s_1 \rightarrow s_2 \ldots$, $\exists s_i$ in path with $M, s_i \models \varphi$.
- $M, s \models E[\varphi\ U \psi]$ iff $\exists$ path $s = s_1 \rightarrow s_2 \ldots$, $\exists s_i$ such that $M, s_i \models \psi$ and $\forall j < i$, $M, s_j \models \varphi$.

If the outermost connective starts with E (resp. A), then for some (resp. all) path(s) starting from s, 'LTL formula' obtained by removing the top E (resp. A) is satisfied.

An example: Consider M from before...

From any given state, we can unwind the system as an infinite tree of computation paths.





Exercise 24.11 Which of the following hold? Why or why not? Justify!

1. $M, s_0 \models (p \wedge q) \wedge \neg r$ ✓
2. $M, s_0 \models EX(q \wedge r)$ ✓
3. $M, s_0 \models AX(q \wedge r)$ ×
4. $M, s_0 \models \neg EF(p \wedge r)$ ✓
5. $M, s_1 \models EG(q \wedge r)$ ✓
6. $M, s_0 \models AF\ r$ ✓
7. $M, s_0 \models E[(p \wedge q)\ U\ (q \wedge r)]$ ✓
8. $M, s_0 \models A[(p \wedge q)\ U\ (q \wedge r)]$ ×

Some of these were also in the LTL lecture last time. Which ones?

CS228 : Logic for Computer Science

Module 5: Temporal Logic

Lecture 25: CTL, Expressiveness, and Model Checking

Instructor: S. Akshay

IIT Bombay, India

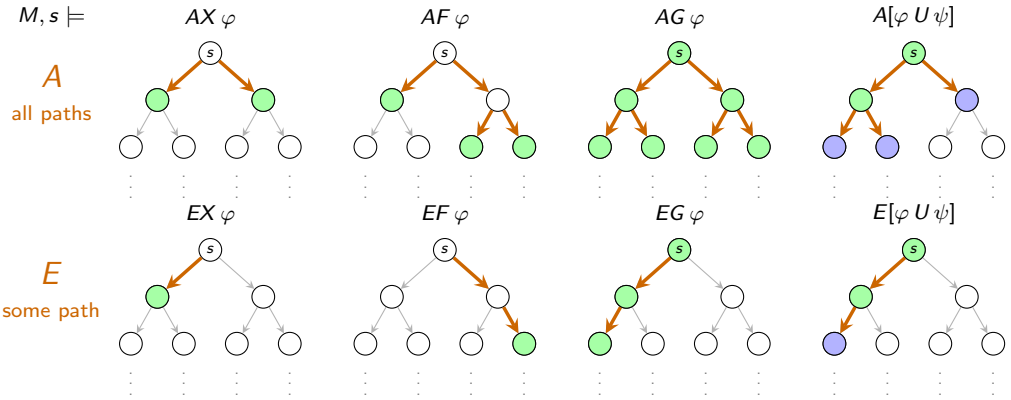
Recap: CTL

$\varphi ::= \bot \mid \top \mid p \mid (\neg\varphi) \mid (\varphi \vee \varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \to \varphi) \mid$
$AX\varphi \mid EX\varphi \mid AF\varphi \mid EF\varphi \mid AG\varphi \mid EG\varphi \mid A[\varphi\ U\ \varphi] \mid E[\varphi\ U\ \varphi]$
where $p \in AP$ are atomic propositions.

- A CTL formula is evaluated at a *state* s of a model M: $M, s \models \varphi$.
- Every temporal operator (X, F, G, U) has a *path quantifier* in front: A (*all* paths from s) or E (*some* path from s).

	X (next)	F (eventually)	G (always)	U (until)
A: all paths	$AX\,\varphi$ φ at every next state	$AF\,\varphi$ every path reaches φ	$AG\,\varphi$ on every path, φ throughout	$A[\varphi\,U\,\psi]$ on every path, φ until ψ
E: some path	$EX\,\varphi$ φ at some next state	$EF\,\varphi$ some path reaches φ	$EG\,\varphi$ some path with φ throughout	$E[\varphi\,U\,\psi]$ some path with φ until ψ

Recap: CTL semantics on a computation tree

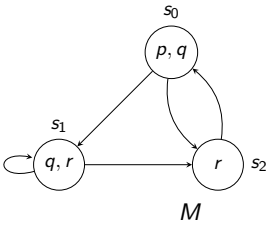


φ-state ψ-state does not matter the path(s) the formula talks about: all branches for A, one for E

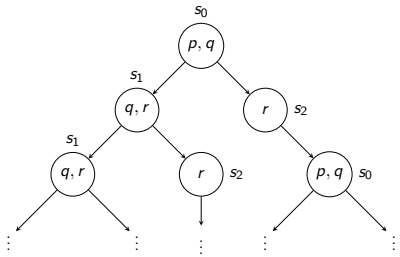
Atoms: $M, s \models p$ iff $p \in L(s)$; $\neg, \wedge, \vee, \rightarrow$ are as usual.

Recall example M from before...

From any given state, we can unwind the system as an infinite tree of computation paths.



M



unwinding of M from s_0

Exercise 24.11 Which of the following hold? Why or why not? Justify!

1. $M, s_0 \models (p \wedge q) \wedge \neg r$ ✓
2. $M, s_0 \models EX(q \wedge r)$ ✓
3. $M, s_0 \models AX(q \wedge r)$ ×
4. $M, s_0 \models \neg EF(p \wedge r)$ ✓
5. $M, s_1 \models EG(q \wedge r)$ ✓
6. $M, s_0 \models AF\ r$ ✓
7. $M, s_0 \models E[(p \wedge q)\ U\ (q \wedge r)]$ ✓
8. $M, s_0 \models A[(p \wedge q)\ U\ (q \wedge r)]$ ×

Is $\neg EF\,\varphi$ "the same as" $AG\,\neg\varphi$? Is $\neg AX\,\varphi$ equivalent to $EX\,\neg\varphi$? And so on...

Proving equivalence in CTL

Same idea as for LTL, now on states

Two CTL formulas φ, ψ are equivalent, $\varphi \equiv \psi$, if for all M, s, $M, s \models \varphi$ iff $M, s \models \psi$.

Claim: $\neg AX\varphi \equiv EX\neg\varphi$. Proof for an arbitrary model M and state s of M:

	$M, s \models \neg AX\varphi$	
$\iff$	$M, s \not\models AX\varphi$	semantics of $\neg$
$\iff$	not (for all s_1 with $s \to s_1$: $M, s_1 \models \varphi$)	semantics of AX
$\iff$	for some s_1 with $s \to s_1$: $M, s_1 \not\models \varphi$	negating $\forall$
$\iff$	for some s_1 with $s \to s_1$: $M, s_1 \models \neg\varphi$	semantics of $\neg$
$\iff$	$M, s \models EX\neg\varphi$	semantics of EX

Equivalences and Adequate sets

$$\varphi ::= \bot \mid \top \mid p \mid (\neg\varphi) \mid (\varphi \vee \varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \to \varphi) \mid$$
$$AX\varphi \mid EX\varphi \mid AF\varphi \mid EF\varphi \mid AG\varphi \mid EG\varphi \mid A[\varphi \ U \ \varphi] \mid E[\varphi \ U \ \varphi]$$

Exercise 25.1 (PS10) Check the following equivalences

1. $\neg AF\varphi \equiv EG\neg\varphi$
2. $\neg EF\varphi \equiv AG\neg\varphi$
3. $\neg AX\varphi \equiv EX\neg\varphi$ ✓
4. $AF\varphi \equiv A[\top \ U \ \varphi]$
5. $EF\varphi \equiv E[\top \ U \ \varphi]$
6. $AF\varphi \equiv \varphi \vee AX \ AF\varphi$

From the above equivalences (1–5), it follows that

AU, EU, EX are a universal/adequate set of CTL temporal connectives

Exercise 25.2. Show that $\{EG, EU, EX\}$ is also adequate. Are there yet other sets?

We can also define Weak-until and Release, AR, ER, AW, EW in CTL as for LTL.

Some practical examples of CTL specifications

Take atomic propositions: **busy**, **ready**, **started**, **req**, **grant**, **enabled**, **running**, etc.

1. It is possible to reach a state where **started** holds but **ready** does not.
   $EF($ **started** $\wedge \neg$ **ready**$)$
2. In any state, if there is a request (of some resource), then it will be eventually granted.
   $AG($**req** $\rightarrow AF$ **grant**$)$. recall the LTL formula! $G($**req** $\rightarrow F$ **grant**$)$
3. If a process is enabled infinitely often, then it runs infinitely often. Expressible in CTL??
   Recall the LTL formula for this. What does $AG\ AF$ **enabled** $\rightarrow AG\ AF$ **running** do?
   Weaker: it only says that if all paths have enabled infinitely often, then all have running infinitely often.
4. From any state we reach, we can get to some ready state. $AG\ EF$ **ready**. Can this be expressed in LTL?

Expressiveness between CTL and LTL

- Some properties are expressible both in LTL and CTL.
  - E.g., every request is eventually granted: CTL $AG(p \rightarrow AFq)$, $G(p \rightarrow Fq)$.

Theorem: CTL and LTL are expressively incomparable

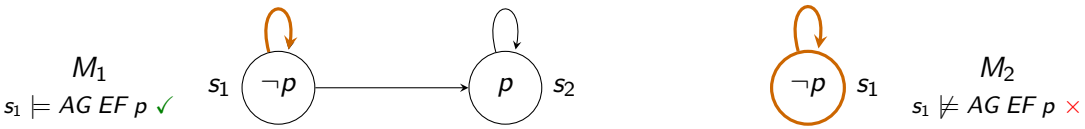
1. There exist properties that are expressible in CTL but not LTL.
   - e.g., from any state we can get to a ready state: CTL $AG\ EFp$.
   - Not expressible in LTL. Intuitively, it cannot be reduced to a property of a path or all paths.
2. There exist properties that are expressible in LTL but not CTL.
   - e.g., if p occurs infinitely often, then so does q: LTL $GFp \rightarrow GFq$.
   - Not expressible in CTL. Intuitively, we need to talk about that very path, so it's a path property!

Finally, are there properties that are expressible neither in LTL nor in CTL? There exists a path with infinitely many p's!

- Question: How do we prove these inexpressibility results formally?
- This is often difficult (and fun!). But there are techniques...

Formally proving Inexpressibility results: An example!

- Given an LTL formula ψ and a CTL formula φ, we say $\psi \equiv \varphi$ if for all M, s, $M, s \models \psi$ iff $M, s \models \varphi$. For example, $AG(p \rightarrow AFq) \equiv G(p \rightarrow Fq)$.



Theorem: There exists no LTL formula equivalent to $\varphi = (AG\ EF\ p)$.

Proof: Suppose not, let ψ be an LTL formula s.t, $\psi \equiv \varphi$, i.e., $\forall M, s$, $M, s \models \psi$ iff $M, s \models \varphi$.

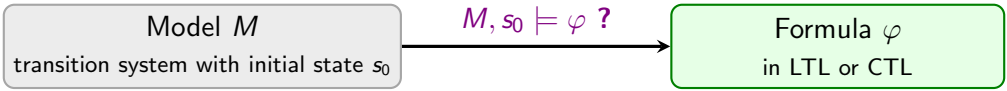
- Consider M_1, s_1. $M_1, s_1 \models (AG\ EF\ p)$ so $M_1, s_1 \models \psi$.
- By definition of satisfaction in LTL, for all paths starting at s_1 in M_1, LTL formula ψ holds.
- Consider M_2: paths from s_1 in M_2 are a subset of paths from s_1 in M_1, hence $M_2, s_1 \models \psi$.
- But $M_2, s_1 \not\models \varphi$, hence contradiction.

Exercise 25.3. Is $(FGp) \equiv (AF\ AG\ p)$? Exhibit a model that distinguishes them.

Topic 25.1

Model checking

The model checking problem



Basic Model checking Question

Given a formula φ in LTL or CTL, and given a transition system/model M with an initial state s_0: is $M, s_0 \models \varphi$?

- LTL model checking beyond the scope of this course. Great for reading further!
- CTL model checking, though, has a simple polynomial algorithm. Coming up!

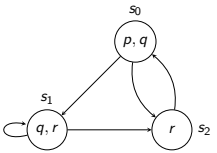
In fact, we will show something slightly stronger.

Given a CTL formula and model, we will output the set of all states that satisfy the formula!

A labelling algo for CTL-model checking

Input: CTL formula φ and model $M = (S, \to, L)$
Output: Set of states of M that satisfy φ.

- Convert φ into a CTL formula φ' that only uses $EG, EU, EX, \wedge, \neg, \bot$.
- Idea: Label states of M with all subformulas of φ' that are satisfied there, smallest subformulas first.
- Take the next subformula ψ (all its proper subformulas are already labelled):
  - $\psi = \bot$. Nothing to do.
  - $\psi = p$. Label s with p if $p \in L(s)$.
  - $\psi = \neg\psi'$. Label s with $\neg\psi'$ unless it is already labelled ψ'.
  - $\psi = \psi_1 \wedge \psi_2$. Label s with ψ if it is already labelled with ψ_1 and ψ_2.
  - $\psi = EX\psi_1$. Label s with $EX\,\psi_1$ if some successor of s is labelled ψ_1.



Example: $\varphi = \neg p \wedge EX(q \wedge r)$ (so $\varphi' = \varphi$)

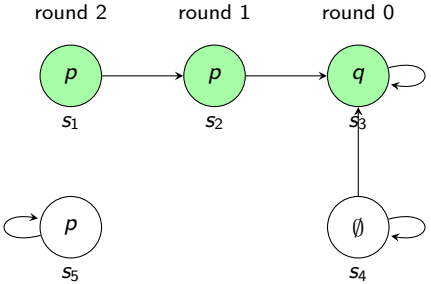
	s_0	s_1	s_2
p	✓		
q	✓	✓	
r		✓	✓
$\neg p$		✓	✓
$q \wedge r$		✓	
$EX(q \wedge r)$	✓	✓	
φ		✓	

Output: $\{s_1\}$, so $M, s_0 \not\models \varphi$

A labelling algo for CTL-model checking (contd.)

- Case $\psi = E[\psi_1\ U\ \psi_2]$. If any s is labelled ψ_2, label it with ψ. Repeat till fixpoint: label any state with ψ if it is labelled ψ_1 and has a successor labelled ψ.
- Last case: $\psi = EG\,\psi_1$: next slide.

Example: $\psi = E[p\ U\ q]$, so $\psi_1 = p$ and $\psi_2 = q$



Round 3 adds nothing: output $\{s_1, s_2, s_3\}$

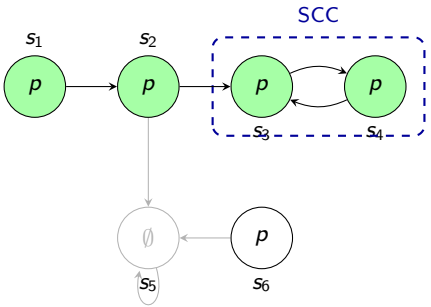
s_4 is not p; s_5 is p but never reaches q

A labelling algo for CTL-model checking: the EG case

- Last case: $\psi = EG\psi_1$.
  - Consider the subgraph consisting of all states labelled ψ_1.
  - In this, find maximum strongly connected components (MSCCs)[1].
  - Use backwards breadth-first search (BFS) on this subgraph to find all states that can reach such an MSCC. Label each such state with ψ.

This completes the algo. At end we check if initial state is labelled by input CTL formula φ'.

Example: $\psi = EG\, p$, so $\psi_1 = p$



Output: $\{s_1, s_2, s_3, s_4\}$

s_6 is p, but its only successor s_5 is not: it cannot stay in p forever

[1] A set of states is *strongly connected* if every state in it has a path to every other state in it. A maximal such set is an MSCC; we only need those with at least one edge (a single state counts only if it has a self-loop).

CS228 : Logic for Computer Science

Module 5: Temporal Logic

Lecture 26: Wrap-up, Summary, and a NuSMV Demo

Instructor: S. Akshay

IIT Bombay, India

Logistics

Course feedback

- **8am pop quiz!** Your task: **fill in the course feedback form on ASC** (deadline: **today**!)
- A second form, with questions about the TAs, will be sent out separately.

End-semester exam

- **Monday, Nov 16th, 9am**

The course in numbers, and a challenge

- 26 lectures (Lecture 0 and Lectures 1–25)
- 10 problem sheets
- In addition, we will release a tough problem sheet with advanced problems, as a challenge
- Try it *only after* you have solved the regular problem sheets!

Topic 26.1

Wrap-up and Summary

Recap: the labelling algorithm for CTL model checking

- *Input:* a CTL formula φ and a model M. *Output:* the set of all states of M satisfying φ.
- Rewrite φ using only $EX, EU, EG, \wedge, \neg, \bot$. Then label every state with the subformulas it satisfies, smallest first.
- The cases p, $\neg\psi'$ and $\psi_1 \wedge \psi_2$ are immediate. The real work is in three cases:

$EX\ \psi_1$

Label s if some successor of s is labelled ψ_1.

$E[\psi_1\ U\ \psi_2]$

Start with the ψ_2-states. Repeat till fixpoint: also label a ψ_1-state that has a successor already labelled ψ.

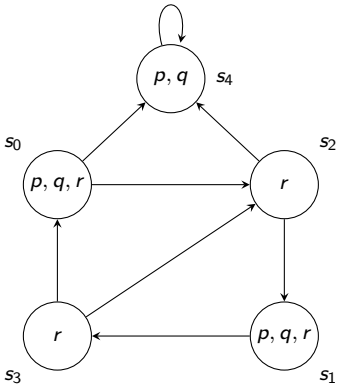
$EG\ \psi_1$

Keep only the ψ_1-states, find the strongly connected components that contain a cycle, and label every state that can reach one (backwards BFS).

Quick recall: which of the three cases needs strongly connected components, and why?

EG: an infinite path that stays among the ψ_1-states must end up going round a cycle forever.

Exercise 26.1: CTL model checking by hand



M, with initial state s_0

Let $\varphi = EG\, p \wedge E[(\neg q \vee r)\ U\ AX\, r]$, and let M be the model on the left.

(a) **Rewrite** φ as an equivalent formula φ' that uses only EX, EU, EG, $\wedge$, $\neg$ and $\bot$.

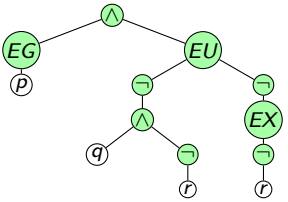
(b) **Model check:** run the labelling algorithm on M with φ', labelling the states with every subformula of φ', smallest first. Which states satisfy φ? Does $M, s_0 \models \varphi$?

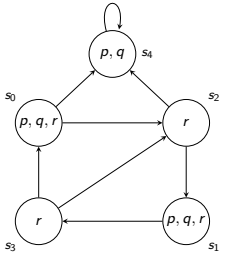
Solution to Exercise 26.1

(a) $\neg q \vee r \;\equiv\; \neg(q \wedge \neg r)$

$AX\, r \;\equiv\; \neg EX \neg r$

so $\varphi' \;=\; EG\, p \wedge E[\neg(q \wedge \neg r)\; U\; \neg EX \neg r]$





(b)	s_0	s_1	s_2	s_3	s_4
$EG\, p$	✓				✓
$\neg r$					✓
$q \wedge \neg r$					✓
$\neg(q \wedge \neg r)$	✓	✓	✓	✓	
$EX \neg r$	✓		✓		✓
$\neg EX \neg r$		✓		✓	
$E[\neg(q \wedge \neg r)\; U\; \neg EX \neg r]$	2	0	1	0	
φ'	✓				

Output: $\{s_0\}$, so $M, s_0 \models \varphi$

Step 8: φ'

- labelled $EG\, p$ and the U formula: s_0

Complexity of the labelling algorithm

What is the complexity of this algo?

- All the cases except EG are clearly polynomial time.
- EG needs strongly connected components (SCCs): this is just graph reachability, so a (backwards) BFS can do it, but carefully
  - never pass over a node twice: this takes linear time, $O(V + E)$
- Altogether: $O(N \cdot (V + E))$, where N is the number of connectives in φ', V the number of states and E the number of transitions.

But the model itself can be huge: the state explosion problem

- M is often exponential in the number of variables and of parallel components: each extra boolean variable can double the number of states ($2^{30} \approx 10^9$)

In reality, we do symbolic model checking

Use symbolic states (binary decision diagrams) instead of explicit states!

Out of syllabus, but read Section 6.3 from Huth & Ryan!

Closing remarks

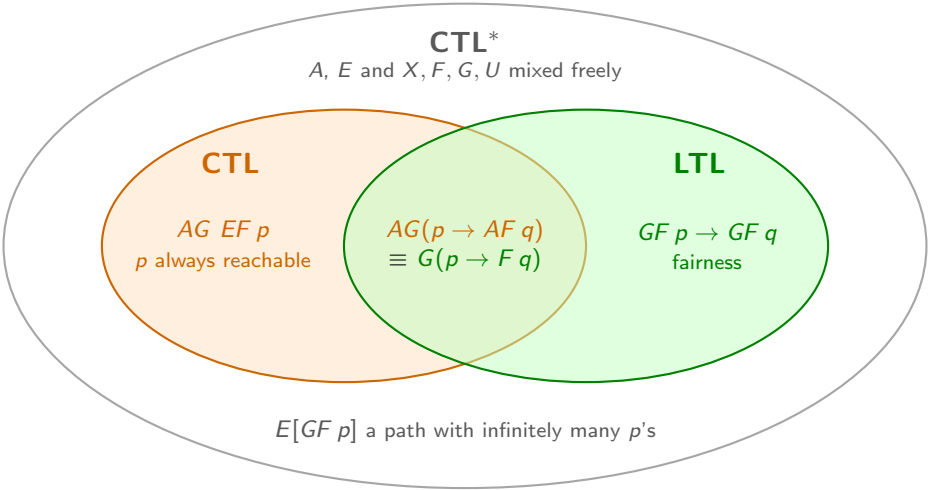
Model checking algo for LTL

- We have to reason on infinite paths not states!
- Complexity goes from PTIME (for CTL) to (believed to be) more than PTIME (for LTL). How much more?

Combining the power of CTL and LTL $\longrightarrow$ CTL^*

- Drop the CTL requirement that every temporal operator must have A/E in front!
- E.g., we can write $A[X\ p \vee X\ X\ p]$ or $E[G\ F\ p]$.
- **Exercise 26.2**
  1. Show that LTL and CTL are strict subsets of CTL^*.
  2. But is it more than their union? Prove or disprove: there exists φ in CTL^* which cannot be expressed in LTL or CTL!

Expressive power: CTL, LTL and CTL*



Summary

Linear-time → LTL

- evaluated on *paths*; $M, s \models \varphi$ means *all* paths from s satisfy φ
- connectives X, F, G, U, e.g., $G(\textbf{req} \rightarrow F\ \textbf{grant})$
- F, G are duals, but $F(\varphi \wedge \psi) \not\equiv F\varphi \wedge F\psi$
- adequate set: $\{\top, \neg, \wedge, X, U\}$

Branching-time → CTL

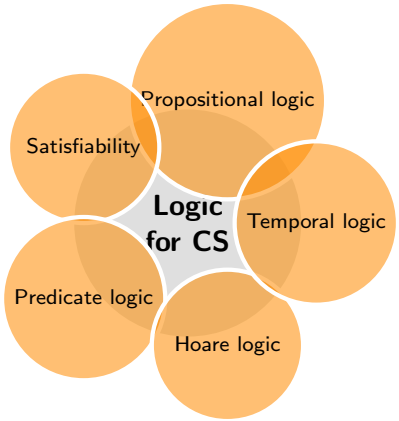
- LTL cannot assert that a path *exists*: add quantifiers A (all paths) and E (some path)
- each temporal connective comes with one, e.g., $AG(\textbf{req} \rightarrow AF\ \textbf{grant})$, $AG\ EF$ **ready**
- evaluated on *states* of the transition system
- adequate set: $\{AU, EU, EX\}$

Model checking

- does $M, s_0 \models \varphi$ hold? For CTL we compute *all* states satisfying φ
- CTL: rewrite with EX, EU, EG, then label states with subformulas, bottom-up; time $O(N \cdot (V + E))$ (N connectives, V states, E transitions)
- LTL: infinite paths; believed to be more than PTIME, while CTL is in PTIME

Neither subsumes the other: $AG\ EF\ p$ is CTL only, $GF\ p \rightarrow GF\ q$ is LTL only

Summary of the course



Propositional logic
- syntax and semantics
- natural deduction
- normal forms

logic puzzles, checking arguments

Satisfiability
- resolution
- DPLL and CDCL
- SAT solver demo

HW/SW verification, planning

Predicate logic
- syntax and semantics
- natural deduction
- limits of FOL, DPLL(T)

databases, SMT solvers

Hoare logic
- triples and proof rules
- invariants and variants
- total correctness

verifying code

Temporal logic
- LTL and CTL
- expressive power
- CTL model checking

reactive systems (OS, lift controller, pacemaker)

Logic: the language and the calculus for reasoning

Topic 26.2

Model checking demo (not in syllabus)